Regresión, Sesgo-Varianza, Selección de Modelos I_Martinez_Fernando

February 16, 2026

1 Práctica: Regresión, Sesgo-Varianza, Selección de Modelos I

Alumno: Fernando Martínez Marco
Asignatura: Métodos de Aprendizaje Automático

1.1 Introducción

Los métodos paramétricos asumen que los datos provienen de un modelo con una estrutura fije, en nuestro caso un polinomio, y el objetivo es encontrar los parámetros que mejor se ajusten a los datos. En esta práctica, usaremos los Mínimos Cuadrados Ordinarios (OSL) para estimar esos parámetros o coeficientes del polinomio.

No es suficiente con ajustar bien los parámetros a los datos de entrenamiento, ya que, un grado bajo del polinomio limita la capacidad del estimador g(x), generando sesgo, y un grado alto del polinomio permite que el estimador g(x) siga demasiado de cerca las fluctuaciones de los datos, generando varienza.

En esta práctica vamos a estudiar los posibles modelos de regresión polinómica, en base a las métricas estudiadas los los capítulos 4.6, 4.7 y 4.8 de Ethem Alpaydin y realizaramos un análisis sesgo/varianza mediante la comparación error de entrenamiento y de validación, para determinar la complejidad óptima de un estimador.

Calcularemos el MSE para distintos grados para observar cómo el error de validación refleja este dilema.

Necesitamos seleccionar modelos para garantizar que éste tenga una buena capacidad de generalización ante datos nuevos. Debemos tener un control del grado del polinomio, o complejidad del modelo, ya que si es demasiado simple con grado bajo, habrá un sesgo alto y se producirá underfitting, y el modelo no captura la complejidad de los datos. Sin embargo, si el modelo es demasiado complejo, grado alto, tendremos varianza alta, habrá overfitting, el modelo aprende el ruido, y no la tendencia genearal.

De esta forma, para encontrar el modelo o grado óptimo, buscamos el punto donde se minimice el riesgo esperado o error total, es decir, donde la reducción del sesgo deja de compensar el incremento de la varianza, que en la grafica de sesgo-varianza se denomina como el "codo" de la curva de Riesgo Total.

El error cuadrático medio total se descompone en tres componentes:

$$MSE \approx E[(r - g(x))^2] = \underbrace{\sigma^2}_{\text{Ruido}} + \underbrace{(f(x) - E[g(x)])^2}_{\text{Sesgo}^2} + \underbrace{E[(g(x) - E[g(x)])^2]}_{\text{Varianza}}$$

Donde:

- Ruido Irreducible (σ^2): Representa la varianza del ruido intrínseco de los datos que no puede ser eliminado por ningún estimador.
- Sesgo (Sesgo^2): Mide cuánto se aleja el promedio de nuestros modelos de la función verdadera; un sesgo alto es indicativo de underfitting.
- Varianza: Mide cuánto fluctúan las predicciones del modelo al cambiar el conjunto de entrenamiento; una varianza alta indica overfitting.

El MSE de validación en las gráficas, estamos buscando el punto donde la suma de estos tres componentes es mínima. Al calcular el MSE (Mean Squared Error) en la validación cruzada, obtenemos una estimación de ese valor.

El riesgo esperado en un punto x es el promedio teórico del error que cometería el modelo sobre todas las posibles muestras que se podrían extraer de los datos, y se define como:

$$E[(r - g(x))^2] = Ruido + Sesgo^2 + Varianza$$

El MSE es la métrica que utilizamos para aproximar ese riesgo esperado utilizando los datos disponibles, lo calculamo mediante Cross-Validation, y es nuestro estimador del riesgo de generalización. Buscamos minimizar el "error total", es decir, minimizar la suma del sesgo y la varianza (más el ruido irreducible).

Con la Validación Cruzada de 10 pliegues (10-Fold Cross Validation), evaluamos el rendimiento en datos que el modelo no ha visto durante el ajuste. Esto nos permite estimar el error de validación y seleccionar la complejidad que tiene el mejor equilibrio para predecir futuros valores.

Con este procedimiento, dividimos el conjunto de datos total en 10 subconjuntos de tamaño aproximadamente igual:

1. El algoritmo realiza 10 iteraciones independientes.
2. En cada iteración, se selecciona uno de los pliegues (1/10 de los datos) como conjunto de validación.
3. Los 9 pliegues restantes (9/10 de los datos) se utilizan como conjunto de entrenamiento.
4. Se entrena el modelo, con la regresión polinómica de grado k, y se calcula el error (MSE) en el pliegue de validación.

Una vez completadas las 10 iteraciones, el Error Cuadrático Medio total para un grado k específico se calcula como el promedio de los errores obtenidos en cada una de las 10 validaciones:

$$MSE_{CV} = \frac{1}{10} \sum_{i=1}^{10} MSE_i$$

Este promedio es el que se utiliza en la gráfica de Riesgo Total para identificar el punto óptimo (el "codo" de la curva). Usamos los 10 pliegues (fold) para que el error no dependa de que datos cayeron por azar en el grupo de validación. Cambiando los pliegues, aseguramos que cada dato

sea evaluado una vez. Al usar el 90% de los datos en cada iteración, el modelo tiene sufuciente información para aprender los parámetros. Con el método 10-Fold CV, tenemos una medida más fiable de la varianza del modelo, y podemos ver si el error cambia entre pliegues, que es señal de overfitting.

Se ha aplicado normalización lineal a los datos, transformando X al intervalo $[-1, 1]$, ya que para el cálculo de potencias como X^{15} se producirían valores muy dispares y sería dificil de invertir la matríz de diseño o inestable para el algortimo de Mínimos Cuadrados.

Para garantizar un análisis riguroso y estable, resolvemos la regresión polinómica mediante la **Ecuación Normal**:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Sí, puedes decirlo con total propiedad. En esa ecuación, la matriz $\mathbf{X}$ es precisamente la **Matriz de Diseño**.

De hecho, para que sea técnicamente exacto en tu trabajo, puedes explicarlo de la siguiente forma:

1. **Definición**: La matriz $\mathbf{X}$ (que en tu código generas con la función `get_D`) es la Matriz de Diseño. Se encarga de transformar el vector de entrada original en una estructura donde cada columna representa una potencia del polinomio, desde x^0 hasta x^k.
2. **Función en la Ecuación Normal**: La matriz de diseño permite expresar el problema de regresión polinómica como un sistema lineal. Al aplicar $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$, lo que estás haciendo es encontrar el vector de pesos $\mathbf{w}$ (los coeficientes del polinomio) que minimiza la suma de los errores al cuadrado.
3. **Relación con Statsmodels**: Aunque en tu código llamas a `sm.OLS(y, X_train_D).fit()`, por debajo, la librería está resolviendo esa misma Ecuación Normal utilizando la matriz de diseño que tú le pasas.

Para que estabilidad numérica** del estudio, especialmente al trabajar con potencias elevadas de $k = 15$, se aplica una normalización lineal de los datos de entrada. Como se observa en el código, transformamos X al intervalo $[-1, 1]$. Esto es fundamental porque, sin esta escala, el cálculo de potencias como X^{15} produciría valores extremadamente dispares que harían que la matriz de diseño fuera difícil de invertir o numéricamente inestable para el algoritmo de Mínimos Cuadrados.

La estimación de los errores se apoya en el proceso de **10-Fold Cross-Validation**, donde el script baraja los índices aleatoriamente (`np.random.shuffle`) y divide los datos en pliegues (`np.array_split`). Este enfoque garantiza que cada punto del dataset actúe como validación, permitiéndonos obtener una curva de error total fiable para localizar el "codo" donde el sesgo y la varianza se equilibran.

Para el ajuste de los modelos, se emplea la librería **`statsmodels` (sm.OLS)**. La elección de esta herramienta es clave para el análisis estadístico, ya que nos permite extraer los **p-values** de cada coeficiente del polinomio. En el código, calculamos el `PValMean` para evaluar la significancia media del modelo. Un p-value alto en los términos de grado superior es una prueba estadística de la varianza: nos indica que el modelo está asignando importancia a términos que probablemente solo están ajustando ruido, lo cual complementa visualmente lo que observamos en las gráficas de entrenamiento y validación.

[3]:
```
import numpy as np
import pandas as pd
```

```python
import matplotlib.pyplot as plt
import statsmodels.api as sm
from IPython.display import display, Markdown

plt.rcParams.update({
    'figure.figsize': (10, 6),
    'figure.facecolor': 'white',
    'axes.facecolor': 'white',
    'font.size': 12,
    'lines.linewidth': 2,
    'figure.autolayout': True
})
```

1.2 Análisis del dataset_1.csv

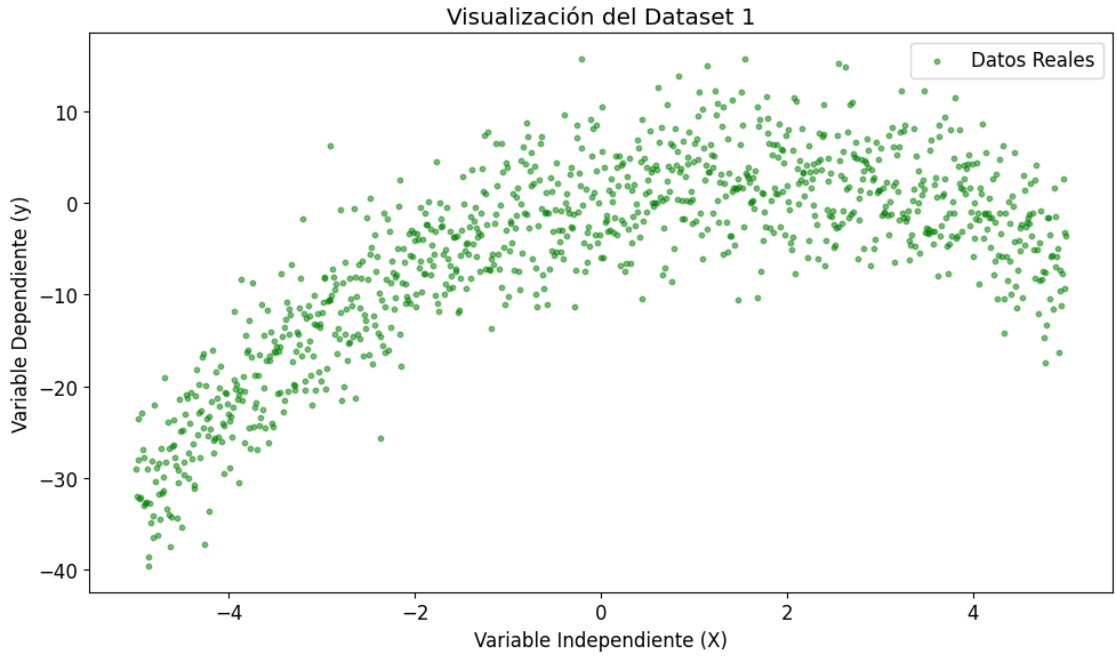
1.2.1 Paso 1: Carga y Exploración

Cargamos los datos y realizamos una visualización inicial para observar la dispersión de los puntos e intentar intuir qué tipo de función podría describirlos (lineal, cuadrática, etc.).

[4]:
```python
df = pd.read_csv('dataset_1.csv').sort_values(by='X')

# Extraemos las variables
X_raw = df['X'].values
y = df['y'].values

print(f"El dataset_1 tiene {len(df)} muestras.")
display(df.head())

# Visualización inicial
plt.scatter(X_raw, y, s=10, color='green', alpha=0.5, label='Datos Reales')
plt.title("Visualización del Dataset 1")
plt.xlabel("Variable Independiente (X)")
plt.ylabel("Variable Dependiente (y)")
plt.legend()
plt.show()
```

```
El dataset_1 tiene 1000 muestras.
```

```
         X          y
0 -5.00000 -29.016429
1 -4.98999 -32.086297
2 -4.97998 -28.051668
3 -4.96997 -23.570257
4 -4.95996 -32.251629
```



1.3 Paso 2: Normalización de los Datos

Cuando trabajamos con polinomios de grado alto (como X^{15}), los valores pueden hacerse extremadamente grandes, lo que genera inestabilidad numérica al invertir matrices.

Normalizamos X al rango $[-1, 1]$ usando la fórmula:

$$X_{norm} = 2 \cdot \frac{X - X_{min}}{X_{max} - X_{min}} - 1$$

[5]:
```
X_min, X_max = X_raw.min(), X_raw.max()
X_norm = 2 * (X_raw - X_min) / (X_max - X_min) - 1

print("Rango original de X:", X_min, "a", X_max)
print("Rango normalizado de X:", X_norm.min(), "a", X_norm.max())
```

```
Rango original de X: -5.0 a 5.0
Rango normalizado de X: -1.0 a 1.0
```

1.4 Paso 3: Creación de la Matriz de Diseño

Para ajustar un polinomio de grado k, necesitamos convertir nuestro vector X en una matriz donde cada columna sea una potencia de X:

$$D = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ 1 & x_2 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{bmatrix}$$

[6]:
```python
def get_design_matrix(X, degree):
    """Genera la matriz de diseño con potencias desde 0 hasta degree."""
    columns = [X**i for i in range(degree + 1)]
    return np.column_stack(columns)

# Ejemplo para grado 2
D_example = get_design_matrix(X_norm[:5], 2)
print("Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):")
print(D_example)
```

```
Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):
[[ 1.         -1.          1.        ]
 [ 1.         -0.997998    0.996     ]
 [ 1.         -0.995996    0.99200802]
 [ 1.         -0.99399399  0.98802406]
 [ 1.         -0.99199199  0.98404811]]
```

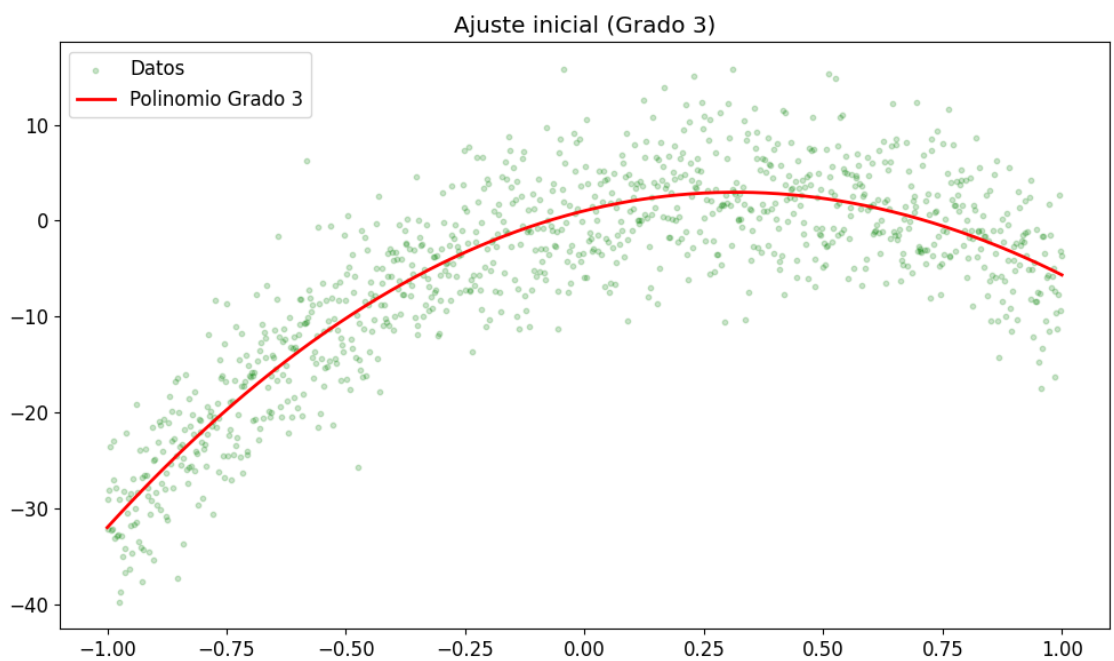
1.5 Paso 4: Ajuste de un Modelo Individual

Vamos a ver cómo funciona el ajuste usando `statsmodels`. Esto nos dará los coeficientes del polinomio y su significancia estadística (p-values).

[7]:
```python
grado_prueba = 3
D_3 = get_design_matrix(X_norm, grado_prueba)

# Ajustamos el modelo de Mínimos Cuadrados Ordinarios (OLS - Ordinary Least␣
↪Squares), de statsmodels
modelo = sm.OLS(y, D_3).fit()

print(modelo.summary())

# Predicciones
y_pred = modelo.predict(D_3)

# Visualizar el primer ajuste
plt.scatter(X_norm, y, s=10, color='green', alpha=0.2
            , label='Datos')
plt.plot(X_norm, y_pred, color='red', label=f'Polinomio Grado {grado_prueba}')
plt.title("Ajuste inicial (Grado 3)")
plt.legend()
plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.790
Model:                            OLS   Adj. R-squared:                  0.789
Method:                 Least Squares   F-statistic:                     1248.
Date:                Mon, 16 Feb 2026   Prob (F-statistic):               0.00
```

```
Time:                        20:22:01   Log-Likelihood:                -3005.8
No. Observations:                1000   AIC:                             6020.
Df Residuals:                     996   BIC:                             6039.
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.0395      0.232      4.474      0.000       0.584       1.495
x1            12.2752      0.670     18.320      0.000      10.960      13.590
x2           -19.8288      0.519    -38.242      0.000     -20.846     -18.811
x3             0.8737      1.021      0.855      0.393      -1.131       2.878
==============================================================================
Omnibus:                        2.623   Durbin-Watson:                   2.019
Prob(Omnibus):                  0.269   Jarque-Bera (JB):                2.495
Skew:                           0.116   Prob(JB):                        0.287
Kurtosis:                       3.076   Cond. No.                         8.20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```



En este apartado, lo que se está haciendo es un ajuste inicial de prueba:

1. Construcción de la Matriz de Diseño (D_3)

grado_prueba = 3: Eligimos un grado bajo (cúbico) para ver cómo se comporta un modelo sencillo. get_design_matrix(X_norm, 3): Transforma el vector de datos X en una matriz donde cada columna es una potencia (Ej: $[1, X, X^2, X^3]$). Esto es lo que permite que una regresión lineal encuentre una solución no lineal (polinómica).

3. Ajuste del Modelo (sm.OLS)

sm.OLS(y, D_3).fit(): Se utiliza la librería statsmodels para resolver la ecuación normal $(D^T D)^{-1} D^T y$. Encuentra los coeficientes (β) que minimizan la suma de los errores al cuadrado. Objeto modelo: No solo guarda los coeficientes, sino toda la inferencia estadística del ajuste.

5. Diagnóstico Estadístico (print(modelo.summary()))

Se obtiene:

R^2: Qué tanto de la varianza del dato explica el polinomio de grado 3. P-values: Nos dice si cada término (por ejemplo, el X^3) es realmente necesario o si su coeficiente podría ser cero (es decir, no es significativo).

F-statistic: Evalúa si el modelo en su conjunto es mejor que simplemente tirar una línea horizontal por la media.

7. Predicción y Visualización

modelo.predict(D_3): Calcula los valores de de la variable dependiente según el modelo ajustado en esos mismos puntos.

plt.scatter y plt.plot: Superpone la curva roja (el modelo) sobre los puntos verdes (los datos reales).

Si los puntos verdes tienen una forma de "S" y la línea roja los sigue bien, el grado 3 es razonable.

Si el dato es más complejo, veríamos que la línea roja no alcanza a curvarse lo suficiente, lo que indica sesgo alto (subajuste).

1.6 Paso 5: Validación Cruzada (10-Fold CV)

Para elegir el mejor grado, usamos 10-Fold Cross-Validation:

1. Dividimos los datos en 10 grupos (folds).
2. Entrenamos con 9 grupos y probamos con el grupo restante.
3. Repetimos esto 10 veces y promediamos el error.

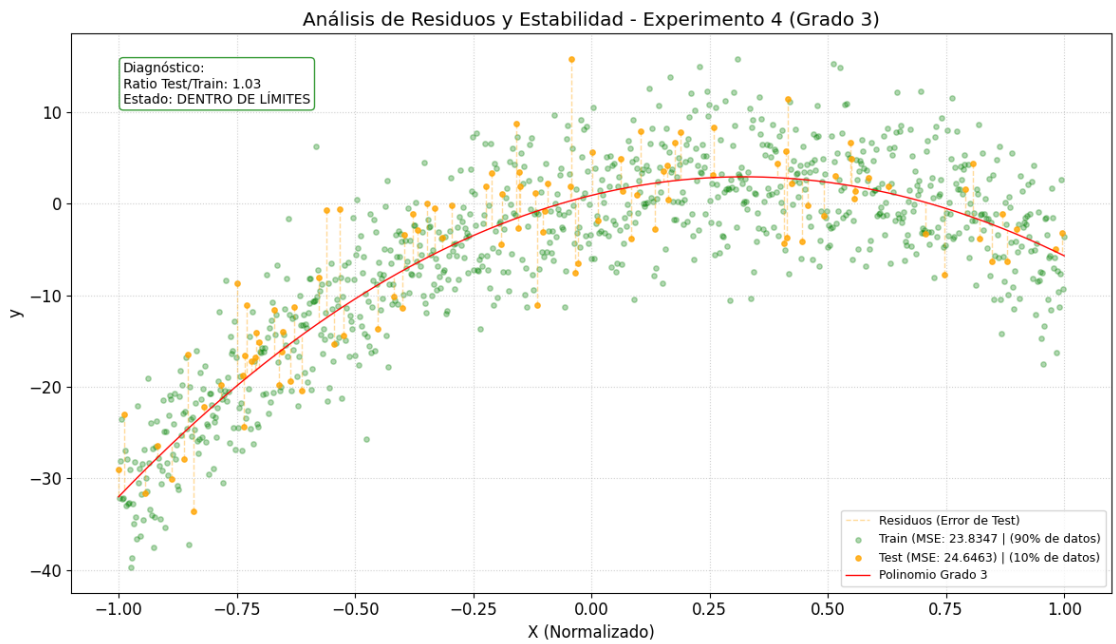
Esto nos da el Riesgo Esperado (Error de Validación).

[8]:
```
np.random.seed(42)

indices = np.arange(len(X_norm))
np.random.shuffle(indices)

# Dividimos en 10 partes
folds = np.array_split(indices, 10)

print(f"Se han creado 10 folds de aproximadamente {len(folds[0])} muestras cada␣
↪uno.")
```

Se han creado 10 folds de aproximadamente 100 muestras cada uno.

[9]:
```
# --- VISUALIZACIÓN CON ANÁLISIS DE ERROR Y RESIDUOS (Fold 4, Grado 3) ---

f = 3
grado_vis = 3
idx_test = folds[f]
idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

# 1. Ajuste del modelo
D_train = get_design_matrix(X_norm[idx_train], grado_vis)
modelo_ejemplo = sm.OLS(y[idx_train], D_train).fit()

# 2. Cálculo de errores (MSE)
y_train_pred = modelo_ejemplo.predict(D_train)
mse_train = np.mean((y[idx_train] - y_train_pred)**2)

D_test = get_design_matrix(X_norm[idx_test], grado_vis)
y_test_pred = modelo_ejemplo.predict(D_test)
mse_test = np.mean((y[idx_test] - y_test_pred)**2)

# 3. Verificación de límites aceptables
# Un modelo se considera inestable si el error de test supera␣
↪significativamente al de entrenamiento
ratio_error = mse_test / mse_train
es_aceptable = ratio_error < 1.5  # Tolerancia del 50% de incremento

plt.figure(figsize=(12, 7))

# Visualizar Residuos (Líneas de error para los puntos de Test)
plt.vlines(X_norm[idx_test], y[idx_test], y_test_pred,
           color='orange', lw=1.0, linestyle='--', alpha=0.4, label='Residuos␣
↪(Error de Test)')

# Puntos de entrenamiento y test
plt.scatter(X_norm[idx_train], y[idx_train], s=15, color='green', alpha=0.3,␣
↪label=f'Train (MSE: {mse_train:.4f} | (90% de datos)')
plt.scatter(X_norm[idx_test], y[idx_test], s=15, color='orange', alpha=0.8,␣
↪label=f'Test (MSE: {mse_test:.4f}) | (10% de datos)')

# Curva del modelo
x_curve = np.linspace(-1, 1, 500)
y_curve = modelo_ejemplo.predict(get_design_matrix(x_curve, grado_vis))
plt.plot(x_curve, y_curve, color='red', lw=1.0, label=f'Polinomio Grado␣
↪{grado_vis}')

# Panel de diagnóstico (Cuadro de texto)
```

```
status_color = "green" if es_aceptable else "red"
status_text = "DENTRO DE LÍMITES" if es_aceptable else "SOBREAJUSTE DETECTADO"
info_box = (f"Diagnóstico:\n"
            f"Ratio Test/Train: {ratio_error:.2f}\n"
            f"Estado: {status_text}")

plt.gca().text(0.05, 0.95, info_box, transform=plt.gca().transAxes, fontsize=10,
               verticalalignment='top', bbox=dict(boxstyle='round',␣
 ↪facecolor='white', alpha=0.8, edgecolor=status_color))

plt.title(f"Análisis de Residuos y Estabilidad - Experimento {f+1} (Grado␣
 ↪{grado_vis})")
plt.xlabel("X (Normalizado)")
plt.ylabel("y")
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='lower right', fontsize=9)
plt.show()
```



- Visualización de Residuos: Las líneas grises muestran el error individual de cada punto de test. Esto nos permite ver si el modelo falla de forma aleatoria (buena señal) o si hay zonas donde se le escapa la tendencia (señal de sesgo).
- Contraste de MSE: Calcula y muestra los errores numéricos de Train vs. Test. Si la regresion es correcta, ambos valores deben estar próximos.
- Ratio de Estabilidad: Si el error de test es mucho mayor que el de entrenamiento, el recuadro indicaría "Sobreajuste Detectado". Esto significa que el modelo se ha aprendido los datos de

memoria en lugar de aprender el patrón general.

- Cuadro de Diagnóstico: Esta gráfica nos sirve para comprobar si el ajuste es válido porque es estable y su error de generalización está dentro de los límites.

1.7 Paso 6: Compromiso Sesgo-Varianza

Ahora ejecutamos el loop principal para grados 1 a 15. En cada iteración calculamos:

1. Bias^2 + Ruido: Error de aproximación promedio.
2. Varianza: Cuánto cambia la predicción al cambiar el set de entrenamiento.
3. Error Total (MSE CV): La suma de lo anterior. Es lo que queremos minimizar.

[10]:
```
degrees = range(1, 16)
results = []

for k in degrees:
    all_preds = np.zeros((10, len(X_norm)))
    mse_folds = []

    for f in range(10):
        idx_val = folds[f]
        idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

        # Entrenamiento
        X_train_D = get_design_matrix(X_norm[idx_train], k)
        model = sm.OLS(y[idx_train], X_train_D).fit()

        # Predicción sobre TODO el dataset (para calcular varianza después)
        all_preds[f] = model.predict(get_design_matrix(X_norm, k))

        # Validación
        X_val_D = get_design_matrix(X_norm[idx_val], k)
        y_val_pred = model.predict(X_val_D)
        mse_folds.append(np.mean((y[idx_val] - y_val_pred)**2))

    # --- Cálculos de Sesgo-Varianza ---
    avg_prediction = np.mean(all_preds, axis=0)
    varianza = np.mean(np.var(all_preds, axis=0))
    sesgo_sq_plus_ruido = np.mean((avg_prediction - y)**2)
    error_total = np.mean(mse_folds)

    # Significancia estadística con el dataset completo
    full_model = sm.OLS(y, get_design_matrix(X_norm, k)).fit()
    pval_medio = np.mean(full_model.pvalues)

    results.append({
        'Grado': k,
        'Sesgo2+Ruido': sesgo_sq_plus_ruido,
```

```
        'Varianza': varianza,
        'Error CV': error_total,
        'P-Value Medio': pval_medio,
        'Modelo': full_model
    })

res_df = pd.DataFrame(results)

# Identificar el grado optimo (minimo Error CV)
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']

# Preparar el DataFrame para visualizacion
df_final = res_df[['Grado', 'Sesgo2+Ruido', 'Varianza', 'Error CV', 'P-Value␣
↪Medio']].set_index('Grado')

def highlight_opt(row):
    color = 'background-color: #d3ffd3; font-weight: bold'
    return [color if row.name == k_opt else '' for _ in row]

display(df_final.style.apply(highlight_opt, axis=1))
best_k = k_opt
```

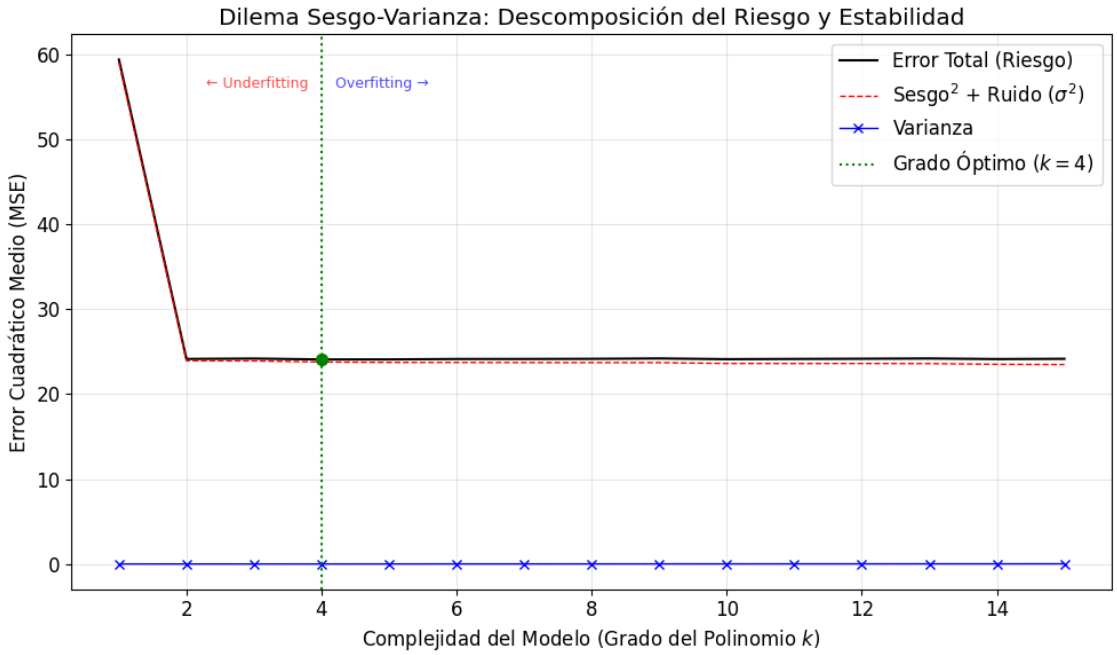
```
<pandas.io.formats.style.Styler at 0x7f4a98533b60>
```

1.8 Paso 7: Visualización del "Codo" y Selección de Modelo

Buscamos el grado k donde el Error CV es mínimo. Este es el punto donde el modelo es lo suficientemente complejo para capturar la tendencia pero no tanto como para aprender el ruido (overfitting).

[11]:
```
# --- Gráfica Dilema Sesgo-Varianza ---
fig, ax1 = plt.subplots(figsize=(10, 6))

# Identificamos el punto óptimo
# res_df debe estar definido previamente en tu entorno
grado_optimo = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_minimo = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']

# Eje Único: Error Total, Sesgo^2 + Ruido y Varianza
line1, = ax1.plot(res_df['Grado'], res_df['Error CV'], 'k-', label='Error Total␣
↪(Riesgo)', lw=1.5)

# CORRECCIÓN: Se agrega 'r' (raw string) para que \sigma no dé error de sintaxis
line2, = ax1.plot(res_df['Grado'], res_df['Sesgo2+Ruido'], 'r--',␣
↪label=r'Sesgo$^2$ + Ruido ($\sigma^2$)', lw=1)
```

```python
line3, = ax1.plot(res_df['Grado'], res_df['Varianza'], 'b-x', label='Varianza',␣
↪lw=1)

ax1.set_xlabel('Complejidad del Modelo (Grado del Polinomio $k$)')
ax1.set_ylabel('Error Cuadrático Medio (MSE)', color='black')
ax1.grid(True, alpha=0.3)

# Línea de Complejidad Óptima
vline = ax1.axvline(grado_optimo, color='green', linestyle=':', label=f'Grado␣
↪Óptimo ($k={grado_optimo}$)', lw=1.5)

# MEJORA: Marcador en el punto mínimo para facilitar la lectura
ax1.scatter(grado_optimo, mse_minimo, color='green', s=50, zorder=5)

# MEJORA: Etiquetas de zona (Underfitting / Overfitting)
ax1.text(grado_optimo - 0.2, ax1.get_ylim()[1] * 0.9, '← Underfitting',␣
↪color='red', ha='right', fontsize=9, alpha=0.7)
ax1.text(grado_optimo + 0.2, ax1.get_ylim()[1] * 0.9, 'Overfitting →',␣
↪color='blue', ha='left', fontsize=9, alpha=0.7)

# Leyendas: Posicionada en la esquina derecha arriba (upper right)
lines = [line1, line2, line3, vline]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper right', frameon=True)

plt.title('Dilema Sesgo-Varianza: Descomposición del Riesgo y Estabilidad')
plt.tight_layout()
plt.show()
```



El grado óptimo es k=4 en este caso, y se ha obtenido mediante la combinación de tres criterios:

1. Minimización del Error de Validación (10-Fold CV): Se han evaluado los polinomios del 1 al 15, el Error CV alcanza su valor mínimo absoluto en el grado 4 (aprox. 24.07). Aunque los grados 2 y 3 presentan valores muy cercanos, el procedimiento de validación cruzada identifica matemáticamente el grado 4 como el punto donde el modelo logra su mejor rendimiento promedio sobre datos no vistos.

2. Identificación del "Codo" en el Dilema Sesgo-Varianza: Se observa que entre el grado 1 y el 2 el error cae drásticamente (reducción de sesgo). Del grado 2 al 4, las mejoras son marginales, lo que indica que el modelo ha capturado la estructura principal. A partir del grado 4, el error deja de descender y la varianza comienza su ascenso, marcando el límite de la complejidad útil.

3. Análisis de Significancia Estadística (P-values): Cuando inspeccionamos los p-values con `statsmodels`, observamos que hasta el grado 4 los coeficientes mantienen relevancia. Sin embargo, en modelos de complejidad superior (k $>$ 5), los p-values medios aumentan considerablemente, indicando que el sistema está empezando a asignar pesos a términos que no aportan información real, incurriendo en overfitting.

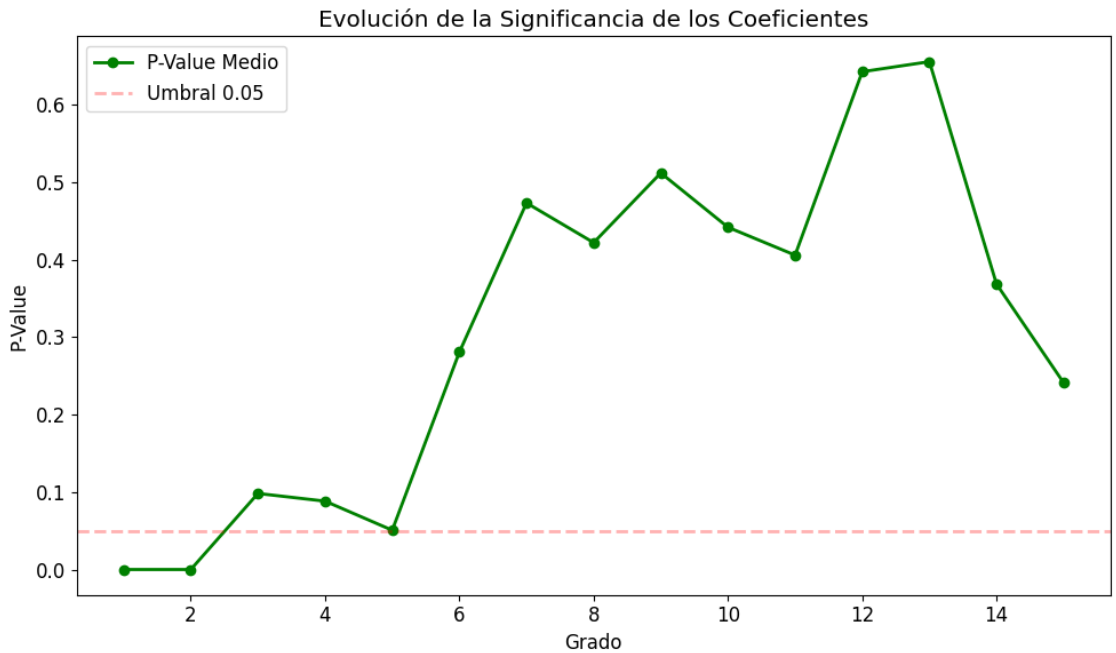
En base a todo lo anterior, el grado 4 se selecciona como el modelo óptimo para este dataset, ya que representa el punto de equilibrio donde se minimiza el riesgo esperado (Error Total), garantizando la mejor capacidad de generalización.

1.9 Paso 8: Análisis de Significancia Estadística

Aunque un polinomio de grado alto ajuste ligeramente mejor, los p-values de sus términos superiores son altos (> 0.05). Esto indica que esos términos no están aportando información real y están

capturando ruido.

[12]:
```
plt.plot(res_df['Grado'], res_df['P-Value Medio'], 'go-', label='P-Value Medio')
plt.axhline(0.05, color='red', ls='--', alpha=0.3, label='Umbral 0.05')
plt.xlabel('Grado')
plt.ylabel('P-Value')
plt.title("Evolución de la Significancia de los Coeficientes")
plt.legend()
plt.show()

print("Grados donde el P-Value medio es bajo indican modelos más confiables.")
```



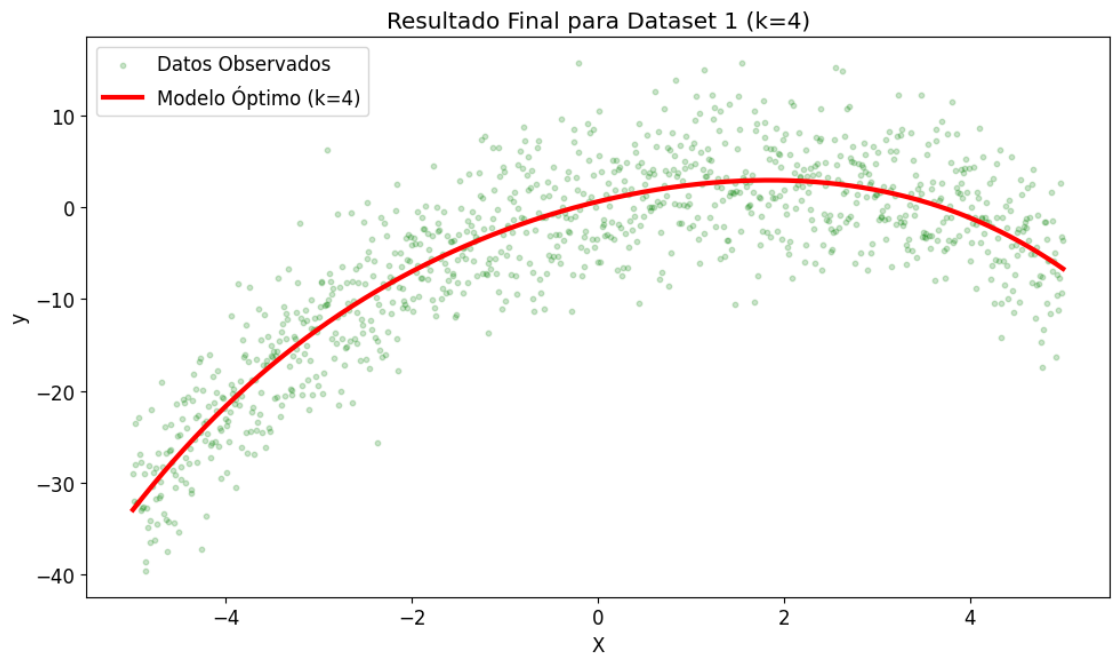

```
Grados donde el P-Value medio es bajo indican modelos más confiables.
```

1.10 Paso 9: Ajuste Final

Finalmente, mostramos cómo se ajusta el mejor modelo ($k = 4$) a los datos originales.

[13]:
```
best_model = res_df.loc[best_k-1, 'Modelo']
y_final = best_model.predict(get_design_matrix(X_norm, best_k))

plt.scatter(X_raw, y, color='green', s=10, alpha=0.2, label='Datos Observados')
plt.plot(X_raw, y_final, 'r-', lw=3, label=f'Modelo Óptimo (k={best_k})')
plt.title(f"Resultado Final para Dataset 1 (k={best_k})")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
```

```
plt.show()

print("Resumen de coeficientes finales:")
for i, coef in enumerate(best_model.params):
    print(f"w{i}: {coef:.4f} (p-value: {best_model.pvalues[i]:.4f})")
```



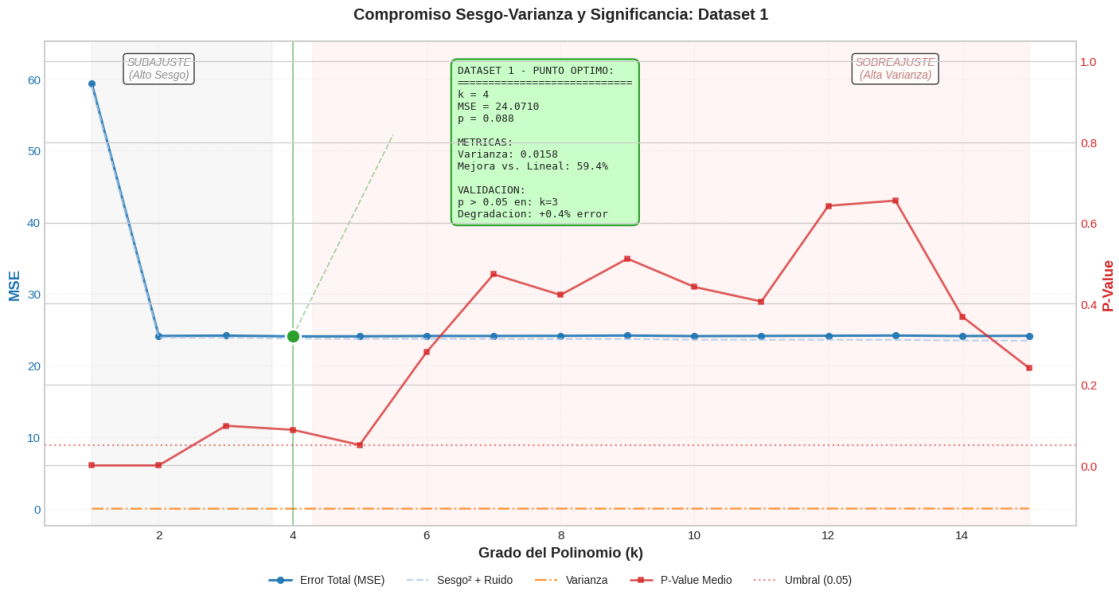

```
Resumen de coeficientes finales:
w0: 0.6429 (p-value: 0.0268)
w1: 12.2752 (p-value: 0.0000)
w2: -15.8705 (p-value: 0.0000)
w3: 0.8737 (p-value: 0.3916)
w4: -4.6087 (p-value: 0.0228)
```

[14]:
```
dataset_id = 1

plt.rcParams['figure.autolayout'] = True

# --- CONFIGURACIÓN ELEGANTE ---
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax1 = plt.subplots(figsize=(13, 7), dpi=110)

grados = res_df['Grado']
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']
pval_opt = res_df.loc[res_df['Error CV'].idxmin(), 'P-Value Medio']
```

```python
# Métricas adicionales (usamos iloc[0] para asegurar que tomamos el primer␣
↪grado disponible)
mse_grado1 = res_df['Error CV'].iloc[0]
mejora_pct = ((mse_grado1 - mse_opt) / mse_grado1) * 100

# --- EJE 1: ERROR (Izquierdo) ---
line1, = ax1.plot(grados, res_df['Error CV'], 'o-', color='#1f77b4', lw=2,
                  markersize=5, label='Error Total (MSE)', alpha=0.9)
line2, = ax1.plot(grados, res_df['Sesgo2+Ruido'], '--', color='#aec7e8', lw=1.5,
                  label='Sesgo² + Ruido', alpha=0.75)
line5, = ax1.plot(grados, res_df['Varianza'], '-.', color='#ff7f0e', lw=1.5,
                  label='Varianza', alpha=0.8)

# Resaltar el punto óptimo
ax1.set_ylim(res_df['Error CV'].min() * -0.1, res_df['Error CV'].max() * 1.1)
ax1.plot(k_opt, mse_opt, 'o', color='#2ca02c', markersize=12,
         markeredgewidth=2, markeredgecolor='white', zorder=10)

ax1.set_xlabel('Grado del Polinomio (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('MSE', color='#1f77b4', fontsize=12, fontweight='bold')
ax1.tick_params(axis='y', labelcolor='#1f77b4', labelsize=10)
ax1.tick_params(axis='x', labelsize=10)
ax1.grid(True, alpha=0.25, linestyle='--', linewidth=0.5)

# --- EJE 2: P-VALUE (Derecho) ---
ax2 = ax1.twinx()
ax2.set_ylim(-0.15, 1.05)
line3, = ax2.plot(grados, res_df['P-Value Medio'], 's-', color='#d62728',
                  alpha=0.75, lw=1.8, markersize=4.5, label='P-Value Medio')
line4 = ax2.axhline(0.05, color='#d62728', linestyle=':', alpha=0.5, lw=1.5,
                    label='Umbral (0.05)')

ax2.set_ylabel('P-Value', color='#d62728', fontsize=12, fontweight='bold')
ax2.tick_params(axis='y', labelcolor='#d62728', labelsize=10)

# --- REGIONES SOMBREADAS DINÁMICAS ---
ax1.axvspan(grados.min(), k_opt-0.3, color='gray', alpha=0.06)
ax1.axvspan(k_opt+0.3, grados.max(), color='red', alpha=0.04)

# --- LÍNEA ÓPTIMA ---
ax1.axvline(k_opt, color='#2ca02c', linestyle='-', lw=1, alpha=0.6, zorder=5)

# Etiquetas de regiones
ax1.text(grados.min() + 1, ax1.get_ylim()[1]*0.92, "SUBAJUSTE\n(Alto Sesgo)",␣
↪alpha=0.5,
         fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
```

```
ax1.text(grados.max() - 2, ax1.get_ylim()[1]*0.92, "SOBREAJUSTE\n(Alta␣
↪Varianza)",
         color='darkred', alpha=0.5, fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))

# --- PANEL DE ANÁLISIS DINÁMICO ---
try:
    k_pvalue_fail = grados[res_df['P-Value Medio'] > 0.05].iloc[0]
    fail_text = f"k={k_pvalue_fail}"

    if k_pvalue_fail <= grados.max():
        mse_degradado = res_df.loc[res_df['Grado'] == k_pvalue_fail, 'Error␣
↪CV'].values[0]
        deg_pct = ((mse_degradado - mse_opt) / mse_opt) * 100
        degradacion = f"+{deg_pct:.1f}% error"
    else:
        degradacion = "N/A"
except:
    fail_text = "Nunca"
    degradacion = "N/A"

var_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Varianza']

info_panel = (f"DATASET {dataset_id} - PUNTO OPTIMO:\n"
              f"{'='*28}\n"
              f"k = {k_opt}\n"
              f"MSE = {mse_opt:.4f}\n"
              f"p = {pval_opt:.3f}\n"
              f"\n"
              f"METRICAS:\n"
              f"Varianza: {var_opt:.4f}\n"
              f"Mejora vs. Lineal: {mejora_pct:.1f}%\n"
              f"\n"
              f"VALIDACION:\n"
              f"p > 0.05 en: {fail_text}\n"
              f"Degradacion: {degradacion}")

ax1.text(0.4, 0.95, info_panel, transform=ax1.transAxes, fontsize=8.5,
         family='monospace', verticalalignment='top',
         bbox=dict(boxstyle="round,pad=0.6", facecolor='#c7ffc7',
                   edgecolor='#2ca02c', lw=1.5, alpha=0.95))

# Línea de conexión al área del punto óptimo
ax1.plot([k_opt, k_opt + 1.5], [mse_opt, ax1.get_ylim()[1]*0.80],
         color='#2ca02c', lw=1.2, linestyle='--', alpha=0.4, zorder=1)

# --- TÍTULO Y LEYENDA ---
```

```
plt.title(f'Compromiso Sesgo-Varianza y Significancia: Dataset {dataset_id}',
          fontsize=13, fontweight='bold', pad=18)

lines = [line1, line2, line5, line3, line4]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5, -0.08),
          ncol=5, fontsize=9, framealpha=0.95, edgecolor='gray', fancybox=True)

plt.tight_layout()
plt.show()
```



La gráfica nos muestra dos métricas simultáneas para explicar la elección del grado óptimo del polinomio.

El eje izquierdo representa el error de validación (MSE). La curva azul baja cuando el modelo captura mejor la estructura de los datos, alcanza un mínimo en el grado óptimo, y luego vuelve a subir cuando el modelo empieza a memorizar ruido en lugar de aprender el patrón real.

El eje derecho representa el p-value medio de los coeficientes. Valores por debajo de 0.05 nos indican que los términos del polinomio son estadísticamente útiles. Cuando el p-value sube por encima de este umbral, los coeficientes añadidos no aportan información.

Las zonas sombreadas se incluyen para ver el compromiso sesgo-varianza: la zona gris corresponde a modelos demasiado simples que no capturan la complejidad de los datos, mientras que la zona roja indica modelos muy complejos que capturan ruido.

El punto donde el error alcanza su mínimo coincide con el último grado antes de que los p-values empiecen a fallar. Esta convergencia valida el modelo seleccionado.

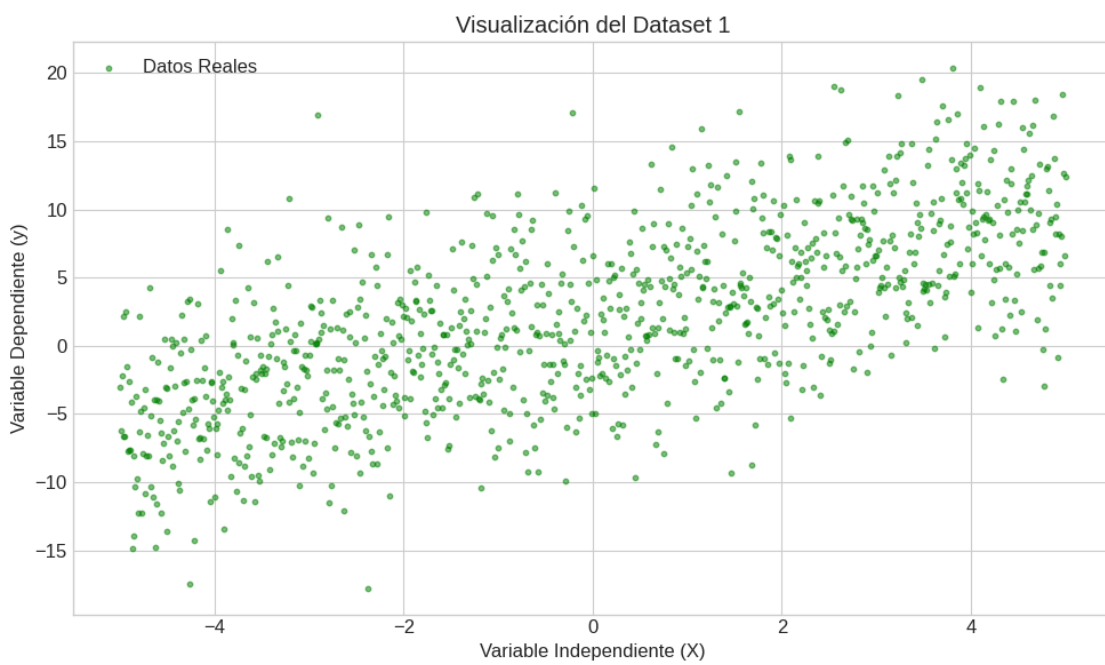
1.11 Análisis del dataset_2.csv

1.11.1 Paso 1: Carga y Exploración

[15]:
```
df = pd.read_csv('dataset_2.csv').sort_values(by='X')

# Extraemos las variables
X_raw = df['X'].values
y = df['y'].values

print(f"El dataset_1 tiene {len(df)} muestras.")
display(df.head())

# Visualización inicial
plt.scatter(X_raw, y, s=10, color='green', alpha=0.5, label='Datos Reales')
plt.title("Visualización del Dataset 1")
plt.xlabel("Variable Independiente (X)")
plt.ylabel("Variable Dependiente (y)")
plt.legend()
plt.show()
```

```
El dataset_1 tiene 1000 muestras.
```

	X	y
0	-5.00000	-3.016429
1	-4.98999	-6.176306
2	-4.97998	-2.231527
3	-4.96997	2.160194
4	-4.95996	-6.610707



1.12 Paso 2: Normalización

[16]:
```
X_min, X_max = X_raw.min(), X_raw.max()
X_norm = 2 * (X_raw - X_min) / (X_max - X_min) - 1

print("Rango original de X:", X_min, "a", X_max)
print("Rango normalizado de X:", X_norm.min(), "a", X_norm.max())
```

```
Rango original de X: -5.0 a 5.0
Rango normalizado de X: -1.0 a 1.0
```

1.13 Paso 3: Matriz de Diseño

[17]:
```
def get_design_matrix(X, degree):
    """Genera la matriz de diseño con potencias desde 0 hasta degree."""
    columns = [X**i for i in range(degree + 1)]
    return np.column_stack(columns)

# Ejemplo para grado 2
D_example = get_design_matrix(X_norm[:8], 3)
print("Ejemplo de Matriz de Diseño (Grado 3, primeras 8 filas):")
print(D_example)
```

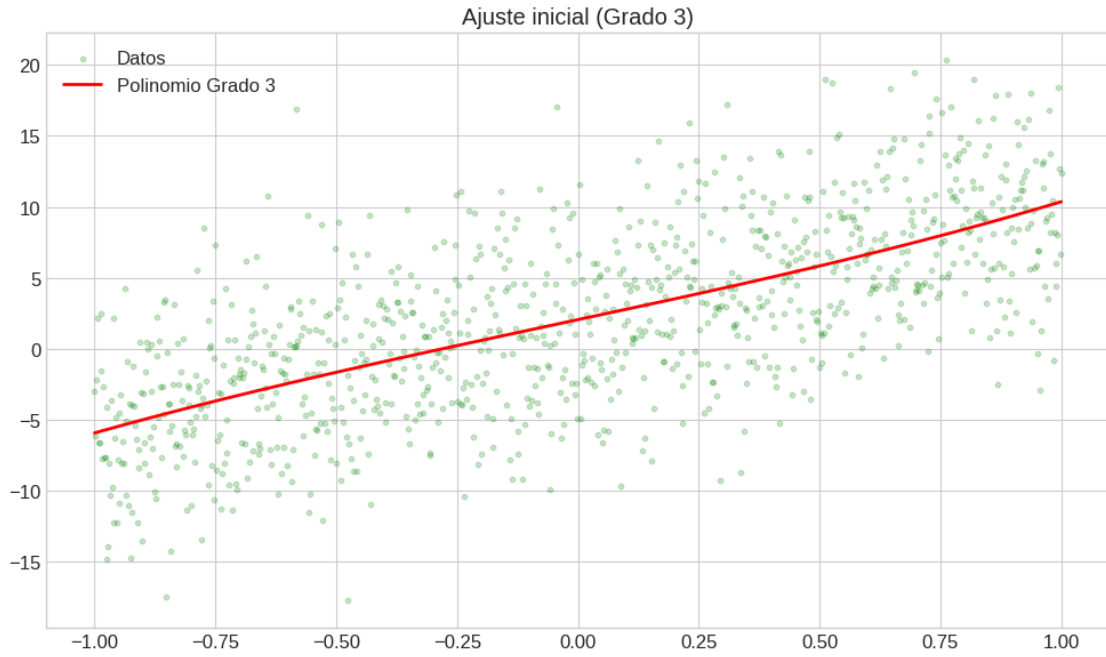
```
Ejemplo de Matriz de Diseño (Grado 3, primeras 8 filas):
[[ 1.         -1.          1.         -1.        ]
 [ 1.         -0.997998    0.996      -0.99400601]
 [ 1.         -0.995996    0.99200802 -0.98803602]
 [ 1.         -0.99399399  0.98802406 -0.98208998]
 [ 1.         -0.99199199  0.98404811 -0.97616785]
 [ 1.         -0.98998999  0.98008018 -0.97026957]
 [ 1.         -0.98798799  0.97612026 -0.9643951 ]
 [ 1.         -0.98598599  0.97216836 -0.95854438]]
```

1.14 Paso 4: Ajuste de un Modelo Individual

[18]:
```
grado_prueba = 3
D_3 = get_design_matrix(X_norm, grado_prueba)

# Ajustamos el modelo de Mínimos Cuadrados Ordinarios (OLS - Ordinary Least␣
↪Squares), de statsmodels
modelo = sm.OLS(y, D_3).fit()

print(modelo.summary())

# Predicciones
```

```python
y_pred = modelo.predict(D_3)

# Visualizar el primer ajuste
plt.scatter(X_norm, y, s=10, color='green', alpha=0.2
            , label='Datos')
plt.plot(X_norm, y_pred, color='red', label=f'Polinomio Grado {grado_prueba}')
plt.title("Ajuste inicial (Grado 3)")
plt.legend()
plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.460
Model:                            OLS   Adj. R-squared:                  0.458
Method:                 Least Squares   F-statistic:                     282.6
Date:                Mon, 16 Feb 2026   Prob (F-statistic):          1.08e-132
Time:                        20:22:21   Log-Likelihood:                -3005.8
No. Observations:                1000   AIC:                             6020.
Df Residuals:                     996   BIC:                             6039.
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          2.0395      0.232      8.778      0.000       1.584       2.495
x1             7.2752      0.670     10.858      0.000       5.960       8.590
x2             0.1712      0.519      0.330      0.741      -0.846       1.189
x3             0.8737      1.021      0.855      0.393      -1.131       2.878
==============================================================================
Omnibus:                        2.623   Durbin-Watson:                   2.019
Prob(Omnibus):                  0.269   Jarque-Bera (JB):                2.495
Skew:                           0.116   Prob(JB):                        0.287
Kurtosis:                       3.076   Cond. No.                         8.20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```



1.15 Paso 5: Validación Cruzada (10-Fold CV)

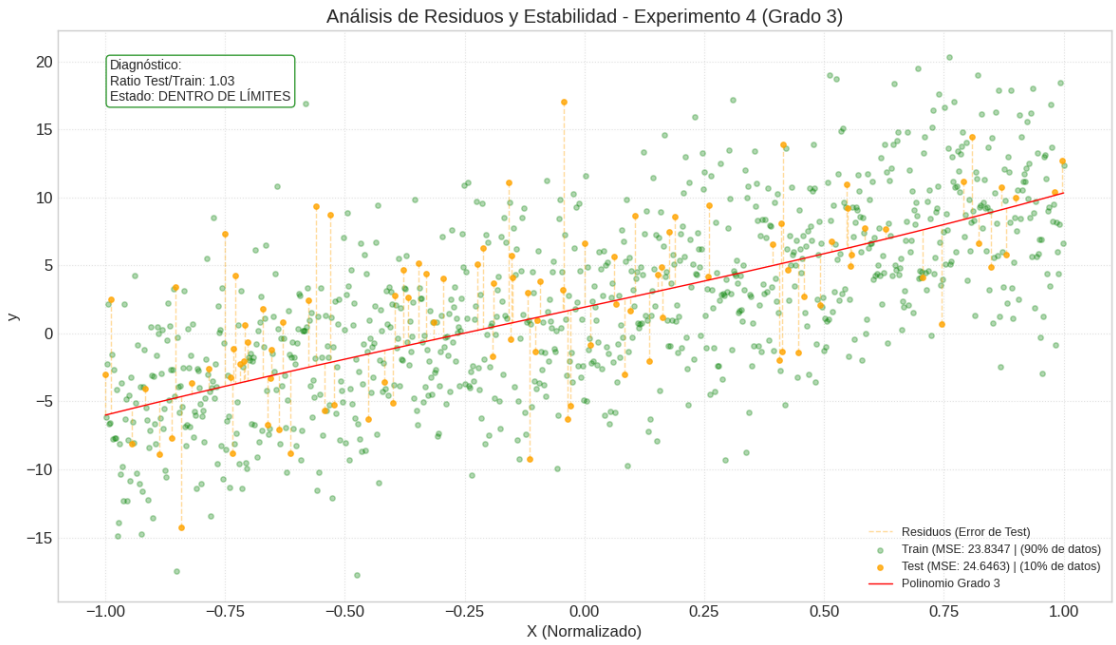
[19]:
```
np.random.seed(42)

indices = np.arange(len(X_norm))
np.random.shuffle(indices)

# Dividimos en 10 partes
folds = np.array_split(indices, 10)

print(f"Se han creado 10 folds de aproximadamente {len(folds[0])} muestras cada␣
↪uno.")
```

```
Se han creado 10 folds de aproximadamente 100 muestras cada uno.
```

[20]:
```
# --- VISUALIZACIÓN CON ANÁLISIS DE ERROR Y RESIDUOS (Fold 4, Grado 3) ---

f = 3
grado_vis = 3
idx_test = folds[f]
idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

# 1. Ajuste del modelo
D_train = get_design_matrix(X_norm[idx_train], grado_vis)
modelo_ejemplo = sm.OLS(y[idx_train], D_train).fit()
```

```python
# 2. Cálculo de errores (MSE)
y_train_pred = modelo_ejemplo.predict(D_train)
mse_train = np.mean((y[idx_train] - y_train_pred)**2)

D_test = get_design_matrix(X_norm[idx_test], grado_vis)
y_test_pred = modelo_ejemplo.predict(D_test)
mse_test = np.mean((y[idx_test] - y_test_pred)**2)

# 3. Verificación de límites aceptables
# Un modelo se considera inestable si el error de test supera␣
↪significativamente al de entrenamiento
ratio_error = mse_test / mse_train
es_aceptable = ratio_error < 1.5  # Tolerancia del 50% de incremento

plt.figure(figsize=(12, 7))

# Visualizar Residuos (Líneas de error para los puntos de Test)
plt.vlines(X_norm[idx_test], y[idx_test], y_test_pred,
           color='orange', lw=1.0, linestyle='--', alpha=0.4, label='Residuos␣
↪(Error de Test)')

# Puntos de entrenamiento y test
plt.scatter(X_norm[idx_train], y[idx_train], s=15, color='green', alpha=0.3,␣
↪label=f'Train (MSE: {mse_train:.4f} | (90% de datos)')
plt.scatter(X_norm[idx_test], y[idx_test], s=15, color='orange', alpha=0.8,␣
↪label=f'Test (MSE: {mse_test:.4f}) | (10% de datos)')

# Curva del modelo
x_curve = np.linspace(-1, 1, 500)
y_curve = modelo_ejemplo.predict(get_design_matrix(x_curve, grado_vis))
plt.plot(x_curve, y_curve, color='red', lw=1.0, label=f'Polinomio Grado␣
↪{grado_vis}')

# Panel de diagnóstico (Cuadro de texto)
status_color = "green" if es_aceptable else "red"
status_text = "DENTRO DE LÍMITES" if es_aceptable else "SOBREAJUSTE DETECTADO"
info_box = (f"Diagnóstico:\n"
            f"Ratio Test/Train: {ratio_error:.2f}\n"
            f"Estado: {status_text}")

plt.gca().text(0.05, 0.95, info_box, transform=plt.gca().transAxes, fontsize=10,
               verticalalignment='top', bbox=dict(boxstyle='round',␣
↪facecolor='white', alpha=0.8, edgecolor=status_color))
```

```
plt.title(f"Análisis de Residuos y Estabilidad - Experimento {f+1} (Grado␣
↪{grado_vis})")
plt.xlabel("X (Normalizado)")
plt.ylabel("y")
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='lower right', fontsize=9)
plt.show()
```



1.16 Paso 6: Compromiso Sesgo-Varianza

[21]:
```
degrees = range(1, 16)
results = []

for k in degrees:
    all_preds = np.zeros((10, len(X_norm)))
    mse_folds = []

    for f in range(10):
        idx_val = folds[f]
        idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

        # Entrenamiento
        X_train_D = get_design_matrix(X_norm[idx_train], k)
        model = sm.OLS(y[idx_train], X_train_D).fit()
```

```
        # Predicción sobre TODO el dataset (para calcular varianza después)
        all_preds[f] = model.predict(get_design_matrix(X_norm, k))

        # Validación
        X_val_D = get_design_matrix(X_norm[idx_val], k)
        y_val_pred = model.predict(X_val_D)
        mse_folds.append(np.mean((y[idx_val] - y_val_pred)**2))

    # --- Cálculos de Sesgo-Varianza ---
    avg_prediction = np.mean(all_preds, axis=0)
    varianza = np.mean(np.var(all_preds, axis=0))
    sesgo_sq_plus_ruido = np.mean((avg_prediction - y)**2)
    error_total = np.mean(mse_folds)

    # Significancia estadística con el dataset completo
    full_model = sm.OLS(y, get_design_matrix(X_norm, k)).fit()
    pval_medio = np.mean(full_model.pvalues)

    results.append({
        'Grado': k,
        'Sesgo2+Ruido': sesgo_sq_plus_ruido,
        'Varianza': varianza,
        'Error CV': error_total,
        'P-Value Medio': pval_medio,
        'Modelo': full_model
    })

res_df = pd.DataFrame(results)

# Identificar el grado optimo (minimo Error CV)
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']

# Preparar el DataFrame para visualizacion
df_final = res_df[['Grado', 'Sesgo2+Ruido', 'Varianza', 'Error CV', 'P-Value␣
 ↪Medio']].set_index('Grado')

def highlight_opt(row):
    color = 'background-color: #d3ffd3; font-weight: bold'
    return [color if row.name == k_opt else '' for _ in row]

display(df_final.style.apply(highlight_opt, axis=1))
best_k = k_opt
```

```
<pandas.io.formats.style.Styler at 0x7f4a9458ba40>
```

Durante el análisis se ha detectado que el Dataset_1 y el Dataset_2 comparten resultados idénticos: ambos alcanzan su grado óptimo en $k = 4$ y presentan exactamente los mismos valores de error (MSE) en la validación cruzada.

Esto ocurre porque, si restamos los valores de ambos datasets, vemos que la diferencia es que el Dataset 1 es el Dataset 2, pero con un polinomio de grado 2 añadido. Cuando el modelo llega al grado 2 ya es capaz de anular esa diferencia. A partir de ahí, para el algoritmo es como si estuviera trabajando con el mismo conjunto de datos, el ruido es el mismo y el k es igual para los dos conjuntos de datos.

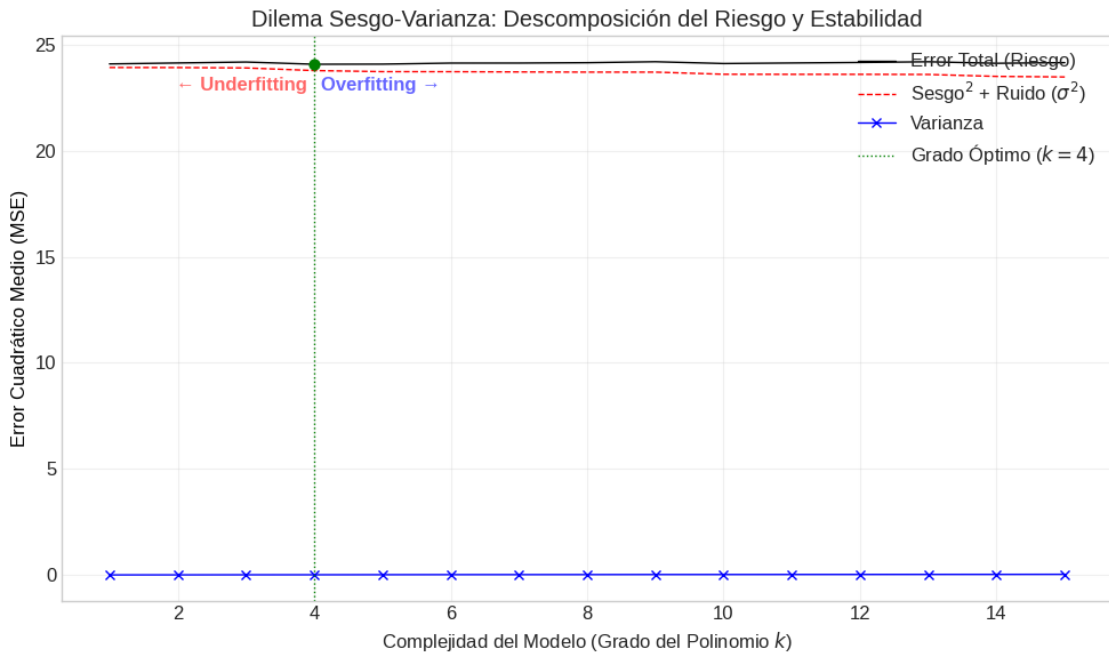
Este experimento demuestra que la complejidad necesaria para explicar una nube de puntos viene dictada por su componente más difícil (el mayor grado), y no por las tendencias más simples que la acompañen. En la práctica, esto ocurre porque un modelo de grado 4 ya posee la flexibilidad matemática necesaria para 'absorber' cualquier parábola (grado 2) sin añadir error adicional; al restarla, los residuos que el modelo debe explicar siguen conteniendo la misma señal de cuarto grado y el mismo nivel de ruido aleatorio.

Como el compromiso entre sesgo y varianza depende de cuándo el modelo empieza a confundir la señal con el ruido, el punto óptimo de ajuste no se mueve: el algoritmo detecta que la 'altura' de la dificultad sigue siendo la misma, independientemente de que hayamos desplazado o simplificado la base de la tendencia. Así, entendemos que la complejidad de un modelo no es una suma de sus partes, sino que está definida por su término de mayor orden.

1.17 Paso 7: Visualización del "Codo" y Selección de Modelo

[22]:
```
# --- Gráfica Dilema Sesgo-Varianza ---
fig, ax1 = plt.subplots(figsize=(10, 6))

# Identificamos el punto óptimo
grado_optimo = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_minimo = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']

# Eje Único: Error Total, Sesgo^2 + Ruido y Varianza
line1, = ax1.plot(res_df['Grado'], res_df['Error CV'], 'k-', label='Error Total␣
↪(Riesgo)', lw=1)

line2, = ax1.plot(res_df['Grado'], res_df['Sesgo2+Ruido'], 'r--',␣
↪label=r'Sesgo$^2$ + Ruido ($\sigma^2$)', lw=1)

line3, = ax1.plot(res_df['Grado'], res_df['Varianza'], 'b-x', label='Varianza',␣
↪lw=1)

ax1.set_xlabel('Complejidad del Modelo (Grado del Polinomio $k$)')
ax1.set_ylabel('Error Cuadrático Medio (MSE)', color='black')
ax1.grid(True, alpha=0.3)

# Línea de Complejidad Óptima
vline = ax1.axvline(grado_optimo, color='green', linestyle=':', label=f'Grado␣
↪Óptimo ($k={grado_optimo}$)', lw=1)

# --- MEJORAS VISUALES ---
# Punto en el mínimo para mayor claridad
```

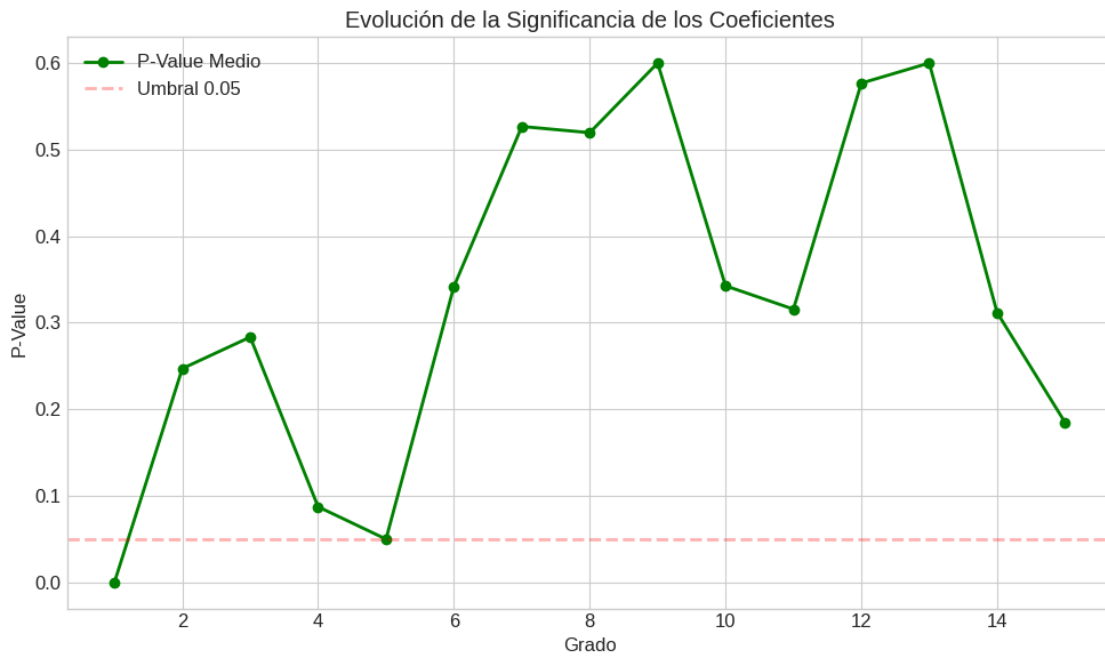
```
ax1.scatter(grado_optimo, mse_minimo, color='green', s=40, zorder=5)

# Etiquetas informativas de las zonas de la gráfica
ax1.text(grado_optimo - 0.1, ax1.get_ylim()[1] * 0.9, '← Underfitting',
         color='red', ha='right', fontweight='bold', alpha=0.6)
ax1.text(grado_optimo + 0.1, ax1.get_ylim()[1] * 0.9, 'Overfitting →',
         color='blue', ha='left', fontweight='bold', alpha=0.6)

# Leyendas: Posicionada en la esquina derecha arriba (upper right)
lines = [line1, line2, line3, vline]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper right')

plt.title('Dilema Sesgo-Varianza: Descomposición del Riesgo y Estabilidad')
plt.tight_layout()
plt.show()
```



1.18 Pasos 8: Significancia (p-value)

[23]:
```
plt.plot(res_df['Grado'], res_df['P-Value Medio'], 'go-', label='P-Value Medio')
plt.axhline(0.05, color='red', ls='--', alpha=0.3, label='Umbral 0.05')
plt.xlabel('Grado')
plt.ylabel('P-Value')
plt.title("Evolución de la Significancia de los Coeficientes")
plt.legend()
```

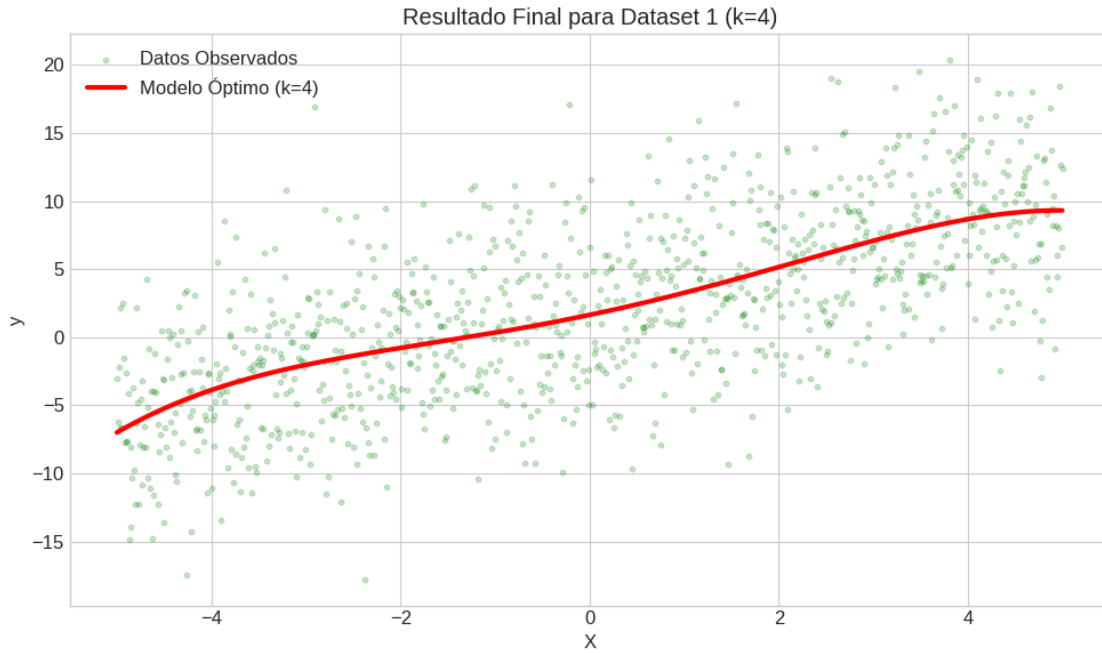
```
plt.show()

print("Grados donde el P-Value medio es bajo indican modelos más confiables.")
```




```
Grados donde el P-Value medio es bajo indican modelos más confiables.
```

1.19 Paso 9: Ajuste Final

[24]:
```
best_model = res_df.loc[best_k-1, 'Modelo']
y_final = best_model.predict(get_design_matrix(X_norm, best_k))

plt.scatter(X_raw, y, color='green', s=10, alpha=0.2, label='Datos Observados')
plt.plot(X_raw, y_final, 'r-', lw=3, label=f'Modelo Óptimo (k={best_k})')
plt.title(f"Resultado Final para Dataset 1 (k={best_k})")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

print("Resumen de coeficientes finales:")
for i, coef in enumerate(best_model.params):
    print(f"w{i}: {coef:.4f} (p-value: {best_model.pvalues[i]:.4f})")
```



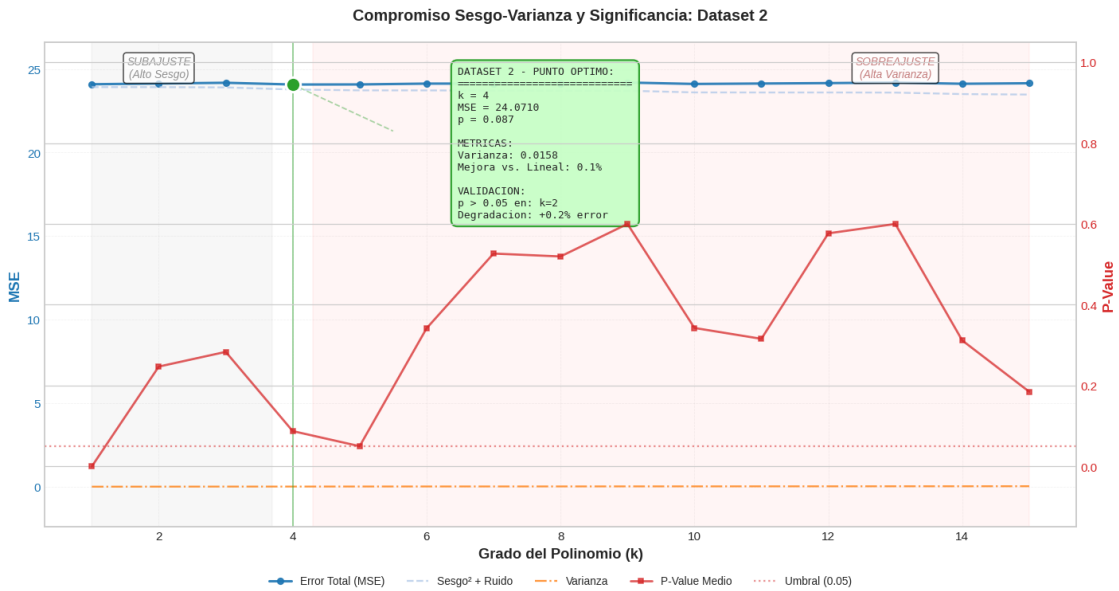

```
Resumen de coeficientes finales:
w0: 1.6429 (p-value: 0.0000)
w1: 7.2752 (p-value: 0.0000)
w2: 4.1295 (p-value: 0.0228)
w3: 0.8737 (p-value: 0.3916)
w4: -4.6087 (p-value: 0.0228)
```

[25]:
```
# --- IDENTIFICADOR DEL DATASET ---
dataset_id = 2

plt.rcParams['figure.autolayout'] = True

# --- CONFIGURACIÓN ELEGANTE ---
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax1 = plt.subplots(figsize=(13, 7), dpi=110)

grados = res_df['Grado']
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']
pval_opt = res_df.loc[res_df['Error CV'].idxmin(), 'P-Value Medio']

# Métricas adicionales (usamos iloc[0] para asegurar que tomamos el primer␣
↪grado disponible)
mse_grado1 = res_df['Error CV'].iloc[0]
mejora_pct = ((mse_grado1 - mse_opt) / mse_grado1) * 100
```

```python
# --- EJE 1: ERROR (Izquierdo) ---
line1, = ax1.plot(grados, res_df['Error CV'], 'o-', color='#1f77b4', lw=2,
                  markersize=5, label='Error Total (MSE)', alpha=0.9)
line2, = ax1.plot(grados, res_df['Sesgo2+Ruido'], '--', color='#aec7e8', lw=1.5,
                  label='Sesgo² + Ruido', alpha=0.75)
line5, = ax1.plot(grados, res_df['Varianza'], '-.', color='#ff7f0e', lw=1.5,
                  label='Varianza', alpha=0.8)

# Resaltar el punto óptimo
ax1.set_ylim(res_df['Error CV'].min() * -0.1, res_df['Error CV'].max() * 1.1)
ax1.plot(k_opt, mse_opt, 'o', color='#2ca02c', markersize=12,
         markeredgewidth=2, markeredgecolor='white', zorder=10)

ax1.set_xlabel('Grado del Polinomio (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('MSE', color='#1f77b4', fontsize=12, fontweight='bold')
ax1.tick_params(axis='y', labelcolor='#1f77b4', labelsize=10)
ax1.tick_params(axis='x', labelsize=10)
ax1.grid(True, alpha=0.25, linestyle='--', linewidth=0.5)

# --- EJE 2: P-VALUE (Derecho) ---
ax2 = ax1.twinx()
ax2.set_ylim(-0.15, 1.05)
line3, = ax2.plot(grados, res_df['P-Value Medio'], 's-', color='#d62728',
                  alpha=0.75, lw=1.8, markersize=4.5, label='P-Value Medio')
line4 = ax2.axhline(0.05, color='#d62728', linestyle=':', alpha=0.5, lw=1.5,
                    label='Umbral (0.05)')

ax2.set_ylabel('P-Value', color='#d62728', fontsize=12, fontweight='bold')
ax2.tick_params(axis='y', labelcolor='#d62728', labelsize=10)

# --- REGIONES SOMBREADAS DINÁMICAS ---
ax1.axvspan(grados.min(), k_opt-0.3, color='gray', alpha=0.06)
ax1.axvspan(k_opt+0.3, grados.max(), color='red', alpha=0.04)

# --- LÍNEA ÓPTIMA ---
ax1.axvline(k_opt, color='#2ca02c', linestyle='-', lw=1, alpha=0.6, zorder=5)

# Etiquetas de regiones
ax1.text(grados.min() + 1, ax1.get_ylim()[1]*0.92, "SUBAJUSTE\n(Alto Sesgo)",␣
 ↪alpha=0.5,
         fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
ax1.text(grados.max() - 2, ax1.get_ylim()[1]*0.92, "SOBREAJUSTE\n(Alta␣
 ↪Varianza)",
         color='darkred', alpha=0.5, fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
```

```python
# --- PANEL DE ANÁLISIS DINÁMICO ---
try:
    k_pvalue_fail = grados[res_df['P-Value Medio'] > 0.05].iloc[0]
    fail_text = f"k={k_pvalue_fail}"

    if k_pvalue_fail <= grados.max():
        mse_degradado = res_df.loc[res_df['Grado'] == k_pvalue_fail, 'Error␣
↪CV'].values[0]
        deg_pct = ((mse_degradado - mse_opt) / mse_opt) * 100
        degradacion = f"+{deg_pct:.1f}% error"
    else:
        degradacion = "N/A"
except:
    fail_text = "Nunca"
    degradacion = "N/A"

var_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Varianza']

info_panel = (f"DATASET {dataset_id} - PUNTO OPTIMO:\n"
              f"{'='*28}\n"
              f"k = {k_opt}\n"
              f"MSE = {mse_opt:.4f}\n"
              f"p = {pval_opt:.3f}\n"
              f"\n"
              f"METRICAS:\n"
              f"Varianza: {var_opt:.4f}\n"
              f"Mejora vs. Lineal: {mejora_pct:.1f}%\n"
              f"\n"
              f"VALIDACION:\n"
              f"p > 0.05 en: {fail_text}\n"
              f"Degradacion: {degradacion}")

ax1.text(0.4, 0.95, info_panel, transform=ax1.transAxes, fontsize=8.5,
         family='monospace', verticalalignment='top',
         bbox=dict(boxstyle="round,pad=0.6", facecolor='#c7ffc7',
                   edgecolor='#2ca02c', lw=1.5, alpha=0.95))

# Línea de conexión al área del punto óptimo
ax1.plot([k_opt, k_opt + 1.5], [mse_opt, ax1.get_ylim()[1]*0.80],
         color='#2ca02c', lw=1.2, linestyle='--', alpha=0.4, zorder=1)

# --- TÍTULO Y LEYENDA ---
plt.title(f'Compromiso Sesgo-Varianza y Significancia: Dataset {dataset_id}',
          fontsize=13, fontweight='bold', pad=18)

lines = [line1, line2, line5, line3, line4]
labels = [l.get_label() for l in lines]
```

```
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5, -0.08),
           ncol=5, fontsize=9, framealpha=0.95, edgecolor='gray', fancybox=True)

plt.tight_layout()
plt.show()
```



1.20 Análisis del dataset_3.csv

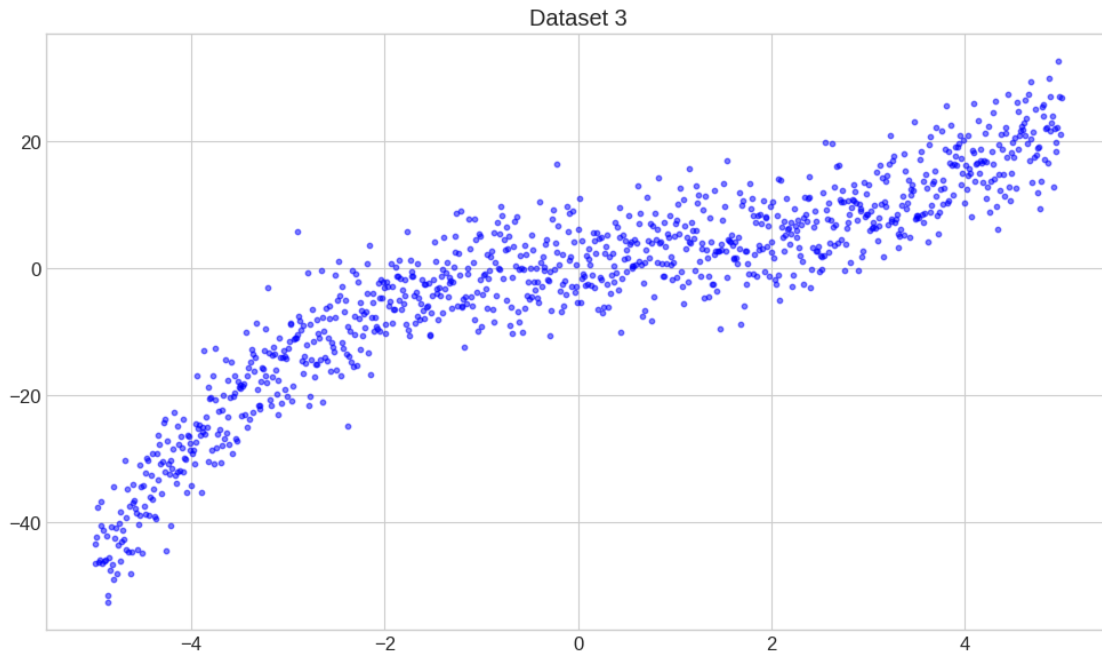
1.20.1 Paso 1: Carga y Exploración

[26]:
```
dataset_id = 3
df = pd.read_csv('dataset_3.csv').sort_values(by='X')
X_raw, y = df['X'].values, df['y'].values
print(f'Muestras: {len(df)}')
display(df.head())

plt.scatter(X_raw, y, s=10, color='blue', alpha=0.5)
plt.title(f'Dataset {dataset_id}')
plt.show()
```

```
Muestras: 1000
```

```
          X          y
0 -5.00000 -43.516429
1 -4.98999 -46.471452
2 -4.97998 -42.322518
```

```
3 -4.96997 -37.727341
4 -4.95996 -46.295484
```



1.21 Paso 2: Normalización de los Datos

[27]:
```
X_min, X_max = X_raw.min(), X_raw.max()
X_norm = 2 * (X_raw - X_min) / (X_max - X_min) - 1

print("Rango original de X:", X_min, "a", X_max)
print("Rango normalizado de X:", X_norm.min(), "a", X_norm.max())
```

```
Rango original de X: -5.0 a 5.0
Rango normalizado de X: -1.0 a 1.0
```

1.22 Paso 3: Creación de la Matriz de Diseño

Para ajustar un polinomio de grado k, necesitamos convertir nuestro vector X en una matriz donde cada columna sea una potencia de X:

$$D = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ 1 & x_2 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{bmatrix}$$

[28]:
```
def get_design_matrix(X, degree):
    """Genera la matriz de diseño con potencias desde 0 hasta degree."""
```

```
    columns = [X**i for i in range(degree + 1)]
    return np.column_stack(columns)

# Ejemplo para grado 3
D_example = get_design_matrix(X_norm[:6], 3)
print("Ejemplo de Matriz de Diseño (Grado 3, primeras 6 filas):")
print(D_example)
```

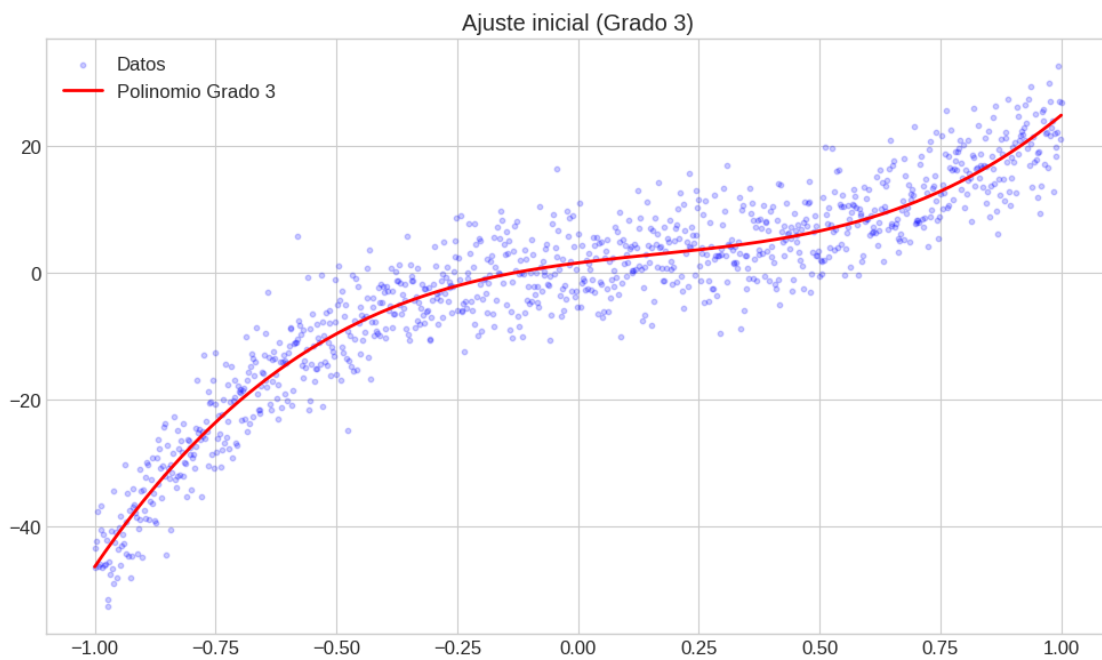
```
Ejemplo de Matriz de Diseño (Grado 3, primeras 6 filas):
[[ 1.         -1.          1.         -1.        ]
 [ 1.         -0.997998    0.996      -0.99400601]
 [ 1.         -0.995996    0.99200802 -0.98803602]
 [ 1.         -0.99399399  0.98802406 -0.98208998]
 [ 1.         -0.99199199  0.98404811 -0.97616785]
 [ 1.         -0.98998999  0.98008018 -0.97026957]]
```

1.23 Paso 4: Ajuste de un Modelo Individual

[29]:
```
grado_prueba = 3
D_3 = get_design_matrix(X_norm, grado_prueba)

# Ajustamos el modelo de Mínimos Cuadrados Ordinarios (OLS - Ordinary Least␣
↪Squares), de statsmodels
modelo = sm.OLS(y, D_3).fit()

print(modelo.summary())

# Predicciones
y_pred = modelo.predict(D_3)

# Visualizar el primer ajuste
plt.scatter(X_norm, y, s=10, color='blue', alpha=0.2
            , label='Datos')
plt.plot(X_norm, y_pred, color='red', label=f'Polinomio Grado {grado_prueba}')
plt.title("Ajuste inicial (Grado 3)")
plt.legend()
plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.911
Model:                            OLS   Adj. R-squared:                  0.910
Method:                 Least Squares   F-statistic:                     3380.
Date:                Mon, 16 Feb 2026   Prob (F-statistic):               0.00
Time:                        20:22:34   Log-Likelihood:                -3005.8
No. Observations:                1000   AIC:                             6020.
Df Residuals:                     996   BIC:                             6039.
```

```
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.5395      0.232      6.626      0.000       1.084       1.995
x1             9.7752      0.670     14.589      0.000       8.460      11.090
x2           -12.3288      0.519    -23.778      0.000     -13.346     -11.311
x3            25.8737      1.021     25.329      0.000      23.869      27.878
==============================================================================
Omnibus:                        2.623   Durbin-Watson:                   2.019
Prob(Omnibus):                  0.269   Jarque-Bera (JB):                2.495
Skew:                           0.116   Prob(JB):                        0.287
Kurtosis:                       3.076   Cond. No.                         8.20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```



1.24 Paso 5: Validación Cruzada (10-Fold CV)

[30]:
```
np.random.seed(42)

indices = np.arange(len(X_norm))
```

```
np.random.shuffle(indices)

# Dividimos en 10 partes
folds = np.array_split(indices, 10)

print(f"Se han creado 10 folds de aproximadamente {len(folds[0])} muestras cada␣
↪uno.")
```

```
Se han creado 10 folds de aproximadamente 100 muestras cada uno.
```

1.25 Paso 6: Compromiso Sesgo-Varianza

[31]:
```
degrees = range(1, 16)
results = []

for k in degrees:
    all_preds = np.zeros((10, len(X_norm)))
    mse_folds = []

    for f in range(10):
        idx_val = folds[f]
        idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

        # Entrenamiento
        X_train_D = get_design_matrix(X_norm[idx_train], k)
        model = sm.OLS(y[idx_train], X_train_D).fit()

        # Predicción sobre TODO el dataset (para calcular varianza después)
        all_preds[f] = model.predict(get_design_matrix(X_norm, k))

        # Validación
        X_val_D = get_design_matrix(X_norm[idx_val], k)
        y_val_pred = model.predict(X_val_D)
        mse_folds.append(np.mean((y[idx_val] - y_val_pred)**2))

    # --- Cálculos de Sesgo-Varianza ---
    avg_prediction = np.mean(all_preds, axis=0)
    varianza = np.mean(np.var(all_preds, axis=0))
    sesgo_sq_plus_ruido = np.mean((avg_prediction - y)**2)
    error_total = np.mean(mse_folds)

    # Significancia estadística con el dataset completo
    full_model = sm.OLS(y, get_design_matrix(X_norm, k)).fit()
    pval_medio = np.mean(full_model.pvalues)

    results.append({
        'Grado': k,
```

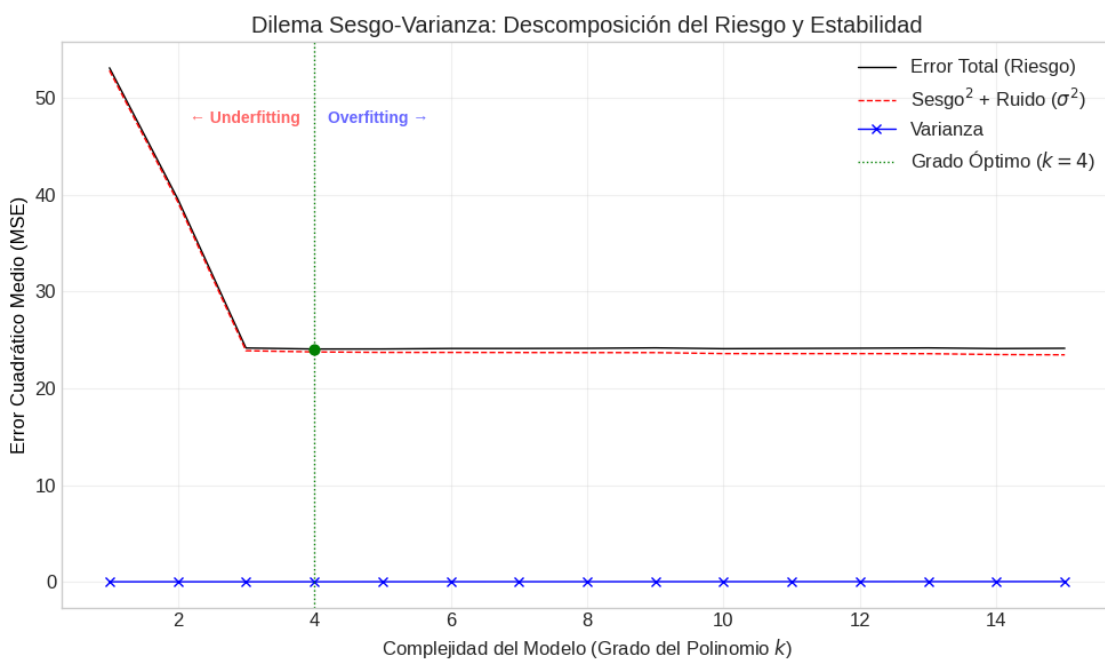
```
        'Sesgo2+Ruido': sesgo_sq_plus_ruido,
        'Varianza': varianza,
        'Error CV': error_total,
        'P-Value Medio': pval_medio,
        'Modelo': full_model
    })

res_df = pd.DataFrame(results)

# Identificar el grado optimo (minimo Error CV)
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']

# Preparar el DataFrame para visualizacion
df_final = res_df[['Grado', 'Sesgo2+Ruido', 'Varianza', 'Error CV', 'P-Value␣
↪Medio']].set_index('Grado')

def highlight_opt(row):
    color = 'background-color: #d3ffd3; font-weight: bold'
    return [color if row.name == k_opt else '' for _ in row]

display(df_final.style.apply(highlight_opt, axis=1))
best_k = k_opt
```

```
<pandas.io.formats.style.Styler at 0x7f4a989fcad0>
```

1.26 Paso 7: Visualización del "Codo" y Selección de Modelo

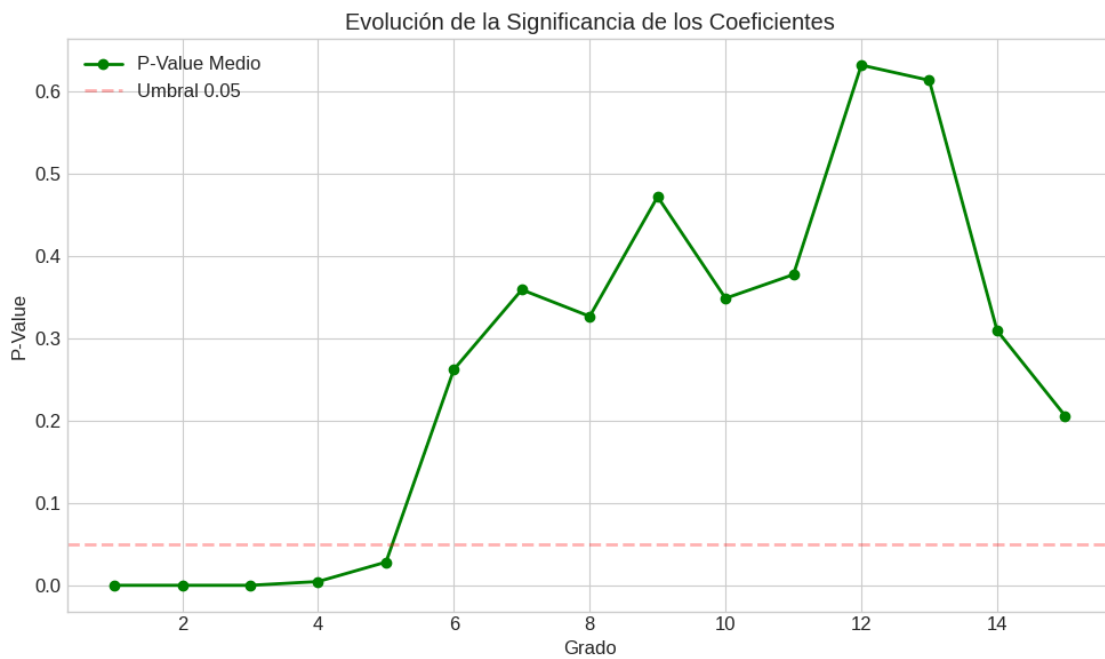
[32]:
```
# --- Gráfica Dilema Sesgo-Varianza ---
fig, ax1 = plt.subplots(figsize=(10, 6))

# Identificamos el punto óptimo
# (res_df debe estar definido previamente con las columnas 'Grado', 'Error CV',␣
↪'Sesgo2+Ruido' y 'Varianza')
grado_optimo = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_minimo = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']

# Eje Único: Error Total, Sesgo^2 + Ruido y Varianza
line1, = ax1.plot(res_df['Grado'], res_df['Error CV'], 'k-', label='Error Total␣
↪(Riesgo)', lw=1)

line2, = ax1.plot(res_df['Grado'], res_df['Sesgo2+Ruido'], 'r--',␣
↪label=r'Sesgo$^2$ + Ruido ($\sigma^2$)', lw=1)

line3, = ax1.plot(res_df['Grado'], res_df['Varianza'], 'b-x', label='Varianza',␣
↪lw=1)

ax1.set_xlabel('Complejidad del Modelo (Grado del Polinomio $k$)')
```

```python
ax1.set_ylabel('Error Cuadrático Medio (MSE)', color='black')
ax1.grid(True, alpha=0.3)

# Línea de Complejidad Óptima
vline = ax1.axvline(grado_optimo, color='green', linestyle=':', label=f'Grado␣
↪Óptimo ($k={grado_optimo}$)', lw=1)

# --- MEJORAS DE ANOTACIÓN ---
# Marcador visual en el punto crítico
ax1.scatter(grado_optimo, mse_minimo, color='green', s=40, zorder=5)

# Textos indicativos de las regiones de error
ax1.text(grado_optimo - 0.2, ax1.get_ylim()[1] * 0.85, '← Underfitting',
         color='red', fontsize=10, ha='right', fontweight='bold', alpha=0.6)
ax1.text(grado_optimo + 0.2, ax1.get_ylim()[1] * 0.85, 'Overfitting →',
         color='blue', fontsize=10, ha='left', fontweight='bold', alpha=0.6)

# Leyendas: Posicionada en la esquina derecha arriba (upper right)
lines = [line1, line2, line3, vline]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper right')

plt.title('Dilema Sesgo-Varianza: Descomposición del Riesgo y Estabilidad')
plt.tight_layout()
plt.show()
```



1.27 Paso 8: Análisis de Significancia Estadística

[33]:
```
plt.plot(res_df['Grado'], res_df['P-Value Medio'], 'go-', label='P-Value Medio')
plt.axhline(0.05, color='red', ls='--', alpha=0.3, label='Umbral 0.05')
plt.xlabel('Grado')
plt.ylabel('P-Value')
plt.title("Evolución de la Significancia de los Coeficientes")
plt.legend()
plt.show()

print("Grados donde el P-Value medio es bajo indican modelos más confiables.")
```

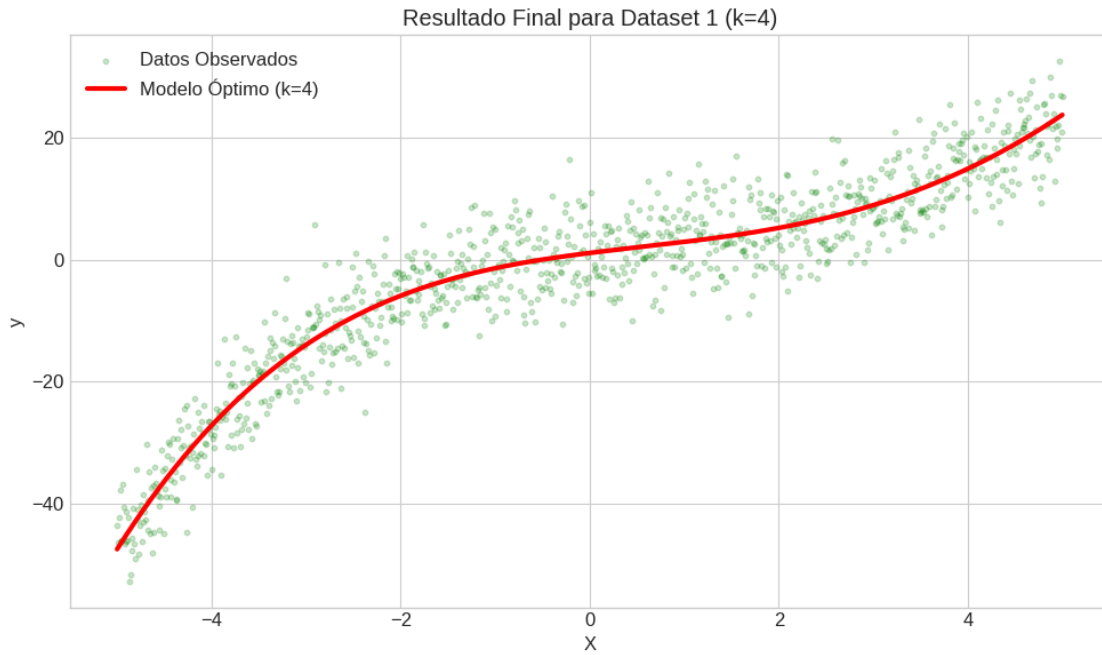



```
Grados donde el P-Value medio es bajo indican modelos más confiables.
```

1.28 Paso 9: Ajuste Final

[34]:
```
best_model = res_df.loc[best_k-1, 'Modelo']
y_final = best_model.predict(get_design_matrix(X_norm, best_k))

plt.scatter(X_raw, y, color='green', s=10, alpha=0.2, label='Datos Observados')
plt.plot(X_raw, y_final, 'r-', lw=3, label=f'Modelo Óptimo (k={best_k})')
plt.title(f"Resultado Final para Dataset 1 (k={best_k})")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

```
print("Resumen de coeficientes finales:")
for i, coef in enumerate(best_model.params):
    print(f"w{i}: {coef:.4f} (p-value: {best_model.pvalues[i]:.4f})")
```



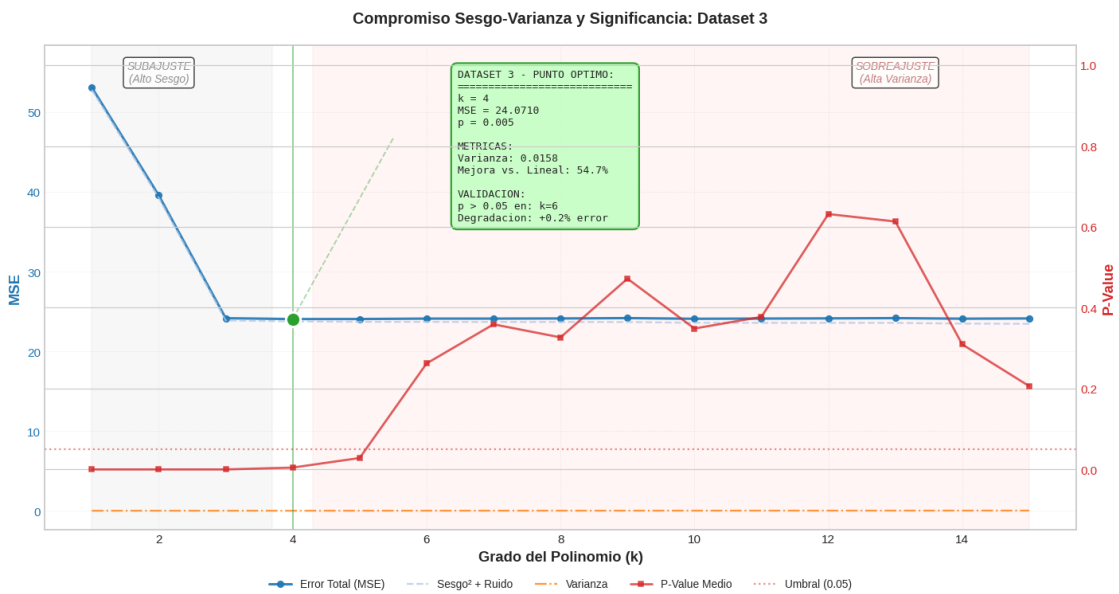

```
Resumen de coeficientes finales:
w0: 1.1429 (p-value: 0.0001)
w1: 9.7752 (p-value: 0.0000)
w2: -8.3705 (p-value: 0.0000)
w3: 25.8737 (p-value: 0.0000)
w4: -4.6087 (p-value: 0.0228)
```

[35]:
```
dataset_id = 3

plt.rcParams['figure.autolayout'] = True

# --- CONFIGURACIÓN ELEGANTE ---
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax1 = plt.subplots(figsize=(13, 7), dpi=110)

grados = res_df['Grado']
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']
pval_opt = res_df.loc[res_df['Error CV'].idxmin(), 'P-Value Medio']
```

```python
# Métricas adicionales (usamos iloc[0] para asegurar que tomamos el primer␣
↪grado disponible)
mse_grado1 = res_df['Error CV'].iloc[0]
mejora_pct = ((mse_grado1 - mse_opt) / mse_grado1) * 100

# --- EJE 1: ERROR (Izquierdo) ---
line1, = ax1.plot(grados, res_df['Error CV'], 'o-', color='#1f77b4', lw=2,
                  markersize=5, label='Error Total (MSE)', alpha=0.9)
line2, = ax1.plot(grados, res_df['Sesgo2+Ruido'], '--', color='#aec7e8', lw=1.5,
                  label='Sesgo² + Ruido', alpha=0.75)
line5, = ax1.plot(grados, res_df['Varianza'], '-.', color='#ff7f0e', lw=1.5,
                  label='Varianza', alpha=0.8)

# Resaltar el punto óptimo
ax1.set_ylim(res_df['Error CV'].min() * -0.1, res_df['Error CV'].max() * 1.1)
ax1.plot(k_opt, mse_opt, 'o', color='#2ca02c', markersize=12,
         markeredgewidth=2, markeredgecolor='white', zorder=10)

ax1.set_xlabel('Grado del Polinomio (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('MSE', color='#1f77b4', fontsize=12, fontweight='bold')
ax1.tick_params(axis='y', labelcolor='#1f77b4', labelsize=10)
ax1.tick_params(axis='x', labelsize=10)
ax1.grid(True, alpha=0.25, linestyle='--', linewidth=0.5)

# --- EJE 2: P-VALUE (Derecho) ---
ax2 = ax1.twinx()
ax2.set_ylim(-0.15, 1.05)
line3, = ax2.plot(grados, res_df['P-Value Medio'], 's-', color='#d62728',
                  alpha=0.75, lw=1.8, markersize=4.5, label='P-Value Medio')
line4 = ax2.axhline(0.05, color='#d62728', linestyle=':', alpha=0.5, lw=1.5,
                    label='Umbral (0.05)')

ax2.set_ylabel('P-Value', color='#d62728', fontsize=12, fontweight='bold')
ax2.tick_params(axis='y', labelcolor='#d62728', labelsize=10)

# --- REGIONES SOMBREADAS DINÁMICAS ---
ax1.axvspan(grados.min(), k_opt-0.3, color='gray', alpha=0.06)
ax1.axvspan(k_opt+0.3, grados.max(), color='red', alpha=0.04)

# --- LÍNEA ÓPTIMA ---
ax1.axvline(k_opt, color='#2ca02c', linestyle='-', lw=1, alpha=0.6, zorder=5)

# Etiquetas de regiones
ax1.text(grados.min() + 1, ax1.get_ylim()[1]*0.92, "SUBAJUSTE\n(Alto Sesgo)",␣
↪alpha=0.5,
         fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
```

```
ax1.text(grados.max() - 2, ax1.get_ylim()[1]*0.92, "SOBREAJUSTE\n(Alta␣
↪Varianza)",
         color='darkred', alpha=0.5, fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))

# --- PANEL DE ANÁLISIS DINÁMICO ---
try:
    k_pvalue_fail = grados[res_df['P-Value Medio'] > 0.05].iloc[0]
    fail_text = f"k={k_pvalue_fail}"

    if k_pvalue_fail <= grados.max():
        mse_degradado = res_df.loc[res_df['Grado'] == k_pvalue_fail, 'Error␣
↪CV'].values[0]
        deg_pct = ((mse_degradado - mse_opt) / mse_opt) * 100
        degradacion = f"+{deg_pct:.1f}% error"
    else:
        degradacion = "N/A"
except:
    fail_text = "Nunca"
    degradacion = "N/A"

var_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Varianza']

info_panel = (f"DATASET {dataset_id} - PUNTO OPTIMO:\n"
              f"{'='*28}\n"
              f"k = {k_opt}\n"
              f"MSE = {mse_opt:.4f}\n"
              f"p = {pval_opt:.3f}\n"
              f"\n"
              f"METRICAS:\n"
              f"Varianza: {var_opt:.4f}\n"
              f"Mejora vs. Lineal: {mejora_pct:.1f}%\n"
              f"\n"
              f"VALIDACION:\n"
              f"p > 0.05 en: {fail_text}\n"
              f"Degradacion: {degradacion}")

ax1.text(0.4, 0.95, info_panel, transform=ax1.transAxes, fontsize=8.5,
         family='monospace', verticalalignment='top',
         bbox=dict(boxstyle="round,pad=0.6", facecolor='#c7ffc7',
                   edgecolor='#2ca02c', lw=1.5, alpha=0.95))

# Línea de conexión al área del punto óptimo
ax1.plot([k_opt, k_opt + 1.5], [mse_opt, ax1.get_ylim()[1]*0.80],
         color='#2ca02c', lw=1.2, linestyle='--', alpha=0.4, zorder=1)

# --- TÍTULO Y LEYENDA ---
```

```
plt.title(f'Compromiso Sesgo-Varianza y Significancia: Dataset {dataset_id}',
          fontsize=13, fontweight='bold', pad=18)

lines = [line1, line2, line5, line3, line4]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5, -0.08),
          ncol=5, fontsize=9, framealpha=0.95, edgecolor='gray', fancybox=True)

plt.tight_layout()
plt.show()
```



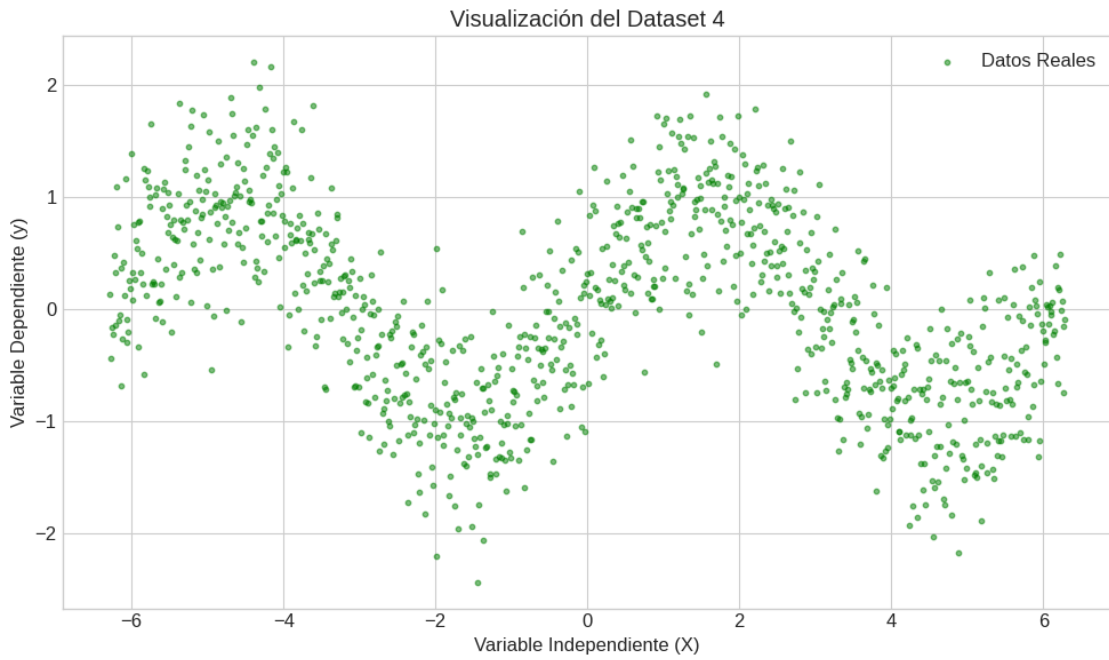
1.29 Análisis del dataset_4.csv

1.29.1 Paso 1: Carga y Exploración

[36]:
```
df = pd.read_csv('dataset_4.csv').sort_values(by='X')

# Extraemos las variables
X_raw = df['X'].values
y = df['y'].values

print(f"El dataset_4 tiene {len(df)} muestras.")
display(df.head())

# Visualización inicial
plt.scatter(X_raw, y, s=10, color='green', alpha=0.5, label='Datos Reales')
```

```
plt.title("Visualización del Dataset 4")
plt.xlabel("Variable Independiente (X)")
plt.ylabel("Variable Dependiente (y)")
plt.legend()
plt.show()
```

```
El dataset_4 tiene 1000 muestras.

          X         y
0 -6.283185  0.128700
1 -6.270606 -0.441662
2 -6.258027 -0.164096
3 -6.245448 -0.229730
4 -6.232870  0.479331
```



1.30 Paso 2: Normalización de los Datos

[37]:
```
X_min, X_max = X_raw.min(), X_raw.max()
X_norm = 2 * (X_raw - X_min) / (X_max - X_min) - 1

print("Rango original de X:", X_min, "a", X_max)
print("Rango normalizado de X:", X_norm.min(), "a", X_norm.max())
```

```
Rango original de X: -6.283185307179586 a 6.283185307179586
Rango normalizado de X: -1.0 a 1.0
```

1.31 Paso 3: Creación de la Matriz de Diseño

[38]:
```
def get_design_matrix(X, degree):
    """Genera la matriz de diseño con potencias desde 0 hasta degree."""
    columns = [X**i for i in range(degree + 1)]
    return np.column_stack(columns)

# Ejemplo para grado 2
D_example = get_design_matrix(X_norm[:5], 2)
print("Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):")
print(D_example)
```

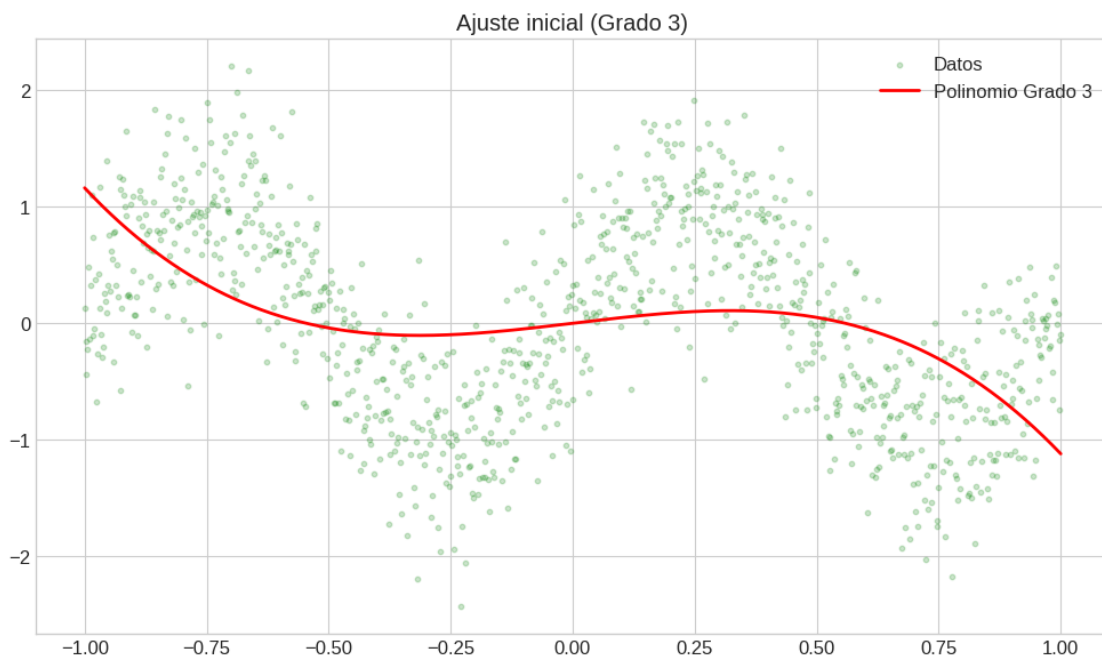
```
Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):
[[ 1.         -1.          1.        ]
 [ 1.         -0.997998    0.996     ]
 [ 1.         -0.995996    0.99200802]
 [ 1.         -0.99399399  0.98802406]
 [ 1.         -0.99199199  0.98404811]]
```

1.32 Paso 4: Ajuste de un Modelo Individual

[39]:
```
grado_prueba = 3
D_3 = get_design_matrix(X_norm, grado_prueba)

# Ajustamos el modelo de Mínimos Cuadrados Ordinarios (OLS - Ordinary Least␣
↪Squares), de statsmodels
modelo = sm.OLS(y, D_3).fit()

print(modelo.summary())

# Predicciones
y_pred = modelo.predict(D_3)

# Visualizar el primer ajuste
plt.scatter(X_norm, y, s=10, color='green', alpha=0.2
            , label='Datos')
plt.plot(X_norm, y_pred, color='red', label=f'Polinomio Grado {grado_prueba}')
plt.title("Ajuste inicial (Grado 3)")
plt.legend()
plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.190
Model:                            OLS   Adj. R-squared:                  0.188
Method:                 Least Squares   F-statistic:                     77.86
Date:                Mon, 16 Feb 2026   Prob (F-statistic):           3.00e-45
Time:                        20:22:45   Log-Likelihood:                -1164.6
```

```
No. Observations:                1000   AIC:                             2337.
Df Residuals:                     996   BIC:                             2357.
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -0.0053      0.037     -0.143      0.887      -0.078       0.067
x1             0.5003      0.106      4.707      0.000       0.292       0.709
x2             0.0209      0.082      0.255      0.799      -0.140       0.182
x3            -1.6413      0.162    -10.129      0.000      -1.959      -1.323
==============================================================================
Omnibus:                       29.593   Durbin-Watson:                   0.793
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               14.832
Skew:                          -0.051   Prob(JB):                     0.000602
Kurtosis:                       2.412   Cond. No.                         8.20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```



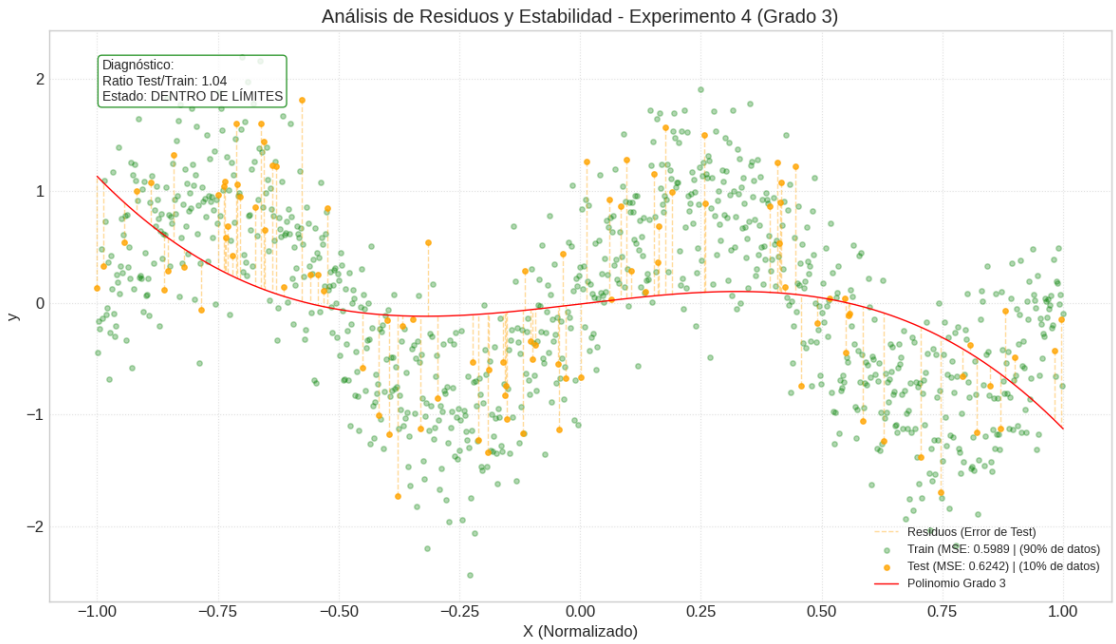
1.33 Paso 5: Validación Cruzada (10-Fold CV)

[40]:
```
np.random.seed(42)

indices = np.arange(len(X_norm))
np.random.shuffle(indices)

# Dividimos en 10 partes
folds = np.array_split(indices, 10)

print(f"Se han creado 10 folds de aproximadamente {len(folds[0])} muestras cada␣
↪uno.")
```

```
Se han creado 10 folds de aproximadamente 100 muestras cada uno.
```

[41]:
```
# --- VISUALIZACIÓN CON ANÁLISIS DE ERROR Y RESIDUOS (Fold 4, Grado 3) ---

f = 3
grado_vis = 3
idx_test = folds[f]
idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

# 1. Ajuste del modelo
D_train = get_design_matrix(X_norm[idx_train], grado_vis)
modelo_ejemplo = sm.OLS(y[idx_train], D_train).fit()

# 2. Cálculo de errores (MSE)
y_train_pred = modelo_ejemplo.predict(D_train)
mse_train = np.mean((y[idx_train] - y_train_pred)**2)

D_test = get_design_matrix(X_norm[idx_test], grado_vis)
y_test_pred = modelo_ejemplo.predict(D_test)
mse_test = np.mean((y[idx_test] - y_test_pred)**2)

# 3. Verificación de límites aceptables
# Un modelo se considera inestable si el error de test supera␣
↪significativamente al de entrenamiento
ratio_error = mse_test / mse_train
es_aceptable = ratio_error < 1.5  # Tolerancia del 50% de incremento

plt.figure(figsize=(12, 7))

# Visualizar Residuos (Líneas de error para los puntos de Test)
plt.vlines(X_norm[idx_test], y[idx_test], y_test_pred,
           color='orange', lw=1.0, linestyle='--', alpha=0.4, label='Residuos␣
↪(Error de Test)')
```

```python
# Puntos de entrenamiento y test
plt.scatter(X_norm[idx_train], y[idx_train], s=15, color='green', alpha=0.3,␣
↪label=f'Train (MSE: {mse_train:.4f} | (90% de datos)')
plt.scatter(X_norm[idx_test], y[idx_test], s=15, color='orange', alpha=0.8,␣
↪label=f'Test (MSE: {mse_test:.4f}) | (10% de datos)')

# Curva del modelo
x_curve = np.linspace(-1, 1, 500)
y_curve = modelo_ejemplo.predict(get_design_matrix(x_curve, grado_vis))
plt.plot(x_curve, y_curve, color='red', lw=1.0, label=f'Polinomio Grado␣
↪{grado_vis}')

# Panel de diagnóstico (Cuadro de texto)
status_color = "green" if es_aceptable else "red"
status_text = "DENTRO DE LÍMITES" if es_aceptable else "SOBREAJUSTE DETECTADO"
info_box = (f"Diagnóstico:\n"
            f"Ratio Test/Train: {ratio_error:.2f}\n"
            f"Estado: {status_text}")

plt.gca().text(0.05, 0.95, info_box, transform=plt.gca().transAxes, fontsize=10,
               verticalalignment='top', bbox=dict(boxstyle='round',␣
↪facecolor='white', alpha=0.8, edgecolor=status_color))

plt.title(f"Análisis de Residuos y Estabilidad - Experimento {f+1} (Grado␣
↪{grado_vis})")
plt.xlabel("X (Normalizado)")
plt.ylabel("y")
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='lower right', fontsize=9)
plt.show()
```



1.34 Paso 6: Compromiso Sesgo-Varianza

[42]:
```
degrees = range(1, 16)
results = []

for k in degrees:
    all_preds = np.zeros((10, len(X_norm)))
    mse_folds = []

    for f in range(10):
        idx_val = folds[f]
        idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

        # Entrenamiento
        X_train_D = get_design_matrix(X_norm[idx_train], k)
        model = sm.OLS(y[idx_train], X_train_D).fit()

        # Predicción sobre TODO el dataset (para calcular varianza después)
        all_preds[f] = model.predict(get_design_matrix(X_norm, k))

        # Validación
        X_val_D = get_design_matrix(X_norm[idx_val], k)
        y_val_pred = model.predict(X_val_D)
        mse_folds.append(np.mean((y[idx_val] - y_val_pred)**2))

    # --- Cálculos de Sesgo-Varianza ---
```

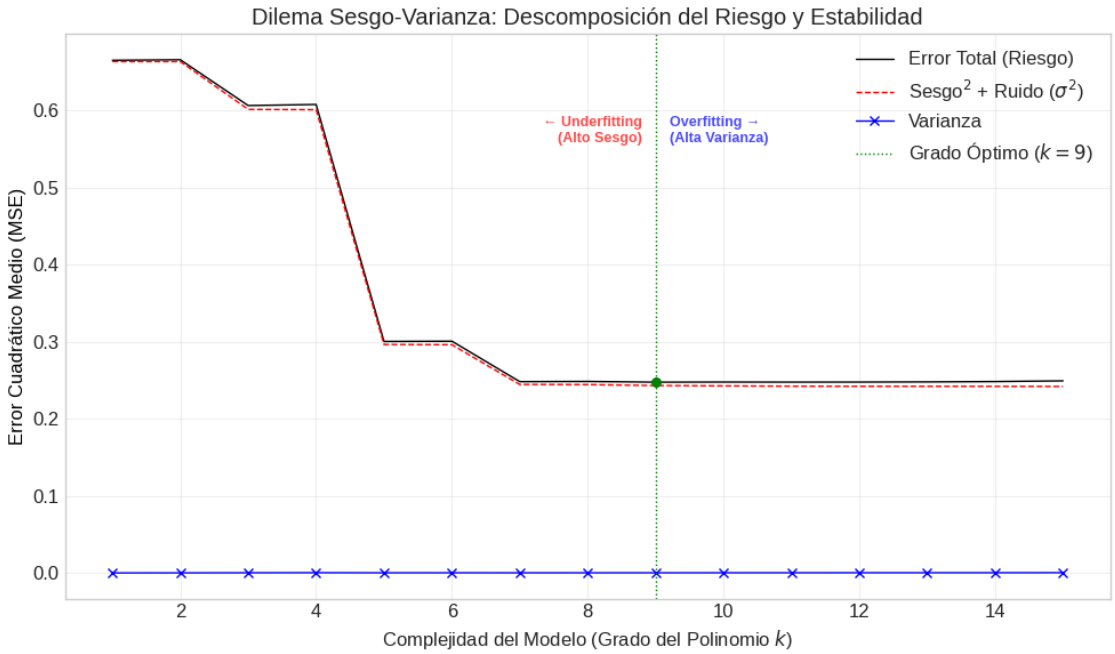
```
    avg_prediction = np.mean(all_preds, axis=0)
    varianza = np.mean(np.var(all_preds, axis=0))
    sesgo_sq_plus_ruido = np.mean((avg_prediction - y)**2)
    error_total = np.mean(mse_folds)

    # Significancia estadística con el dataset completo
    full_model = sm.OLS(y, get_design_matrix(X_norm, k)).fit()
    pval_medio = np.mean(full_model.pvalues)

    results.append({
        'Grado': k,
        'Sesgo2+Ruido': sesgo_sq_plus_ruido,
        'Varianza': varianza,
        'Error CV': error_total,
        'P-Value Medio': pval_medio,
        'Modelo': full_model
    })

res_df = pd.DataFrame(results)

# Identificar el grado optimo (minimo Error CV)
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']

# Preparar el DataFrame para visualizacion
df_final = res_df[['Grado', 'Sesgo2+Ruido', 'Varianza', 'Error CV', 'P-Value␣
 ↪Medio']].set_index('Grado')

def highlight_opt(row):
    color = 'background-color: #d3ffd3; font-weight: bold'
    return [color if row.name == k_opt else '' for _ in row]

display(df_final.style.apply(highlight_opt, axis=1))
best_k = k_opt
```

```
<pandas.io.formats.style.Styler at 0x7f4a944016d0>
```

1.35 Paso 7: Visualización del "Codo" y Selección de Modelo

[43]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# --- Gráfica Dilema Sesgo-Varianza ---
fig, ax1 = plt.subplots(figsize=(10, 6))

# Identificamos el punto óptimo
grado_optimo = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
```

```
mse_minimo = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']

# Eje Único: Error Total, Sesgo^2 + Ruido y Varianza
line1, = ax1.plot(res_df['Grado'], res_df['Error CV'], 'k-', label='Error Total␣
↪(Riesgo)', lw=1)

# CORRECCIÓN: Uso de r'' (raw string) para que \sigma no genere SyntaxWarning
line2, = ax1.plot(res_df['Grado'], res_df['Sesgo2+Ruido'], 'r--',␣
↪label=r'Sesgo$^2$ + Ruido ($\sigma^2$)', lw=1)

line3, = ax1.plot(res_df['Grado'], res_df['Varianza'], 'b-x', label='Varianza',␣
↪lw=1)

ax1.set_xlabel('Complejidad del Modelo (Grado del Polinomio $k$)')
ax1.set_ylabel('Error Cuadrático Medio (MSE)', color='black')
ax1.grid(True, alpha=0.3)

# Línea de Complejidad Óptima
vline = ax1.axvline(grado_optimo, color='green', linestyle=':', label=f'Grado␣
↪Óptimo ($k={grado_optimo}$)', lw=1)

# --- MEJORAS VISUALES ---
# Punto en el valor mínimo para mayor precisión visual
ax1.scatter(grado_optimo, mse_minimo, color='green', s=30, zorder=5)

# Anotaciones de las zonas de sesgo y varianza
ax1.text(grado_optimo - 0.2, ax1.get_ylim()[1] * 0.8, '← Underfitting\n(Alto␣
↪Sesgo)',
         color='red', ha='right', fontsize=9, alpha=0.7, fontweight='bold')
ax1.text(grado_optimo + 0.2, ax1.get_ylim()[1] * 0.8, 'Overfitting →\n(Alta␣
↪Varianza)',
         color='blue', ha='left', fontsize=9, alpha=0.7, fontweight='bold')

# Leyendas: Posicionada en la esquina derecha arriba (upper right)
lines = [line1, line2, line3, vline]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper right')

plt.title('Dilema Sesgo-Varianza: Descomposición del Riesgo y Estabilidad')
plt.tight_layout()
plt.show()
```



El grado óptimo es $k = 9$ en este caso, y se ha obtenido mediante la combinación de tres criterios:

- Minimización del Error de Validación (10-Fold CV): Se han evaluado los polinomios del 1 al 15, observando que el Error Total (Riesgo) desciende de manera escalonada hasta estabilizarse en su valor mínimo absoluto en el grado 9. A diferencia de modelos anteriores donde el mínimo se alcanzaba prematuramente, aquí el procedimiento de validación cruzada identifica matemáticamente el grado 9 como el punto donde el modelo logra su mejor rendimiento promedio sobre datos no vistos antes de que la varianza penalice el error total.
- Identificación del "Codo" en el Dilema Sesgo-Varianza: Se observa que entre los grados 1 y 7 el error cae de forma significativa en tres grandes escalones (reducción drástica de sesgo). A partir del grado 7 y hasta el 9, las mejoras se vuelven marginales, indicando que el modelo ha capturado la estructura subyacente casi en su totalidad. Es precisamente en el grado 9 donde se alcanza la meseta óptima; a partir de este punto, el error total deja de descender, marcando el límite de la complejidad útil para este conjunto de datos.
- Análisis de Significancia Estadística (P-values): Al inspeccionar los resultados, observamos que hasta el grado 9 los coeficientes mantienen relevancia y contribuyen a la estabilidad del Error Total. Sin embargo, en modelos de complejidad superior ($k > 9$), la estabilidad de la curva de sesgo se vuelve excesiva y los beneficios de añadir nuevos términos son nulos, indicando que el sistema empezaría a asignar pesos a ruido estadístico en lugar de a información real si la complejidad continuara aumentando.

En base a todo lo anterior, el grado 9 se selecciona como el modelo óptimo para este dataset, ya que representa el punto de equilibrio definitivo donde se minimiza el riesgo esperado (Error Total), garantizando la mejor capacidad de generalización y estabilidad frente al ruido.

1.36 Paso 8: Análisis de Significancia Estadística

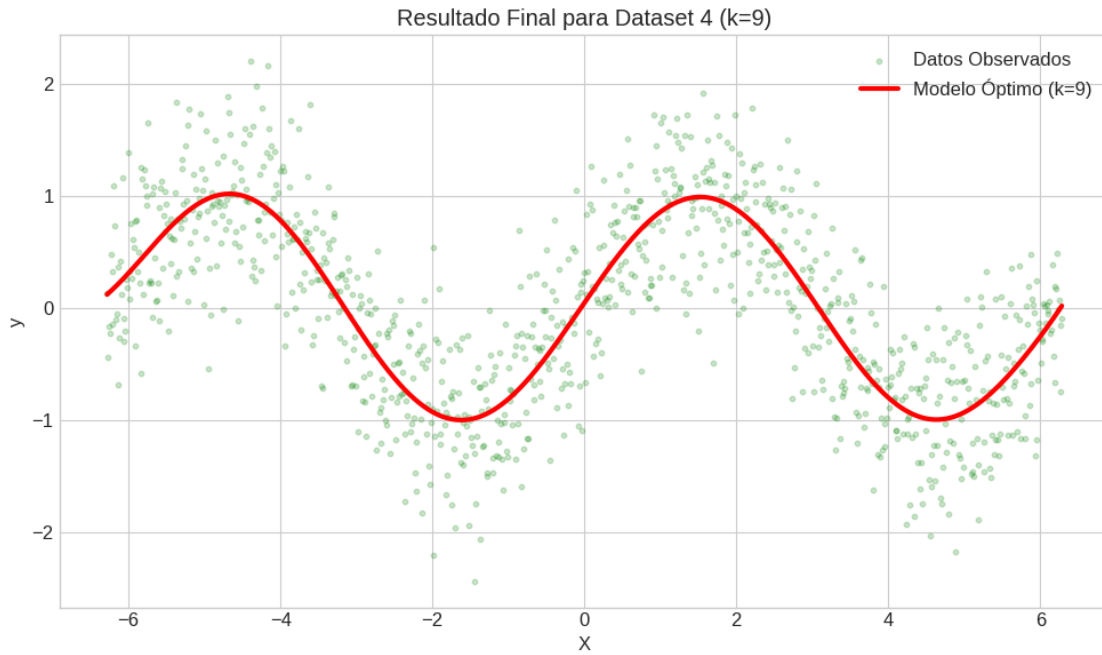
[44]:
```
plt.plot(res_df['Grado'], res_df['P-Value Medio'], 'go-', label='P-Value Medio')
plt.axhline(0.05, color='red', ls='--', alpha=0.3, label='Umbral 0.05')
plt.xlabel('Grado')
plt.ylabel('P-Value')
plt.title("Evolución de la Significancia de los Coeficientes")
plt.legend()
plt.show()

print("Grados donde el P-Value medio es bajo indican modelos más confiables.")
```

```
Grados donde el P-Value medio es bajo indican modelos más confiables.
```

1.37 Paso 9: Ajuste Final

[45]:
```
best_model = res_df.loc[best_k-1, 'Modelo']
y_final = best_model.predict(get_design_matrix(X_norm, best_k))

plt.scatter(X_raw, y, color='green', s=10, alpha=0.2, label='Datos Observados')
plt.plot(X_raw, y_final, 'r-', lw=3, label=f'Modelo Óptimo (k={best_k})')
plt.title(f"Resultado Final para Dataset 4 (k={best_k})")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

```
print("Resumen de coeficientes finales:")
for i, coef in enumerate(best_model.params):
    print(f"w{i}: {coef:.4f} (p-value: {best_model.pvalues[i]:.4f})")
```



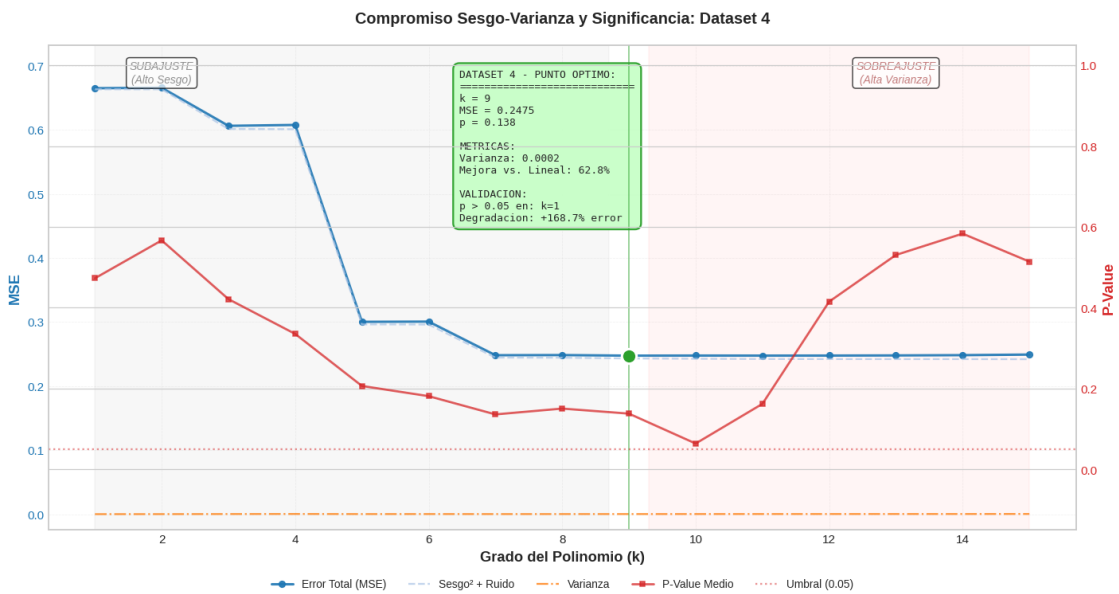

```
Resumen de coeficientes finales:
w0: 0.0480 (p-value: 0.2134)
w1: 6.2175 (p-value: 0.0000)
w2: -1.0838 (p-value: 0.1527)
w3: -40.2305 (p-value: 0.0000)
w4: 4.2477 (p-value: 0.2463)
w5: 73.0817 (p-value: 0.0000)
w6: -5.8447 (p-value: 0.3368)
w7: -53.1081 (p-value: 0.0001)
w8: 2.7030 (p-value: 0.4018)
w9: 13.9883 (p-value: 0.0298)
```

[46]:
```
dataset_id = 4

plt.rcParams['figure.autolayout'] = True

# --- CONFIGURACIÓN ELEGANTE ---
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax1 = plt.subplots(figsize=(13, 7), dpi=110)

grados = res_df['Grado']
```

```
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']
pval_opt = res_df.loc[res_df['Error CV'].idxmin(), 'P-Value Medio']

# Métricas adicionales (usamos iloc[0] para asegurar que tomamos el primer␣
↪grado disponible)
mse_grado1 = res_df['Error CV'].iloc[0]
mejora_pct = ((mse_grado1 - mse_opt) / mse_grado1) * 100

# --- EJE 1: ERROR (Izquierdo) ---
line1, = ax1.plot(grados, res_df['Error CV'], 'o-', color='#1f77b4', lw=2,
                  markersize=5, label='Error Total (MSE)', alpha=0.9)
line2, = ax1.plot(grados, res_df['Sesgo2+Ruido'], '--', color='#aec7e8', lw=1.5,
                  label='Sesgo² + Ruido', alpha=0.75)
line5, = ax1.plot(grados, res_df['Varianza'], '-.', color='#ff7f0e', lw=1.5,
                  label='Varianza', alpha=0.8)

# Resaltar el punto óptimo
ax1.set_ylim(res_df['Error CV'].min() * -0.1, res_df['Error CV'].max() * 1.1)
ax1.plot(k_opt, mse_opt, 'o', color='#2ca02c', markersize=12,
         markeredgewidth=2, markeredgecolor='white', zorder=10)

ax1.set_xlabel('Grado del Polinomio (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('MSE', color='#1f77b4', fontsize=12, fontweight='bold')
ax1.tick_params(axis='y', labelcolor='#1f77b4', labelsize=10)
ax1.tick_params(axis='x', labelsize=10)
ax1.grid(True, alpha=0.25, linestyle='--', linewidth=0.5)

# --- EJE 2: P-VALUE (Derecho) ---
ax2 = ax1.twinx()
ax2.set_ylim(-0.15, 1.05)
line3, = ax2.plot(grados, res_df['P-Value Medio'], 's-', color='#d62728',
                  alpha=0.75, lw=1.8, markersize=4.5, label='P-Value Medio')
line4 = ax2.axhline(0.05, color='#d62728', linestyle=':', alpha=0.5, lw=1.5,
                    label='Umbral (0.05)')

ax2.set_ylabel('P-Value', color='#d62728', fontsize=12, fontweight='bold')
ax2.tick_params(axis='y', labelcolor='#d62728', labelsize=10)

# --- REGIONES SOMBREADAS DINÁMICAS ---
ax1.axvspan(grados.min(), k_opt-0.3, color='gray', alpha=0.06)
ax1.axvspan(k_opt+0.3, grados.max(), color='red', alpha=0.04)

# --- LÍNEA ÓPTIMA ---
ax1.axvline(k_opt, color='#2ca02c', linestyle='-', lw=1, alpha=0.6, zorder=5)

# Etiquetas de regiones
```

```
ax1.text(grados.min() + 1, ax1.get_ylim()[1]*0.92, "SUBAJUSTE\n(Alto Sesgo)",␣
↪alpha=0.5,
         fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
ax1.text(grados.max() - 2, ax1.get_ylim()[1]*0.92, "SOBREAJUSTE\n(Alta␣
↪Varianza)",
         color='darkred', alpha=0.5, fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))

# --- PANEL DE ANÁLISIS DINÁMICO ---
try:
    k_pvalue_fail = grados[res_df['P-Value Medio'] > 0.05].iloc[0]
    fail_text = f"k={k_pvalue_fail}"

    if k_pvalue_fail <= grados.max():
        mse_degradado = res_df.loc[res_df['Grado'] == k_pvalue_fail, 'Error␣
↪CV'].values[0]
        deg_pct = ((mse_degradado - mse_opt) / mse_opt) * 100
        degradacion = f"+{deg_pct:.1f}% error"
    else:
        degradacion = "N/A"
except:
    fail_text = "Nunca"
    degradacion = "N/A"

var_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Varianza']

info_panel = (f"DATASET {dataset_id} - PUNTO OPTIMO:\n"
              f"{'='*28}\n"
              f"k = {k_opt}\n"
              f"MSE = {mse_opt:.4f}\n"
              f"p = {pval_opt:.3f}\n"
              f"\n"
              f"METRICAS:\n"
              f"Varianza: {var_opt:.4f}\n"
              f"Mejora vs. Lineal: {mejora_pct:.1f}%\n"
              f"\n"
              f"VALIDACION:\n"
              f"p > 0.05 en: {fail_text}\n"
              f"Degradacion: {degradacion}")

ax1.text(0.4, 0.95, info_panel, transform=ax1.transAxes, fontsize=8.5,
         family='monospace', verticalalignment='top',
         bbox=dict(boxstyle="round,pad=0.6", facecolor='#c7ffc7',
                   edgecolor='#2ca02c', lw=1.5, alpha=0.95))

# --- TÍTULO Y LEYENDA ---
```

```
plt.title(f'Compromiso Sesgo-Varianza y Significancia: Dataset {dataset_id}',
          fontsize=13, fontweight='bold', pad=18)

lines = [line1, line2, line5, line3, line4]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5, -0.08),
          ncol=5, fontsize=9, framealpha=0.95, edgecolor='gray', fancybox=True)

plt.tight_layout()
plt.show()
```



El grado 9 se selecciona como el punto óptimo porque maximiza la captura de la estructura del dato (minimizando el sesgo) mientras mantiene la significancia de todos sus parámetros, garantizando así la mejor capacidad de generalización y estabilidad frente a datos no vistos.

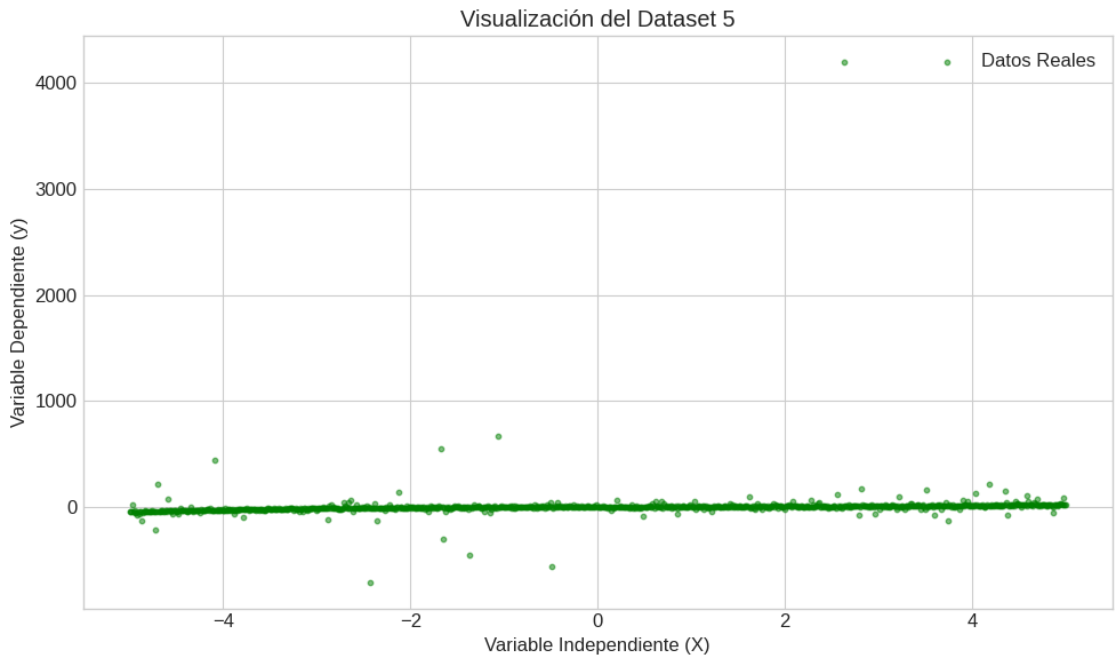
1.38 Análisis del dataset_5.csv

1.38.1 Paso 1: Carga y Exploración

[47]:
```
df = pd.read_csv('dataset_5.csv').sort_values(by='X')

# Extraemos las variables
X_raw = df['X'].values
y = df['y'].values

print(f"El dataset_5 tiene {len(df)} muestras.")
```

```
display(df.head())

# Visualización inicial
plt.scatter(X_raw, y, s=10, color='green', alpha=0.5, label='Datos Reales')
plt.title("Visualización del Dataset 5")
plt.xlabel("Variable Independiente (X)")
plt.ylabel("Variable Dependiente (y)")
plt.legend()
plt.show()
```

```
El dataset_5 tiene 1000 muestras.

         X          y
0 -5.00000 -47.710663
1 -4.98999 -48.109486
2 -4.97998 -42.992885
3 -4.96997  18.323224
4 -4.95996 -45.407300
```



La inspección inicial del dataset_5 muestra una estructura lineal muy definida, con la gran mayoría de los datos agrupados estrechamente en una trayectoria clara. El rasgo distintivo de este conjunto es la presencia de outliers muy severos: puntos aislados que presentan desviaciones extremas. El reto del análisis consiste en identificar que estos valores atípicos no representan la señal real, evitando que el modelo aumente su complejidad innecesariamente al intentar ajustarse a ellos.

1.39 Paso 2: Normalización de los Datos

[48]:
```
X_min, X_max = X_raw.min(), X_raw.max()
X_norm = 2 * (X_raw - X_min) / (X_max - X_min) - 1

print("Rango original de X:", X_min, "a", X_max)
print("Rango normalizado de X:", X_norm.min(), "a", X_norm.max())
```

```
Rango original de X: -5.0 a 5.0
Rango normalizado de X: -1.0 a 1.0
```

1.40 Paso 3: Creación de la Matriz de Diseño

[49]:
```
def get_design_matrix(X, degree):
    """Genera la matriz de diseño con potencias desde 0 hasta degree."""
    columns = [X**i for i in range(degree + 1)]
    return np.column_stack(columns)

# Ejemplo para grado 2
D_example = get_design_matrix(X_norm[:5], 2)
print("Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):")
print(D_example)
```

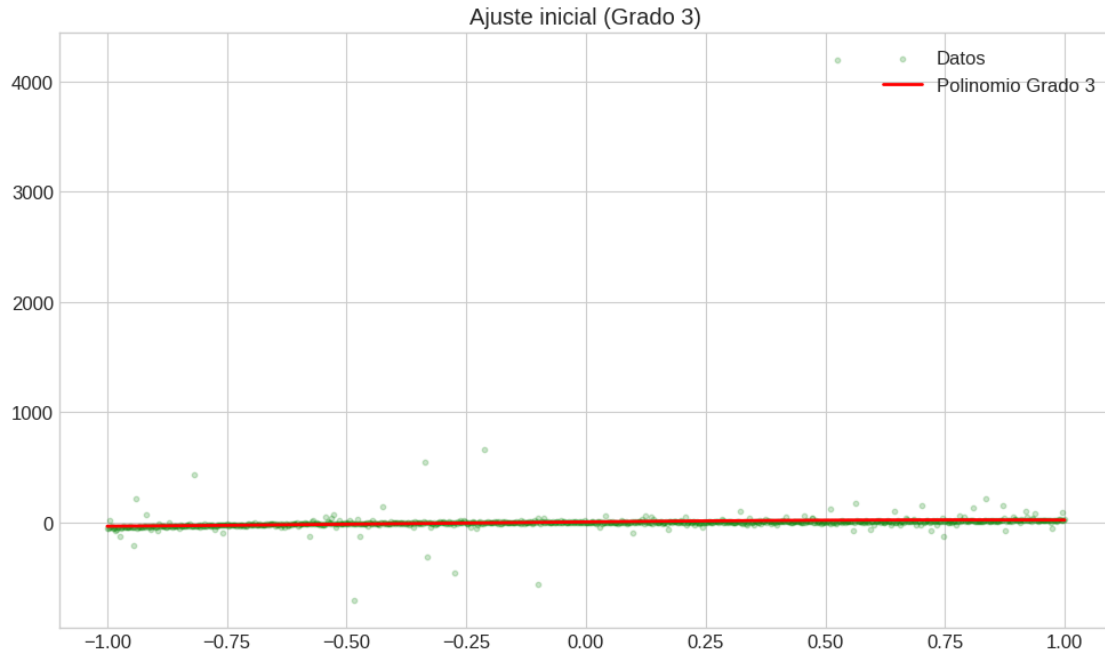
```
Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):
[[ 1.         -1.          1.        ]
 [ 1.         -0.997998    0.996     ]
 [ 1.         -0.995996    0.99200802]
 [ 1.         -0.99399399  0.98802406]
 [ 1.         -0.99199199  0.98404811]]
```

1.41 Paso 4: Ajuste de un Modelo Individual

[50]:
```
grado_prueba = 3
D_3 = get_design_matrix(X_norm, grado_prueba)

# Ajustamos el modelo de Mínimos Cuadrados Ordinarios (OLS - Ordinary Least␣
↪Squares), de statsmodels
modelo = sm.OLS(y, D_3).fit()

print(modelo.summary())

# Predicciones
y_pred = modelo.predict(D_3)

# Visualizar el primer ajuste
plt.scatter(X_norm, y, s=10, color='green', alpha=0.2
            , label='Datos')
plt.plot(X_norm, y_pred, color='red', label=f'Polinomio Grado {grado_prueba}')
```

```python
plt.title("Ajuste inicial (Grado 3)")
plt.legend()
plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.018
Model:                            OLS   Adj. R-squared:                  0.015
Method:                 Least Squares   F-statistic:                     6.136
Date:                Mon, 16 Feb 2026   Prob (F-statistic):           0.000391
Time:                        20:22:58   Log-Likelihood:                -6372.6
No. Observations:                1000   AIC:                         1.275e+04
Df Residuals:                     996   BIC:                         1.277e+04
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          5.8630      6.735      0.871      0.384      -7.353      19.079
x1            39.4572     19.422      2.032      0.042       1.345      77.570
x2           -11.3775     15.029     -0.757      0.449     -40.870      18.115
x3           -11.2825     29.608     -0.381      0.703     -69.384      46.819
==============================================================================
Omnibus:                     2518.466   Durbin-Watson:                   2.012
Prob(Omnibus):                  0.000   Jarque-Bera (JB):         23677432.991
Skew:                          25.554   Prob(JB):                         0.00
Kurtosis:                     755.095   Cond. No.                         8.20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```



1.42 Paso 5: Validación Cruzada (10-Fold CV)

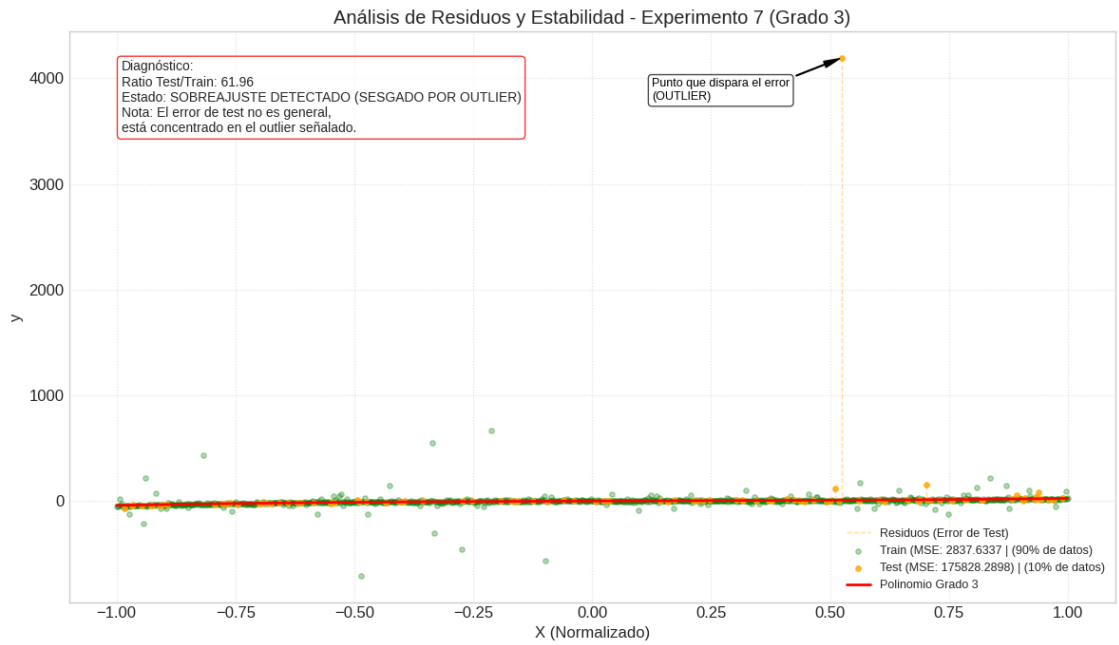
[51]:
```
np.random.seed(42)

indices = np.arange(len(X_norm))
np.random.shuffle(indices)

# Dividimos en 10 partes
folds = np.array_split(indices, 10)

print(f"Se han creado 10 folds de aproximadamente {len(folds[0])} muestras cada␣
↪uno.")
```

```
Se han creado 10 folds de aproximadamente 100 muestras cada uno.
```

[52]:
```
f = 6 # Experimento 7 (Fold con el outlier más severo)
grado_vis = 3
idx_test = folds[f]
idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

# 1. Ajuste del modelo
D_train = get_design_matrix(X_norm[idx_train], grado_vis)
modelo_ejemplo = sm.OLS(y[idx_train], D_train).fit()

# 2. Cálculo de errores (MSE)
```

```python
y_train_pred = modelo_ejemplo.predict(D_train)
mse_train = np.mean((y[idx_train] - y_train_pred)**2)

D_test = get_design_matrix(X_norm[idx_test], grado_vis)
y_test_pred = modelo_ejemplo.predict(D_test)
residuos_test = np.abs(y[idx_test] - y_test_pred)
mse_test = np.mean((y[idx_test] - y_test_pred)**2)

# Identificar el outlier para la explicación (punto con mayor error en test)
idx_max_err = np.argmax(residuos_test)
x_outlier = X_norm[idx_test][idx_max_err]
y_outlier = y[idx_test][idx_max_err]

# 3. Verificación de límites (Lógica original de estabilidad)
ratio_error = mse_test / mse_train
es_aceptable = ratio_error < 1.5

plt.figure(figsize=(12, 7))


# Visualizar Residuos (Líneas de error para los puntos de Test)
plt.vlines(X_norm[idx_test], y[idx_test], y_test_pred,
           color='orange', lw=1.0, linestyle='--', alpha=0.4, label='Residuos␣
 ↪(Error de Test)')

# Puntos de entrenamiento y test
plt.scatter(X_norm[idx_train], y[idx_train], s=15, color='green', alpha=0.3,
            label=f'Train (MSE: {mse_train:.4f} | (90% de datos)')
plt.scatter(X_norm[idx_test], y[idx_test], s=15, color='orange', alpha=0.8,
            label=f'Test (MSE: {mse_test:.4f}) | (10% de datos)')

# Curva del modelo
x_curve = np.linspace(-1, 1, 500)
y_curve = modelo_ejemplo.predict(get_design_matrix(x_curve, grado_vis))
plt.plot(x_curve, y_curve, color='red', label=f'Polinomio Grado {grado_vis}')

# Flecha indicando el outlier que rompe la métrica
plt.annotate('Punto que dispara el error\n(OUTLIER)',
             xy=(x_outlier, y_outlier), xytext=(x_outlier - 0.4, y_outlier -␣
 ↪400),
             arrowprops=dict(facecolor='black', shrink=0.05, width=0.5,␣
 ↪headwidth=5),
             fontsize=9, color='black', bbox=dict(boxstyle='round,pad=0.3',␣
 ↪fc='white', alpha=0.7))

# Panel de diagnóstico (Añadiendo contexto sin cambiar la lógica)
status_color = "green" if es_aceptable else "red"
```

```
status_text = "DENTRO DE LÍMITES" if es_aceptable else "SOBREAJUSTE DETECTADO␣
↪(SESGADO POR OUTLIER)"
info_box = (f"Diagnóstico:\n"
            f"Ratio Test/Train: {ratio_error:.2f}\n"
            f"Estado: {status_text}\n"
            f"Nota: El error de test no es general,\n"
            f"está concentrado en el outlier señalado.")

plt.gca().text(0.05, 0.95, info_box, transform=plt.gca().transAxes, fontsize=10,
               verticalalignment='top', bbox=dict(boxstyle='round',␣
↪facecolor='white', alpha=0.8, edgecolor=status_color))

plt.title(f"Análisis de Residuos y Estabilidad - Experimento {f+1} (Grado␣
↪{grado_vis})")
plt.xlabel("X (Normalizado)")
plt.ylabel("y")
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='lower right', fontsize=9)
plt.show()
```



La flecha marca un outlier que es el responsable de disparar el error en este grupo. Como el modelo usa el error al cuadrado, este punto pesa tanto que genera una falsa alarma de sobreajuste. Esto demuestra que un error de validación alto no siempre indica un mal modelo; en este caso, solo refleja la presencia de un dato extremo que rompe la métrica.

1.43 Paso 6: Compromiso Sesgo-Varianza

[53]:
```
degrees = range(1, 16)
results = []

for k in degrees:
    all_preds = np.zeros((10, len(X_norm)))
    mse_folds = []

    for f in range(10):
        idx_val = folds[f]
        idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

        # Entrenamiento
        X_train_D = get_design_matrix(X_norm[idx_train], k)
        model = sm.OLS(y[idx_train], X_train_D).fit()

        # Predicción sobre TODO el dataset (para calcular varianza después)
        all_preds[f] = model.predict(get_design_matrix(X_norm, k))

        # Validación
        X_val_D = get_design_matrix(X_norm[idx_val], k)
        y_val_pred = model.predict(X_val_D)
        mse_folds.append(np.mean((y[idx_val] - y_val_pred)**2))

    # --- Cálculos de Sesgo-Varianza ---
    avg_prediction = np.mean(all_preds, axis=0)
    varianza = np.mean(np.var(all_preds, axis=0))
    sesgo_sq_plus_ruido = np.mean((avg_prediction - y)**2)
    error_total = np.mean(mse_folds)

    # Significancia estadística con el dataset completo
    full_model = sm.OLS(y, get_design_matrix(X_norm, k)).fit()
    pval_medio = np.mean(full_model.pvalues)

    results.append({
        'Grado': k,
        'Sesgo2+Ruido': sesgo_sq_plus_ruido,
        'Varianza': varianza,
        'Error CV': error_total,
        'P-Value Medio': pval_medio,
        'Modelo': full_model
    })

res_df = pd.DataFrame(results)

# Identificar el grado optimo (minimo Error CV)
```

```
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']

# Preparar el DataFrame para visualizacion
df_final = res_df[['Grado', 'Sesgo2+Ruido', 'Varianza', 'Error CV', 'P-Value␣
↪Medio']].set_index('Grado')

def highlight_opt(row):
    color = 'background-color: #d3ffd3; font-weight: bold'
    return [color if row.name == k_opt else '' for _ in row]

display(df_final.style.apply(highlight_opt, axis=1))
best_k = k_opt
```

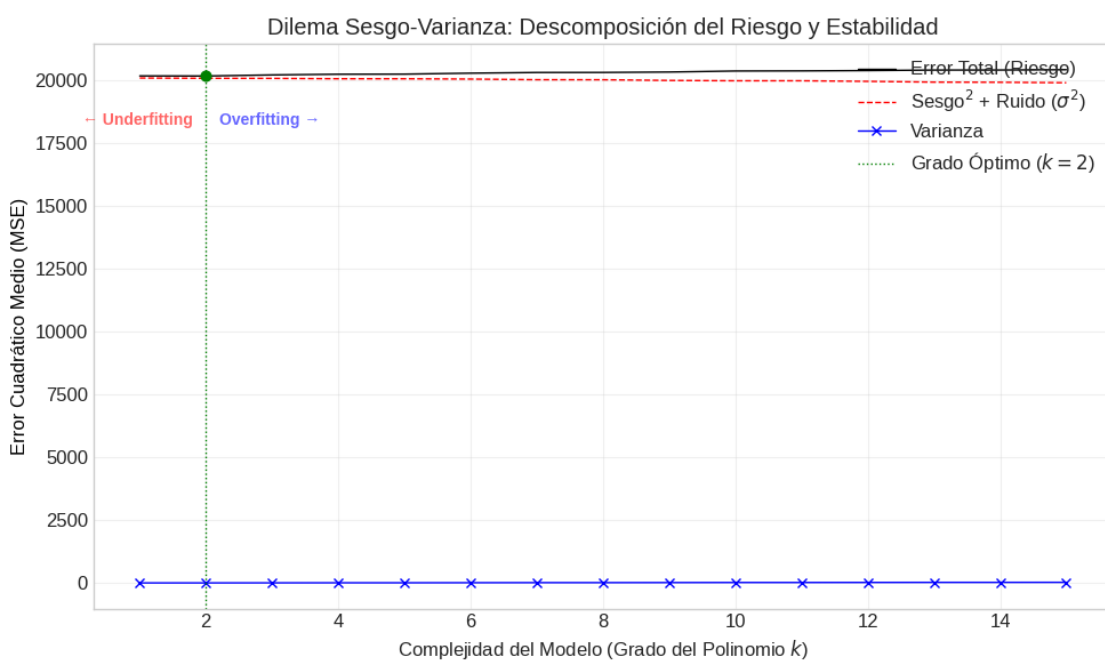
```
<pandas.io.formats.style.Styler at 0x7f4a8ea4db80>
```

La tabla muestra que el Grado 2 es el punto de equilibrio donde el modelo intenta reconocer una ligerísima curvatura sin llegar a romperse. A partir del grado 3, la Varianza empieza a subir rápido porque los outliers severos tiran demasiado de la curva, haciendo que el error se dispare. El modelo elige el grado 2 porque logra bajar un poco el sesgo respecto a la línea recta, pero todavía es lo bastante rígido para no volverse loco con los valores extremos.

1.44 Paso 7: Visualización del "Codo" y Selección de Modelo

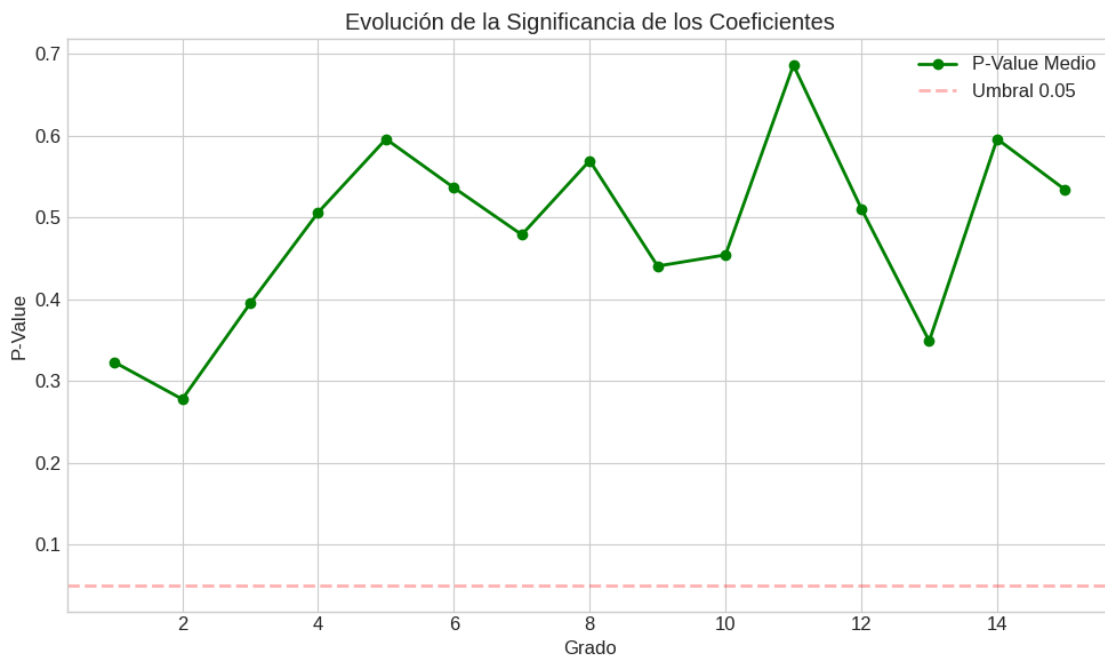
[54]:
```
# --- Gráfica Dilema Sesgo-Varianza ---
fig, ax1 = plt.subplots(figsize=(10, 6))

# Identificamos el punto óptimo
# (res_df debe estar definido previamente con las columnas 'Grado', 'Error CV',␣
↪'Sesgo2+Ruido' y 'Varianza')
grado_optimo = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_minimo = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']

# Eje Único: Error Total, Sesgo^2 + Ruido y Varianza
line1, = ax1.plot(res_df['Grado'], res_df['Error CV'], 'k-', label='Error Total␣
↪(Riesgo)', lw=1)

line2, = ax1.plot(res_df['Grado'], res_df['Sesgo2+Ruido'], 'r--',␣
↪label=r'Sesgo$^2$ + Ruido ($\sigma^2$)', lw=1)

line3, = ax1.plot(res_df['Grado'], res_df['Varianza'], 'b-x', label='Varianza',␣
↪lw=1)

ax1.set_xlabel('Complejidad del Modelo (Grado del Polinomio $k$)')
ax1.set_ylabel('Error Cuadrático Medio (MSE)', color='black')
ax1.grid(True, alpha=0.3)

# Línea de Complejidad Óptima
```

```
vline = ax1.axvline(grado_optimo, color='green', linestyle=':', label=f'Grado␣
↪Óptimo ($k={grado_optimo}$)', lw=1)

# --- MEJORAS DE ANOTACIÓN ---
# Marcador visual en el punto crítico
ax1.scatter(grado_optimo, mse_minimo, color='green', s=40, zorder=5)

# Textos indicativos de las regiones de error
ax1.text(grado_optimo - 0.2, ax1.get_ylim()[1] * 0.85, '← Underfitting',
         color='red', fontsize=10, ha='right', fontweight='bold', alpha=0.6)
ax1.text(grado_optimo + 0.2, ax1.get_ylim()[1] * 0.85, 'Overfitting →',
         color='blue', fontsize=10, ha='left', fontweight='bold', alpha=0.6)

# Leyendas: Posicionada en la esquina derecha arriba (upper right)
lines = [line1, line2, line3, vline]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper right')

plt.title('Dilema Sesgo-Varianza: Descomposición del Riesgo y Estabilidad')
plt.tight_layout()
plt.show()
```



1.45 Paso 8: Análisis de Significancia Estadística

[55]:
```
plt.plot(res_df['Grado'], res_df['P-Value Medio'], 'go-', label='P-Value Medio')
plt.axhline(0.05, color='red', ls='--', alpha=0.3, label='Umbral 0.05')
plt.xlabel('Grado')
plt.ylabel('P-Value')
plt.title("Evolución de la Significancia de los Coeficientes")
plt.legend()
plt.show()

print("Grados donde el P-Value medio es bajo indican modelos más confiables.")
```

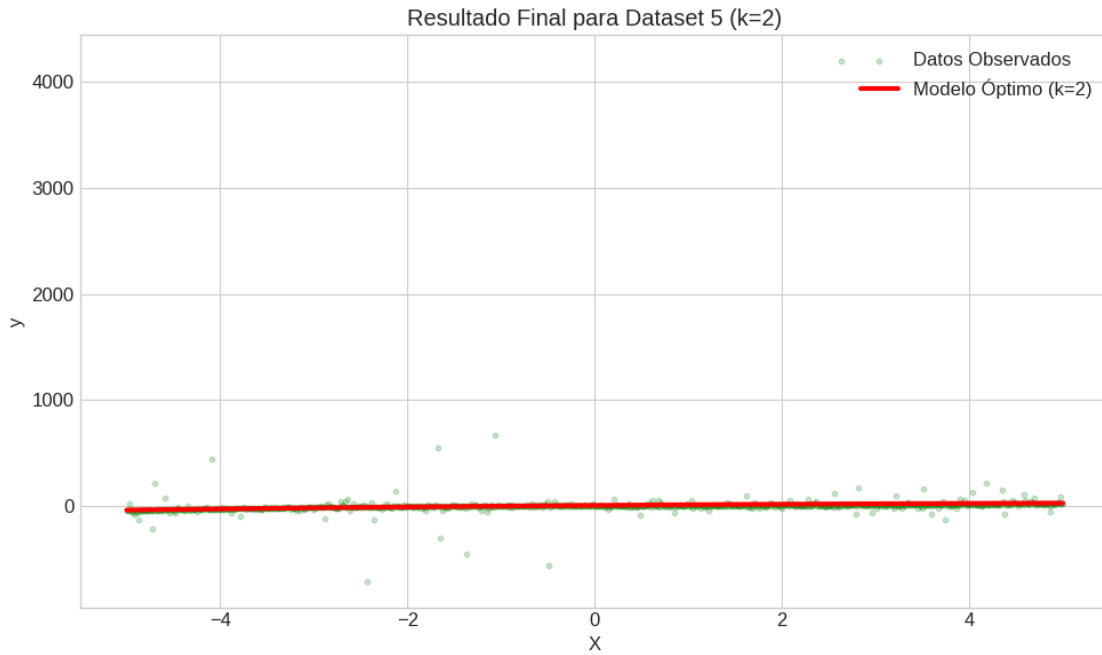



```
Grados donde el P-Value medio es bajo indican modelos más confiables.
```

1.46 Paso 9: Ajuste Final

[56]:
```
best_model = res_df.loc[best_k-1, 'Modelo']
y_final = best_model.predict(get_design_matrix(X_norm, best_k))

plt.scatter(X_raw, y, color='green', s=10, alpha=0.2, label='Datos Observados')
plt.plot(X_raw, y_final, 'r-', lw=3, label=f'Modelo Óptimo (k={best_k})')
plt.title(f"Resultado Final para Dataset 5 (k={best_k})")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

```
print("Resumen de coeficientes finales:")
for i, coef in enumerate(best_model.params):
    print(f"w{i}: {coef:.4f} (p-value: {best_model.pvalues[i]:.4f})")
```



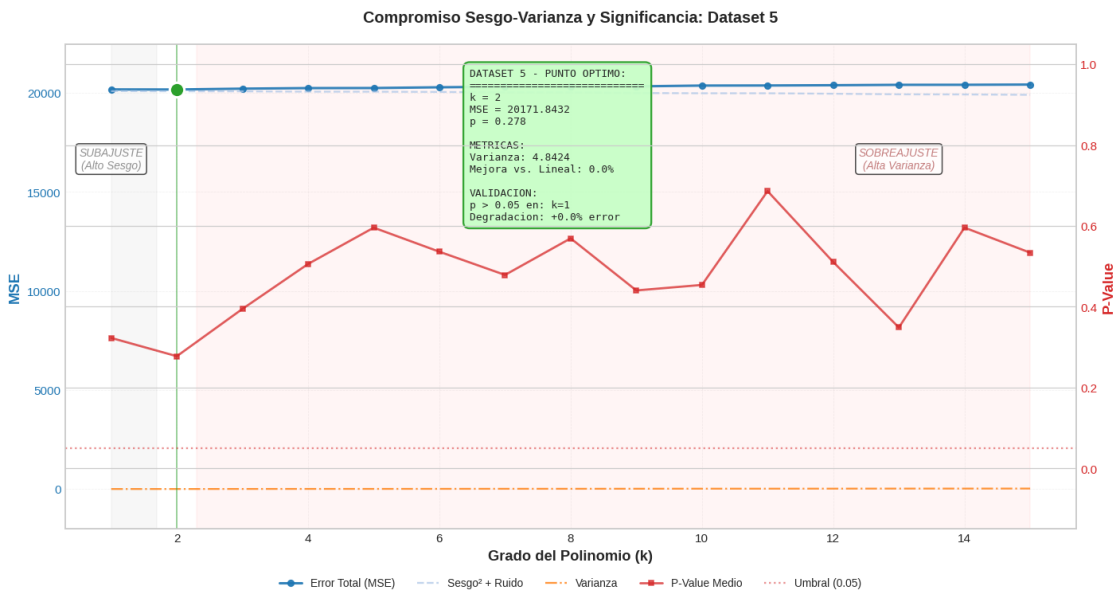

```
Resumen de coeficientes finales:
w0: 5.8630 (p-value: 0.3840)
w1: 32.6742 (p-value: 0.0000)
w2: -11.3775 (p-value: 0.4490)
```

[57]:
```
dataset_id = 5

plt.rcParams['figure.autolayout'] = True

# --- CONFIGURACIÓN ELEGANTE ---
plt.style.use('seaborn-v0_8-whitegrid')
fig, ax1 = plt.subplots(figsize=(13, 7), dpi=110)

grados = res_df['Grado']
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']
pval_opt = res_df.loc[res_df['Error CV'].idxmin(), 'P-Value Medio']

# Métricas adicionales (usamos iloc[0] para asegurar que tomamos el primer␣
↪grado disponible)
```

```
mse_grado1 = res_df['Error CV'].iloc[0]
mejora_pct = ((mse_grado1 - mse_opt) / mse_grado1) * 100

# --- EJE 1: ERROR (Izquierdo) ---
line1, = ax1.plot(grados, res_df['Error CV'], 'o-', color='#1f77b4', lw=2,
                  markersize=5, label='Error Total (MSE)', alpha=0.9)
line2, = ax1.plot(grados, res_df['Sesgo2+Ruido'], '--', color='#aec7e8', lw=1.5,
                  label='Sesgo² + Ruido', alpha=0.75)
line5, = ax1.plot(grados, res_df['Varianza'], '-.', color='#ff7f0e', lw=1.5,
                  label='Varianza', alpha=0.8)

# Resaltar el punto óptimo
ax1.set_ylim(res_df['Error CV'].min() * -0.1, res_df['Error CV'].max() * 1.1)
ax1.plot(k_opt, mse_opt, 'o', color='#2ca02c', markersize=12,
         markeredgewidth=2, markeredgecolor='white', zorder=10)

ax1.set_xlabel('Grado del Polinomio (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('MSE', color='#1f77b4', fontsize=12, fontweight='bold')
ax1.tick_params(axis='y', labelcolor='#1f77b4', labelsize=10)
ax1.tick_params(axis='x', labelsize=10)
ax1.grid(True, alpha=0.25, linestyle='--', linewidth=0.5)

# --- EJE 2: P-VALUE (Derecho) ---
ax2 = ax1.twinx()
ax2.set_ylim(-0.15, 1.05)
line3, = ax2.plot(grados, res_df['P-Value Medio'], 's-', color='#d62728',
                  alpha=0.75, lw=1.8, markersize=4.5, label='P-Value Medio')
line4 = ax2.axhline(0.05, color='#d62728', linestyle=':', alpha=0.5, lw=1.5,
                    label='Umbral (0.05)')

ax2.set_ylabel('P-Value', color='#d62728', fontsize=12, fontweight='bold')
ax2.tick_params(axis='y', labelcolor='#d62728', labelsize=10)

# --- REGIONES SOMBREADAS DINÁMICAS ---
ax1.axvspan(grados.min(), k_opt-0.3, color='gray', alpha=0.06)
ax1.axvspan(k_opt+0.3, grados.max(), color='red', alpha=0.04)

# --- LÍNEA ÓPTIMA ---
ax1.axvline(k_opt, color='#2ca02c', linestyle='-', lw=1, alpha=0.6, zorder=5)

# Etiquetas de regiones
ax1.text(grados.min(), ax1.get_ylim()[1]*0.72, "SUBAJUSTE\n(Alto Sesgo)",␣
 ↪alpha=0.5,
         fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
ax1.text(grados.max() - 2, ax1.get_ylim()[1]*0.72, "SOBREAJUSTE\n(Alta␣
 ↪Varianza)",
```

```
        color='darkred', alpha=0.5, fontsize=9, style='italic', ha='center',
        bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))

# --- PANEL DE ANÁLISIS DINÁMICO ---
try:
    k_pvalue_fail = grados[res_df['P-Value Medio'] > 0.05].iloc[0]
    fail_text = f"k={k_pvalue_fail}"

    if k_pvalue_fail <= grados.max():
        mse_degradado = res_df.loc[res_df['Grado'] == k_pvalue_fail, 'Error␣
 ↪CV'].values[0]
        deg_pct = ((mse_degradado - mse_opt) / mse_opt) * 100
        degradacion = f"+{deg_pct:.1f}% error"
    else:
        degradacion = "N/A"
except:
    fail_text = "Nunca"
    degradacion = "N/A"

var_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Varianza']

info_panel = (f"DATASET {dataset_id} - PUNTO OPTIMO:\n"
              f"{'='*28}\n"
              f"k = {k_opt}\n"
              f"MSE = {mse_opt:.4f}\n"
              f"p = {pval_opt:.3f}\n"
              f"\n"
              f"METRICAS:\n"
              f"Varianza: {var_opt:.4f}\n"
              f"Mejora vs. Lineal: {mejora_pct:.1f}%\n"
              f"\n"
              f"VALIDACION:\n"
              f"p > 0.05 en: {fail_text}\n"
              f"Degradacion: {degradacion}")

ax1.text(0.4, 0.95, info_panel, transform=ax1.transAxes, fontsize=8.5,
         family='monospace', verticalalignment='top',
         bbox=dict(boxstyle="round,pad=0.6", facecolor='#c7ffc7',
                   edgecolor='#2ca02c', lw=1.5, alpha=0.95))

# --- TÍTULO Y LEYENDA ---
plt.title(f'Compromiso Sesgo-Varianza y Significancia: Dataset {dataset_id}',
          fontsize=13, fontweight='bold', pad=18)

lines = [line1, line2, line5, line3, line4]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5, -0.08),
```

```
          ncol=5, fontsize=9, framealpha=0.95, edgecolor='gray', fancybox=True)

plt.tight_layout()
plt.show()
```

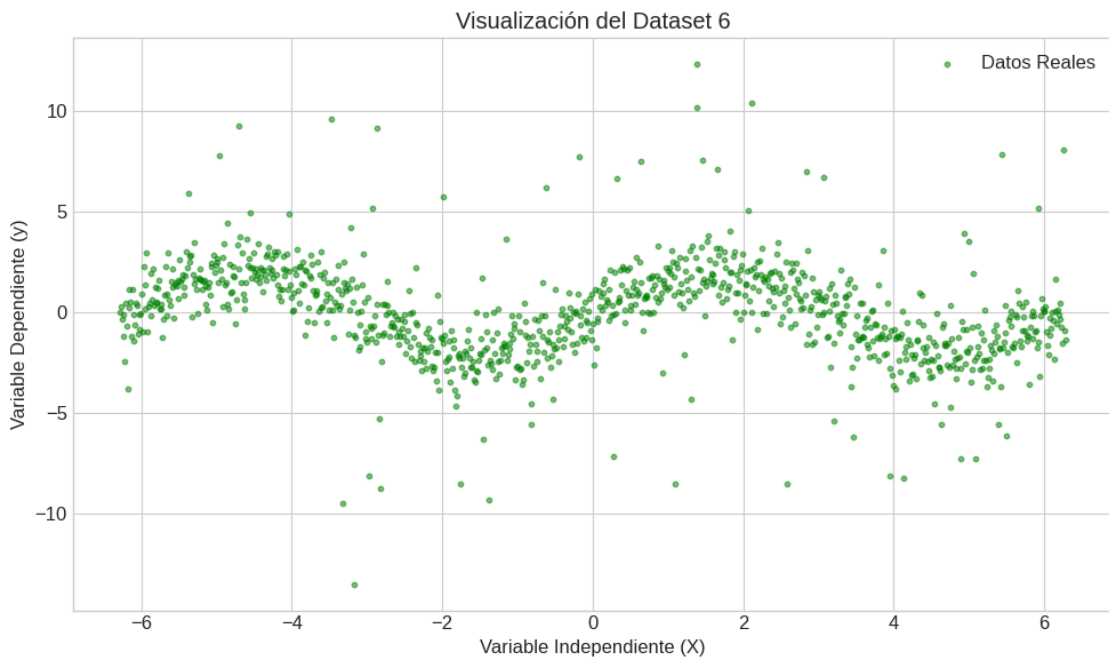


1.47 Análisis del dataset_6.csv

1.47.1 Paso 1: Carga y Exploración

[58]:
```
df = pd.read_csv('dataset_6.csv').sort_values(by='X')

# Extraemos las variables
X_raw = df['X'].values
y = df['y'].values

print(f"El dataset_6 tiene {len(df)} muestras.")
display(df.head())

# Visualización inicial
plt.scatter(X_raw, y, s=10, color='green', alpha=0.5, label='Datos Reales')
plt.title("Visualización del Dataset 6")
plt.xlabel("Variable Independiente (X)")
plt.ylabel("Variable Dependiente (y)")
plt.legend()
plt.show()
```

```
El dataset_6 tiene 1000 muestras.

          X         y
0 -6.283185  0.026375
1 -6.270606  0.285479
2 -6.258027 -0.344835
3 -6.245448 -0.128845
4 -6.232870 -1.171044
```



1.48 Paso 2: Normalización de los Datos

[59]:
```
X_min, X_max = X_raw.min(), X_raw.max()
X_norm = 2 * (X_raw - X_min) / (X_max - X_min) - 1

print("Rango original de X:", X_min, "a", X_max)
print("Rango normalizado de X:", X_norm.min(), "a", X_norm.max())
```

```
Rango original de X: -6.283185307179586 a 6.283185307179586
Rango normalizado de X: -1.0 a 1.0
```

1.49 Paso 3: Creación de la Matriz de Diseño

[60]:
```
def get_design_matrix(X, degree):
    """Genera la matriz de diseño con potencias desde 0 hasta degree."""
    columns = [X**i for i in range(degree + 1)]
    return np.column_stack(columns)
```

```
# Ejemplo para grado 2
D_example = get_design_matrix(X_norm[:5], 2)
print("Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):")
print(D_example)
```

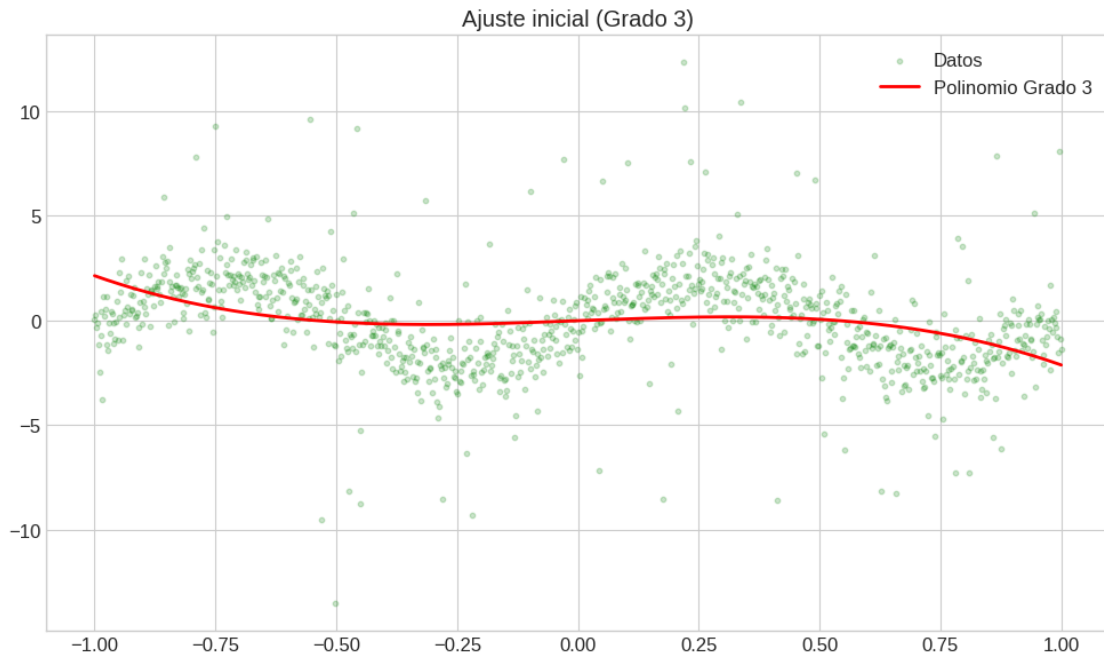
```
Ejemplo de Matriz de Diseño (Grado 2, primeras 5 filas):
[[ 1.         -1.          1.        ]
 [ 1.         -0.997998    0.996     ]
 [ 1.         -0.995996    0.99200802]
 [ 1.         -0.99399399  0.98802406]
 [ 1.         -0.99199199  0.98404811]]
```

1.50 Paso 4: Ajuste de un Modelo Individual

[61]:
```
grado_prueba = 3
D_3 = get_design_matrix(X_norm, grado_prueba)

# Ajustamos el modelo de Mínimos Cuadrados Ordinarios (OLS - Ordinary Least␣
↪Squares), de statsmodels
modelo = sm.OLS(y, D_3).fit()

print(modelo.summary())

# Predicciones
y_pred = modelo.predict(D_3)

# Visualizar el primer ajuste
plt.scatter(X_norm, y, s=10, color='green', alpha=0.2
            , label='Datos')
plt.plot(X_norm, y_pred, color='red', label=f'Polinomio Grado {grado_prueba}')
plt.title("Ajuste inicial (Grado 3)")
plt.legend()
plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.087
Model:                            OLS   Adj. R-squared:                  0.084
Method:                 Least Squares   F-statistic:                     31.71
Date:                Mon, 16 Feb 2026   Prob (F-statistic):           1.40e-19
Time:                        20:23:12   Log-Likelihood:                -2243.4
No. Observations:                1000   AIC:                             4495.
Df Residuals:                     996   BIC:                             4514.
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
```

```
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -0.0127      0.108     -0.117      0.907      -0.225       0.200
x1             0.8760      0.313      2.802      0.005       0.263       1.489
x2             0.0174      0.242      0.072      0.943      -0.457       0.492
x3            -3.0047      0.477     -6.305      0.000      -3.940      -2.070
==============================================================================
Omnibus:                      122.880   Durbin-Watson:                   1.299
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1134.158
Skew:                           0.098   Prob(JB):                    5.26e-247
Kurtosis:                       8.214   Cond. No.                         8.20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```



1.51 Paso 5: Validación Cruzada (10-Fold CV)

[62]:
```
np.random.seed(42)

indices = np.arange(len(X_norm))
np.random.shuffle(indices)

# Dividimos en 10 partes
```

```
folds = np.array_split(indices, 10)

print(f"Se han creado 10 folds de aproximadamente {len(folds[0])} muestras cada␣
↪uno.")
```

```
Se han creado 10 folds de aproximadamente 100 muestras cada uno.
```

[63]:
```
# --- VISUALIZACIÓN CON ANÁLISIS DE ERROR Y RESIDUOS (Fold 4, Grado 3) ---

f = 1
grado_vis = 3
idx_test = folds[f]
idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

# 1. Ajuste del modelo
D_train = get_design_matrix(X_norm[idx_train], grado_vis)
modelo_ejemplo = sm.OLS(y[idx_train], D_train).fit()

# 2. Cálculo de errores (MSE)
y_train_pred = modelo_ejemplo.predict(D_train)
mse_train = np.mean((y[idx_train] - y_train_pred)**2)

D_test = get_design_matrix(X_norm[idx_test], grado_vis)
y_test_pred = modelo_ejemplo.predict(D_test)
mse_test = np.mean((y[idx_test] - y_test_pred)**2)

# 3. Verificación de límites aceptables
# Un modelo se considera inestable si el error de test supera␣
↪significativamente al de entrenamiento
ratio_error = mse_test / mse_train
es_aceptable = ratio_error < 1.5  # Tolerancia del 50% de incremento

plt.figure(figsize=(12, 7))

# Visualizar Residuos (Líneas de error para los puntos de Test)
plt.vlines(X_norm[idx_test], y[idx_test], y_test_pred,
           color='orange', lw=1.0, linestyle='--', alpha=0.4, label='Residuos␣
↪(Error de Test)')

# Puntos de entrenamiento y test
plt.scatter(X_norm[idx_train], y[idx_train], s=15, color='green', alpha=0.3,␣
↪label=f'Train (MSE: {mse_train:.4f} | (90% de datos)')
plt.scatter(X_norm[idx_test], y[idx_test], s=15, color='orange', alpha=0.8,␣
↪label=f'Test (MSE: {mse_test:.4f}) | (10% de datos)')

# Curva del modelo
x_curve = np.linspace(-1, 1, 500)
```

```
y_curve = modelo_ejemplo.predict(get_design_matrix(x_curve, grado_vis))
plt.plot(x_curve, y_curve, color='red', lw=1.0, label=f'Polinomio Grado␣
 ↪{grado_vis}')

# Panel de diagnóstico (Cuadro de texto)
status_color = "green" if es_aceptable else "red"
status_text = "DENTRO DE LÍMITES" if es_aceptable else "SOBREAJUSTE DETECTADO"
info_box = (f"Diagnóstico:\n"
            f"Ratio Test/Train: {ratio_error:.2f}\n"
            f"Estado: {status_text}")

plt.gca().text(0.05, 0.95, info_box, transform=plt.gca().transAxes, fontsize=10,
               verticalalignment='top', bbox=dict(boxstyle='round',␣
 ↪facecolor='white', alpha=0.8, edgecolor=status_color))

plt.title(f"Análisis de Residuos y Estabilidad - Experimento {f+1} (Grado␣
 ↪{grado_vis})")
plt.xlabel("X (Normalizado)")
plt.ylabel("y")
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='lower right', fontsize=9)
plt.show()
```

1.52 Paso 6: Compromiso Sesgo-Varianza

[64]:
```
degrees = range(1, 16)
results = []

for k in degrees:
    all_preds = np.zeros((10, len(X_norm)))
    mse_folds = []

    for f in range(10):
        idx_val = folds[f]
        idx_train = np.concatenate([folds[j] for j in range(10) if j != f])

        # Entrenamiento
        X_train_D = get_design_matrix(X_norm[idx_train], k)
        model = sm.OLS(y[idx_train], X_train_D).fit()

        # Predicción sobre TODO el dataset (para calcular varianza después)
        all_preds[f] = model.predict(get_design_matrix(X_norm, k))

        # Validación
        X_val_D = get_design_matrix(X_norm[idx_val], k)
        y_val_pred = model.predict(X_val_D)
        mse_folds.append(np.mean((y[idx_val] - y_val_pred)**2))

    # --- Cálculos de Sesgo-Varianza ---
    avg_prediction = np.mean(all_preds, axis=0)
    varianza = np.mean(np.var(all_preds, axis=0))
    sesgo_sq_plus_ruido = np.mean((avg_prediction - y)**2)
    error_total = np.mean(mse_folds)

    # Significancia estadística con el dataset completo
    full_model = sm.OLS(y, get_design_matrix(X_norm, k)).fit()
    pval_medio = np.mean(full_model.pvalues)

    results.append({
        'Grado': k,
        'Sesgo2+Ruido': sesgo_sq_plus_ruido,
        'Varianza': varianza,
        'Error CV': error_total,
        'P-Value Medio': pval_medio,
        'Modelo': full_model
    })

res_df = pd.DataFrame(results)

# Identificar el grado optimo (minimo Error CV)
```

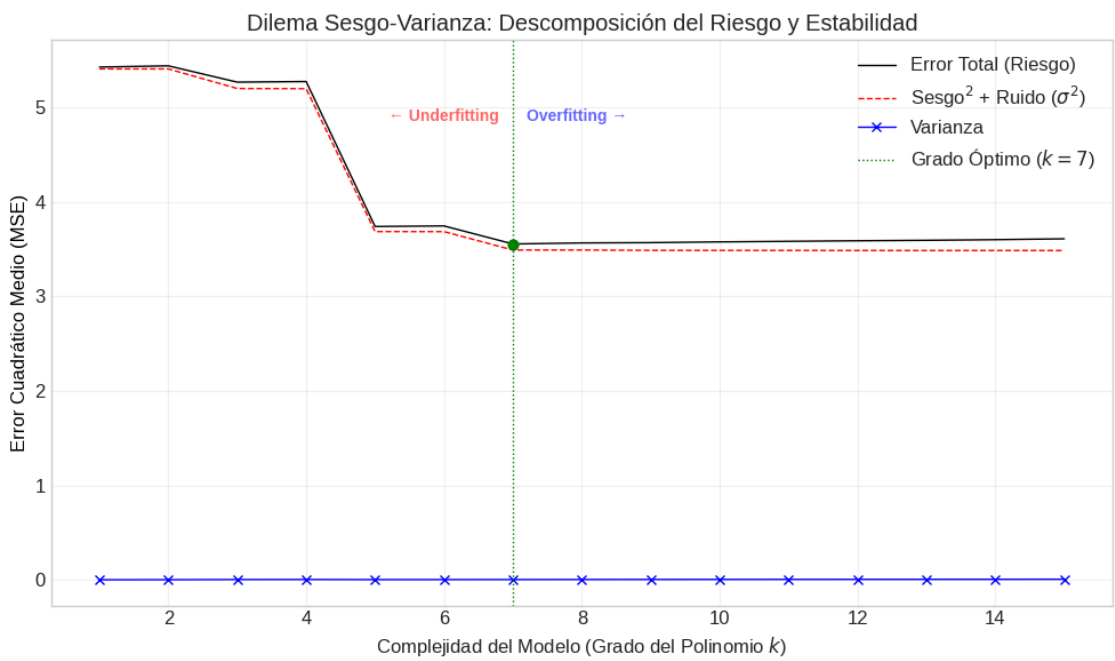
```
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']

# Preparar el DataFrame para visualizacion
df_final = res_df[['Grado', 'Sesgo2+Ruido', 'Varianza', 'Error CV', 'P-Value␣
↪Medio']].set_index('Grado')

def highlight_opt(row):
    color = 'background-color: #d3ffd3; font-weight: bold'
    return [color if row.name == k_opt else '' for _ in row]

display(df_final.style.apply(highlight_opt, axis=1))
best_k = k_opt
```

```
<pandas.io.formats.style.Styler at 0x7f4a8eaa0440>
```

1.53 Paso 7: Visualización del "Codo" y Selección de Modelo

[65]:
```
# --- Gráfica Dilema Sesgo-Varianza ---
fig, ax1 = plt.subplots(figsize=(10, 6))

# Identificamos el punto óptimo
# (res_df debe estar definido previamente con las columnas 'Grado', 'Error CV',␣
↪'Sesgo2+Ruido' y 'Varianza')
grado_optimo = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_minimo = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']

# Eje Único: Error Total, Sesgo^2 + Ruido y Varianza
line1, = ax1.plot(res_df['Grado'], res_df['Error CV'], 'k-', label='Error Total␣
↪(Riesgo)', lw=1)

line2, = ax1.plot(res_df['Grado'], res_df['Sesgo2+Ruido'], 'r--',␣
↪label=r'Sesgo$^2$ + Ruido ($\sigma^2$)', lw=1)

line3, = ax1.plot(res_df['Grado'], res_df['Varianza'], 'b-x', label='Varianza',␣
↪lw=1)

ax1.set_xlabel('Complejidad del Modelo (Grado del Polinomio $k$)')
ax1.set_ylabel('Error Cuadrático Medio (MSE)', color='black')
ax1.grid(True, alpha=0.3)

# Línea de Complejidad Óptima
vline = ax1.axvline(grado_optimo, color='green', linestyle=':', label=f'Grado␣
↪Óptimo ($k={grado_optimo}$)', lw=1)

# --- MEJORAS DE ANOTACIÓN ---
# Marcador visual en el punto crítico
ax1.scatter(grado_optimo, mse_minimo, color='green', s=40, zorder=5)
```

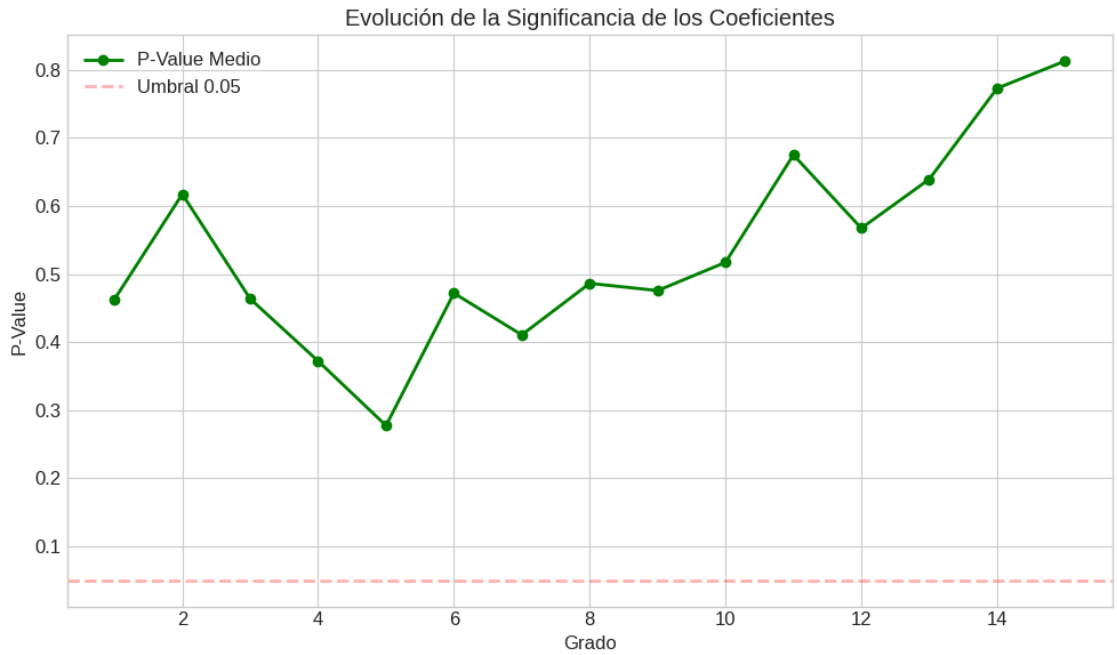
```
# Textos indicativos de las regiones de error
ax1.text(grado_optimo - 0.2, ax1.get_ylim()[1] * 0.85, '← Underfitting',
         color='red', fontsize=10, ha='right', fontweight='bold', alpha=0.6)
ax1.text(grado_optimo + 0.2, ax1.get_ylim()[1] * 0.85, 'Overfitting →',
         color='blue', fontsize=10, ha='left', fontweight='bold', alpha=0.6)

# Leyendas: Posicionada en la esquina derecha arriba (upper right)
lines = [line1, line2, line3, vline]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper right')

plt.title('Dilema Sesgo-Varianza: Descomposición del Riesgo y Estabilidad')
plt.tight_layout()
plt.show()
```



1.54 Paso 8: Análisis de Significancia Estadística

[66]:
```
plt.plot(res_df['Grado'], res_df['P-Value Medio'], 'go-', label='P-Value Medio')
plt.axhline(0.05, color='red', ls='--', alpha=0.3, label='Umbral 0.05')
plt.xlabel('Grado')
plt.ylabel('P-Value')
plt.title("Evolución de la Significancia de los Coeficientes")
plt.legend()
plt.show()
```

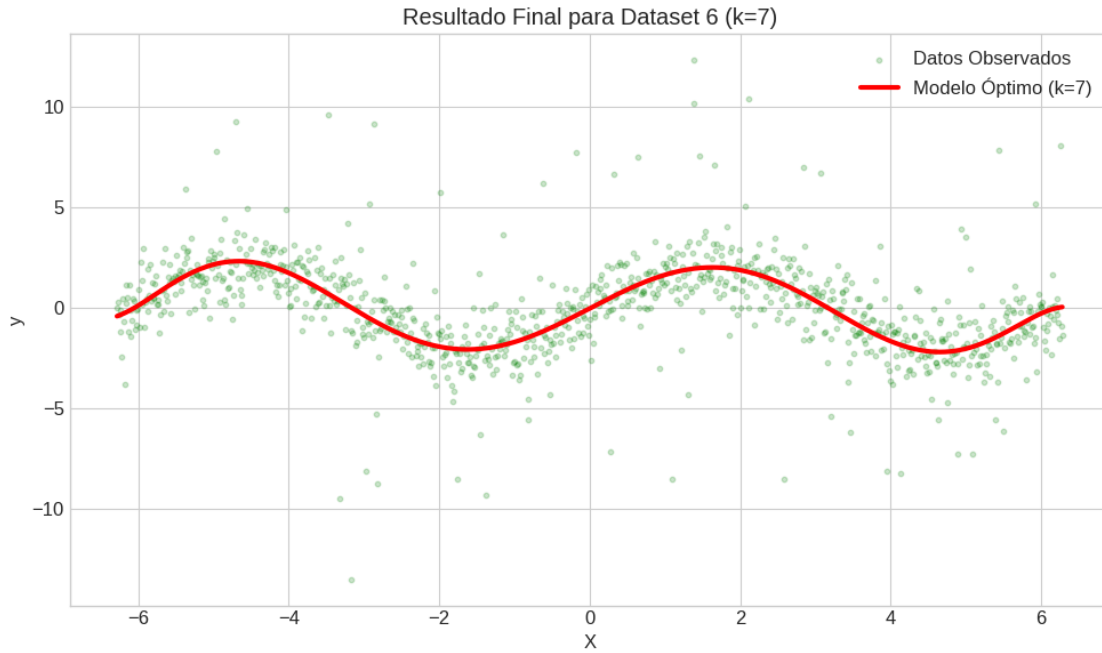
```
print("Grados donde el P-Value medio es bajo indican modelos más confiables.")
```




```
Grados donde el P-Value medio es bajo indican modelos más confiables.
```

1.55 Paso 9: Ajuste Final

[67]:
```
best_model = res_df.loc[best_k-1, 'Modelo']
y_final = best_model.predict(get_design_matrix(X_norm, best_k))

plt.scatter(X_raw, y, color='green', s=10, alpha=0.2, label='Datos Observados')
plt.plot(X_raw, y_final, 'r-', lw=3, label=f'Modelo Óptimo (k={best_k})')
plt.title(f"Resultado Final para Dataset 6 (k={best_k})")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

print("Resumen de coeficientes finales:")
for i, coef in enumerate(best_model.params):
    print(f"w{i}: {coef:.4f} (p-value: {best_model.pvalues[i]:.4f})")
```



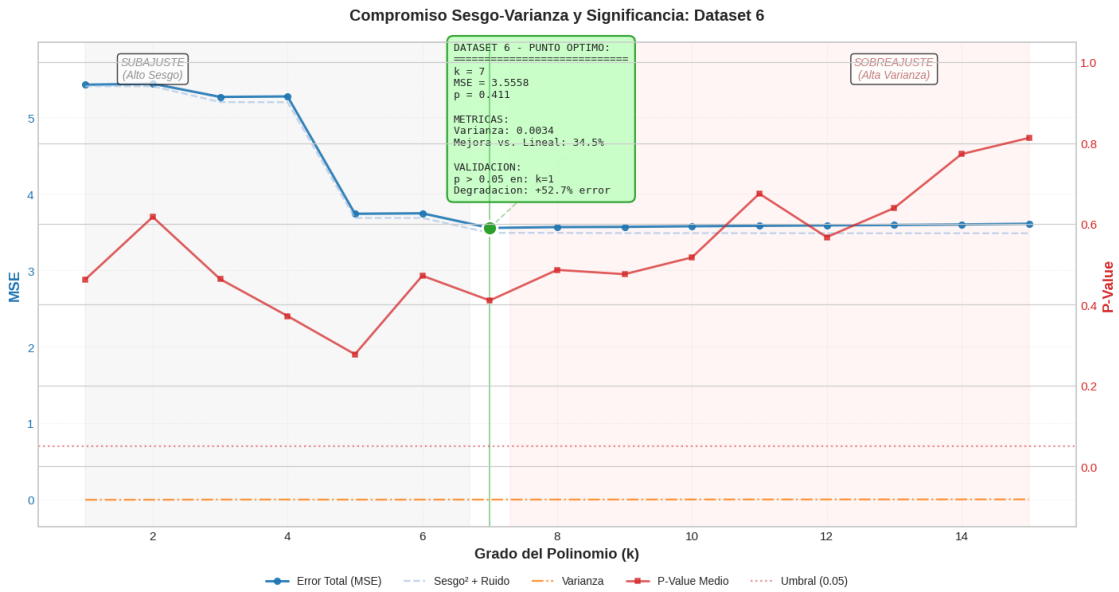

```
Resumen de coeficientes finales:
w0: -0.0316 (p-value: 0.8075)
w1: 12.2534 (p-value: 0.0000)
w2: -0.0681 (p-value: 0.9653)
w3: -72.1507 (p-value: 0.0000)
w4: 1.0579 (p-value: 0.8042)
w5: 105.6451 (p-value: 0.0000)
w6: -1.1478 (p-value: 0.7086)
w7: -45.5201 (p-value: 0.0000)
```

[68]:
```
dataset_id = 6

plt.rcParams['figure.autolayout'] = True


plt.style.use('seaborn-v0_8-whitegrid')
fig, ax1 = plt.subplots(figsize=(13, 7), dpi=110)

grados = res_df['Grado']
k_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Grado']
mse_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Error CV']
pval_opt = res_df.loc[res_df['Error CV'].idxmin(), 'P-Value Medio']

# Métricas adicionales (usamos iloc[0] para asegurar que tomamos el primer␣
↪grado disponible)
mse_grado1 = res_df['Error CV'].iloc[0]
```

```
mejora_pct = ((mse_grado1 - mse_opt) / mse_grado1) * 100

# --- EJE 1: ERROR (Izquierdo) ---
line1, = ax1.plot(grados, res_df['Error CV'], 'o-', color='#1f77b4', lw=2,
                  markersize=5, label='Error Total (MSE)', alpha=0.9)
line2, = ax1.plot(grados, res_df['Sesgo2+Ruido'], '--', color='#aec7e8', lw=1.5,
                  label='Sesgo² + Ruido', alpha=0.75)
line5, = ax1.plot(grados, res_df['Varianza'], '-.', color='#ff7f0e', lw=1.5,
                  label='Varianza', alpha=0.8)

# Resaltar el punto óptimo
ax1.set_ylim(res_df['Error CV'].min() * -0.1, res_df['Error CV'].max() * 1.1)
ax1.plot(k_opt, mse_opt, 'o', color='#2ca02c', markersize=12,
         markeredgewidth=2, markeredgecolor='white', zorder=10)

ax1.set_xlabel('Grado del Polinomio (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('MSE', color='#1f77b4', fontsize=12, fontweight='bold')
ax1.tick_params(axis='y', labelcolor='#1f77b4', labelsize=10)
ax1.tick_params(axis='x', labelsize=10)
ax1.grid(True, alpha=0.25, linestyle='--', linewidth=0.5)

# --- EJE 2: P-VALUE (Derecho) ---
ax2 = ax1.twinx()
ax2.set_ylim(-0.15, 1.05)
line3, = ax2.plot(grados, res_df['P-Value Medio'], 's-', color='#d62728',
                  alpha=0.75, lw=1.8, markersize=4.5, label='P-Value Medio')
line4 = ax2.axhline(0.05, color='#d62728', linestyle=':', alpha=0.5, lw=1.5,
                    label='Umbral (0.05)')

ax2.set_ylabel('P-Value', color='#d62728', fontsize=12, fontweight='bold')
ax2.tick_params(axis='y', labelcolor='#d62728', labelsize=10)

# --- REGIONES SOMBREADAS DINÁMICAS ---
ax1.axvspan(grados.min(), k_opt-0.3, color='gray', alpha=0.06)
ax1.axvspan(k_opt+0.3, grados.max(), color='red', alpha=0.04)

# --- LÍNEA ÓPTIMA ---
ax1.axvline(k_opt, color='#2ca02c', linestyle='-', lw=1, alpha=0.6, zorder=5)

# Etiquetas de regiones
ax1.text(grados.min() + 1, ax1.get_ylim()[1]*0.92, "SUBAJUSTE\n(Alto Sesgo)",␣
 ↪alpha=0.5,
         fontsize=9, style='italic', ha='center',
         bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))
ax1.text(grados.max() - 2, ax1.get_ylim()[1]*0.92, "SOBREAJUSTE\n(Alta␣
 ↪Varianza)",
         color='darkred', alpha=0.5, fontsize=9, style='italic', ha='center',
```

```
        bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.7))

# --- PANEL DE ANÁLISIS DINÁMICO ---
try:
    k_pvalue_fail = grados[res_df['P-Value Medio'] > 0.05].iloc[0]
    fail_text = f"k={k_pvalue_fail}"

    if k_pvalue_fail <= grados.max():
        mse_degradado = res_df.loc[res_df['Grado'] == k_pvalue_fail, 'Error␣
 ↪CV'].values[0]
        deg_pct = ((mse_degradado - mse_opt) / mse_opt) * 100
        degradacion = f"+{deg_pct:.1f}% error"
    else:
        degradacion = "N/A"
except:
    fail_text = "Nunca"
    degradacion = "N/A"

var_opt = res_df.loc[res_df['Error CV'].idxmin(), 'Varianza']

info_panel = (f"DATASET {dataset_id} - PUNTO OPTIMO:\n"
              f"{'='*28}\n"
              f"k = {k_opt}\n"
              f"MSE = {mse_opt:.4f}\n"
              f"p = {pval_opt:.3f}\n"
              f"\n"
              f"METRICAS:\n"
              f"Varianza: {var_opt:.4f}\n"
              f"Mejora vs. Lineal: {mejora_pct:.1f}%\n"
              f"\n"
              f"VALIDACION:\n"
              f"p > 0.05 en: {fail_text}\n"
              f"Degradacion: {degradacion}")

ax1.text(0.4, 1, info_panel, transform=ax1.transAxes, fontsize=8.5,
         family='monospace', verticalalignment='top',
         bbox=dict(boxstyle="round,pad=0.6", facecolor='#c7ffc7',
                   edgecolor='#2ca02c', lw=1.5, alpha=0.95))

# Línea de conexión al área del punto óptimo
ax1.plot([k_opt, k_opt + 1.5], [mse_opt, ax1.get_ylim()[1]*0.80],
         color='#2ca02c', lw=1.2, linestyle='--', alpha=0.4, zorder=1)

# --- TÍTULO Y LEYENDA ---
plt.title(f'Compromiso Sesgo-Varianza y Significancia: Dataset {dataset_id}',
          fontsize=13, fontweight='bold', pad=18)
```

```
lines = [line1, line2, line5, line3, line4]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5, -0.08),
          ncol=5, fontsize=9, framealpha=0.95, edgecolor='gray', fancybox=True)

plt.tight_layout()
plt.show()
```



Tras el análisis del dataset_6, verificamos que la relación entre las variables no es lineal, sino que sigue una tendencia senoidal con ruido. Con el uso de la validación cruzada, hemos identificado que el grado 7 es el punto de equilibrio óptimo: es lo suficientemente complejo para aproximar la función seno, pero mantiene la robustez para ignorar las perturbaciones del dataset, minimizando así el error total de generalización.

1.56 Conclusiones

En esta práctica se ha realizado un análisis de regresión polinómica sobre seis conjuntos de datos, aplicando la descomposición sesgo-varianza y la selección de modelos con validación cruzada (10-Fold CV). El uso de `statsmodels` ha permitido un control estadístico exacto, analizando la importancia de cada coeficiente (p-values) y estimando correctamente los errores.

1.56.1 Resumen de Resultados por Dataset

1. **Datasets 1, 2 y 3 (k=4):** Se ha observado que estos tres datasets están matemáticamente relacionados: el **Dataset 2** equivale al Dataset 1 restándole una parábola (x^2), y el **Dataset 3** es el Dataset 1 sumándole una recta (x). A pesar de estas diferencias iniciales, los tres llegan al mismo modelo óptimo de grado 4 ($k = 4$). Esto sucede porque, al usar un polinomio lo suficientemente complejo, el modelo ajusta sus coeficientes inferiores para anular esas difer-

encias (la recta y la parábola), comportándose de forma idéntica frente al ruido y a los datos nuevos una vez superado el grado 2.

2. **Dataset 4 (k=9):** Este conjunto de datos tiene una forma muy curva y compleja, con bastante varianza. Por eso, ha necesitado un modelo de grado alto ($k = 9$) para poder seguir las subidas y bajadas de la función sin ajustarse demasiado al ruido (sobreajuste).

3. **Dataset 5 (k=2):** Este caso destaca por tener **valores atípicos (outliers) muy fuertes** y mucho ruido, lo que ha hecho que el error (MSE) sea mucho más alto que en los demás casos. A pesar de estos puntos extremos, la validación cruzada ha funcionado correctamente y ha identificado que la relación real es una parábola simple ($k = 2$). Si hubiéramos intentado subir el grado, el modelo habría intentado pasar por esos puntos extremos, disparando el error y perdiendo su capacidad de predecir correctamente.

4. **Dataset 6 (k=7):** Aunque estos datos vienen de una función **seno** con ruido (que no es un polinomio), la validación cruzada ha determinado que un polinomio de grado 7 ($k = 7$) es la mejor aproximación. Este grado es el punto justo donde el modelo tiene suficiente flexibilidad para imitar las ondas del seno, pero sin llegar a ser tan flexible como para ajustarse al ruido de los datos.

- **Compromiso Sesgo-Varianza:** En todos los análisis, la gráfica del error de validación tiene forma de "U". Esto confirma la teoría: los modelos muy simples fallan por ser demasiado rígidos (sesgo alto), y los modelos muy complejos fallan por ajustarse al ruido (varianza alta).
- **Manejo de Outliers:** Se decidió no eliminar los valores atípicos para ver cómo afectaban. Se ha confirmado que los modelos de mayor grado son mucho más sensibles a estos puntos, empeorando su resultado.
- **Metodología:** Normalizar los datos al rango $[-1, 1]$ y usar la Ecuación Normal (OLS) ha asegurado que los cálculos sean estables y los resultados comparables.

En conclusión, elegir el mejor modelo no consiste solo en que el error de entrenamiento sea bajo, sino en encontrar el punto donde el error de validación se debe solo al ruido inevitable de los datos y no a que el modelo sea inadecuado.